

UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL MENDOZA
TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN
TRABAJO FINAL DE TECNICATURA
Escaneo Automatizado de Vulnerabilidades y Generación de Informes en Entornos de Red

Autores:

Lautaro Castillo

Bruno Racconto

Directores de Trabajo Final:

Profesor Alberto Cortez

Profesor Ariel Enferrel

Mendoza, Argentina

Año 2026

Declaración de Originalidad

Declaramos que el presente Trabajo Final de Tecnicatura es de nuestra autoría, que fue elaborado sin recurrir a fuentes distintas de las citadas explícitamente en la Sección 19, y que todo material tomado de terceros (textos, datos, figuras o fragmentos de código) se encuentra debidamente referenciado. El sistema descrito, su código fuente y los datos de medición reportados fueron producidos por los autores en el marco de este trabajo, y se encuentran disponibles para su verificación en el repositorio declarado en la Sección 20.

Asumimos la responsabilidad por el contenido de este documento.

Lautaro Castillo

Bruno Racconto

Mendoza, 28 de septiembre de 2026

Resumen

El análisis manual de vulnerabilidades exige intervención humana en cada transición entre herramientas: ejecutar cada escáner, interpretar salidas incompatibles, unificar severidades dispares y redactar el informe. Ese costo lo vuelve difícil de sostener en laboratorios académicos sin acceso a plataformas comerciales.

Este trabajo presenta VulnScan Platform, construida con herramientas de código abierto sobre microservicios contenerizados: cinco herramientas de escaneo (Nmap, WhatWeb, Nikto, Nuclei y OWASP ZAP) coordinadas por un pipeline en n8n sobre tres servicios propios en Python. Los hallazgos se normalizan, se clasifican por severidad y se almacenan en PostgreSQL. El despliegue requiere un único comando.

La evaluación se realizó sobre un laboratorio contenerizado con dos aplicaciones deliberadamente vulnerables, OWASP Juice Shop y DVWA, en tres campañas de cinco ejecuciones. La descomposición del tiempo muestra que, en la campaña A, la coordinación entre etapas consume 0,85 s frente a 259,06 s de pipeline total, es decir el 0,3 %: el resto es ejecución de herramientas, un piso común a cualquier operador. La cobertura se midió contra catálogos públicos de verdad de referencia. El emparejamiento automático no produjo correspondencias de alta confianza; la anotación manual establece un recall de 0,27 sobre DVWA autenticado, condición opcional no predeterminada. Lo detectado no es aleatorio: el pipeline encuentra lo observable en la respuesta del servidor y no lo que exige completar un flujo de la aplicación. La descomposición por herramienta arroja tres resultados no anticipados: en la campaña A, contra DVWA, el escaneo activo de ZAP concentra el 96,1 % de la varianza temporal del pipeline — participación que en la campaña C, sobre ese mismo objetivo, corresponde a Nuclei—, ZAP es la única herramienta cuya precisión resiste el cambio de objetivo, y el 97,4 % de las severidades reportadas las asigna la política de normalización y no la herramienta que detectó el hallazgo. Una réplica sobre un segundo objetivo y un segundo anfitrión refuta la hipótesis de consistencia entre ejecuciones y expone que el sistema no detecta el fallo parcial de una herramienta.

El brazo manual, ejecutado en dos corridas separadas, una por cada autor (Sección 12.2), insumió 1 h 31 min de trabajo atendido de media frente a los 0,24 s de orquestación que el pipeline registra en la campaña C, la condición equivalente, con el sesgo de familiaridad que la Sección 9.9 declara.

Palabras clave: *escaneo de vulnerabilidades, orquestación de seguridad, microservicios, normalización de hallazgos, OWASP ZAP, automatización.*

Abstract

Manual vulnerability analysis requires human intervention at every transition between tools: running each scanner, interpreting incompatible outputs, reconciling disparate severity criteria, and writing the report. That cost makes the practice hard to sustain, particularly for academic laboratories without access to commercial platforms.

This thesis presents VulnScan Platform, built with open-source tools on containerized microservices: five scanning tools (Nmap, WhatWeb, Nikto, Nuclei, and OWASP ZAP) coordinated by an n8n pipeline over three purpose-built Python services. Findings are normalized into a common structure, classified by severity, and stored in PostgreSQL. Full deployment requires a single command.

Evaluation was carried out on a containerized laboratory hosting two deliberately vulnerable applications, OWASP Juice Shop and DVWA, across three campaigns of five consecutive runs each. Time decomposition shows that inter-stage coordination accounts for 0.85 s against 259.06 s of total pipeline time, that is, 0.3 per cent: the remainder is tool execution, a floor common to any operator, whether human or automated. Coverage was measured against public ground-truth catalogues. Automatic matching produced no high-confidence correspondences; manual annotation establishes a recall of 0.27 on DVWA under authenticated mode, an optional condition that is not the pipeline default. The distribution of what is detected is not random: the pipeline finds what is observable in the server response, and not what requires completing an application flow. Per-tool decomposition yields three further unanticipated results: in campaign A, against DVWA, ZAP's active scan concentrates 96.1 per cent of the pipeline's timing variance, ZAP is the only tool whose precision withstands a change of target, and 97.4 per cent of reported severities are assigned by the normalization policy rather than by the detecting tool. A replication on a second target and a second host refutes the run-to-run consistency hypothesis and exposes that the system does not detect a tool's partial failure.

The manual arm, timed separately by each of the two authors (Section 12.2), took 1 h 31 min of attended work on average against the 0.24 s of orchestration the pipeline records in campaign C, the equivalent condition, subject to the operator-familiarity caveat Section 9.9 states.

Keywords: *vulnerability scanning, security orchestration, microservices, finding normalization, OWASP ZAP, automation.*

Índice General

Declaración de Originalidad.....	2
Resumen.....	3
Abstract.....	4
Índice General.....	5
Lista de Tablas.....	7
Lista de Figuras.....	8
Lista de Acrónimos.....	9
1. Introducción.....	9
2. Planteo del Problema.....	10
3. Justificación.....	11
4. Objetivos.....	12
4.1 Objetivo General.....	12
4.2 Objetivos Específicos.....	12
5. Preguntas de Investigación.....	14
6. Hipótesis.....	15
7. Estado del Arte.....	17
7.1 Antecedentes en la propia universidad.....	20
8. Marco Teórico.....	22
8.1 Vulnerabilidades de Seguridad Informática.....	22
8.2 El Proceso de Escaneo de Vulnerabilidades.....	23
8.3 Nmap: Descubrimiento de Red y Servicios.....	24
8.4 Herramientas de Análisis de Aplicaciones Web: WhatWeb, Nikto, Nuclei y OWASP ZAP	24
8.5 Estándares CVE y CVSS.....	26
8.6 Automatización de Procesos de Seguridad.....	28
8.7 Orquestación mediante n8n.....	29
8.8 Arquitectura de Microservicios y Contenedores.....	29
8.9 La Medición de la Cobertura: Recall, Precisión y Catálogo de Verdad de Referencia.....	30
9. Metodología.....	32
9.1 Enfoque y Diseño del Proyecto.....	32
9.2 Tipo de Investigación y Paradigma Epistemológico.....	33
9.3 Variables del Estudio.....	33
9.4 Entorno de Laboratorio.....	34
9.5 Herramientas y Tecnologías Utilizadas.....	35
9.6 Procedimiento de Evaluación y Protocolo Experimental.....	36

9.7 Criterios de Validez y Confiabilidad.....	38
9.8 Población, Muestra y Muestreo.....	38
9.9 Amenazas a la Validez.....	39
10. Arquitectura del Sistema.....	41
10.1 Descripción General.....	41
10.2 Segmentación de Red y Modelo de Seguridad (Defensa en Profundidad).....	43
10.3 Componentes del Sistema y Puertos.....	44
10.4 Flujo de Datos.....	44
11. Implementación Técnica.....	46
11.1 Configuración del Entorno de Despliegue.....	46
11.2 El Backend y su Arquitectura en Capas.....	46
11.3 El Servicio de Escaneo y el Patrón de Adaptadores.....	47
11.4 Normalización y Clasificación de Severidad.....	48
11.5 El Servicio de Reportes.....	50
11.6 Orquestación en n8n.....	50
11.7 Esquema de Base de Datos.....	52
11.8 Verificación Automatizada.....	54
11.9 Interfaz Web de Operación.....	55
12. Resultados Obtenidos.....	58
12.1 Diseño de la Evaluación.....	58
12.2 Resultados del Proceso Manual.....	58
12.3 Resultados del Sistema Automatizado.....	60
12.3.1 Origen de la variabilidad del pipeline.....	61
12.4 Análisis Comparativo.....	62
12.5 Distribución de Vulnerabilidades por Severidad.....	64
12.5.1 Estabilidad de la precisión entre objetivos.....	68
12.5.2 Procedencia de la severidad asignada.....	70
12.6 Replicación en un segundo anfitrión y un segundo objetivo.....	71
12.6.1 Lo que se sostiene y lo que no.....	72
12.6.2 H2 no se sostiene contra el segundo objetivo.....	72
12.6.3 Reproducibilidad de la medición sobre el mismo objetivo.....	73
12.6.4 Campañas presentes en el repositorio y no integradas al análisis.....	74
Evidencia Visual del Sistema en Ejecución.....	76
13. Análisis de Riesgos.....	78
13.1 Activos, Amenazas y Criterios de Aceptación.....	78
13.2 Riesgos Operativos.....	79

13.3 Riesgos de Seguridad.....	79
13.4 Riesgos de Disponibilidad.....	80
14. Limitaciones del Estudio.....	82
14.1 Limitaciones del Entorno de Evaluación.....	82
14.2 Limitaciones Técnicas del Sistema.....	83
14.3 Limitaciones del Diseño del Estudio.....	84
15. Discusión.....	85
15.1 La orquestación no es donde está el tiempo.....	85
15.2 La cobertura estratificada depende del objetivo.....	85
15.3 La severidad reportada es en parte un artefacto del criterio.....	86
15.4 Qué aporta el enfoque de orquestación.....	87
15.5 Lo que este diseño no permite afirmar.....	88
16. Conclusiones.....	90
16.1 Respuesta a las Preguntas de Investigación.....	90
16.2 Hallazgos principales.....	91
17. Trabajos Futuros.....	94
18. Consideraciones Éticas.....	96
19. Referencias Bibliográficas.....	98
20. Anexos Técnicos.....	102
Anexo A — Esquema Completo de Base de Datos.....	103
Anexo B — Invocaciones Reales de las Herramientas de Escaneo.....	105
Anexo C — Estructura del Informe en Markdown (plantilla Jinja2).....	105
Anexo D — Secuencia de Nodos del Workflow en n8n.....	106
Anexo E — Metodología de Verdad de Referencia (Ground Truth).....	107
Anexo F — docker-compose.yml Completo.....	107
Anexo G — Extracto del Workflow JSON de n8n.....	114
Anexo H — Salida Cruda de una Herramienta de Escaneo.....	116
Anexo I — Informes de Ejemplo Generados.....	118
Anexo J — .env.example Completo.....	119
Anexo K — Protocolo del Brazo Manual.....	121

Lista de Tablas

Tabla 1. Antecedentes directos, su objeto de estudio y el alcance de su evaluación

Tabla 2. Aplicaciones web vulnerables del entorno de laboratorio

Tabla 3. Especificación del entorno de la máquina anfitriona

Tabla 4. Herramientas y tecnologías utilizadas, con versión y licencia

Tabla 5. Componentes del sistema y sus puertos internos

Tabla 6. Endpoints principales del Backend
 Tabla 7. Invocación real de cada herramienta de escaneo por su adaptador
 Tabla 8. Tablas del esquema de base de datos y su responsabilidad
 Tabla 9. Pruebas automatizadas y cobertura de sentencias por componente.
 Tabla 10. Resultados del proceso manual por etapa, en dos corridas, una por autor
 Tabla 11. Resultados del sistema automatizado por ejecución
 Tabla 12. Dispersión del tiempo de ejecución por herramienta sobre cinco corridas contra DVWA.
 Tabla 13. Comparación entre la condición manual y la condición automatizada
 Tabla 14. Distribución de hallazgos por nivel de severidad
 Tabla 15. Recall, precisión y F1 por herramienta contra el catálogo de DVWA
 Tabla 16. Recall, precisión y F1 por herramienta contra el catálogo de Juice Shop
 Tabla 17. Recall, precisión y F1 por herramienta contra el catálogo de DVWA (escaneo autenticado)
 Tabla 18. Cobertura del catálogo de DVWA bajo anotación manual (corrida autenticada)
 Tabla 19. Precisión por herramienta y proporción de hallazgos sin correspondencia, según condición.
 Tabla 20. Severidad asignada y procedencia de la puntuación, por herramienta, sobre las tres condiciones.
 Tabla 21. Composición del tiempo y de la varianza por herramienta en las tres campañas.
 Tabla 22. Matriz de riesgos identificados, probabilidad, impacto y mitigación

Lista de Figuras

Figura 1. Diagrama de componentes (C4 nivel 2) de VulnScan Platform.
 Figura 2. Diagrama de secuencia de las 12 etapas del pipeline orquestado por n8n.
 Figura 3. Diagrama entidad-relación del esquema de PostgreSQL.
 Figura 4. Pantalla de objetivos: registro y listado de las aplicaciones del laboratorio habilitadas por la whitelist.
 Figura 5. Detalle de un objetivo: datos de registro, disparo del pipeline e historial de ejecuciones.
 Figura 6. Generación de informes en los cuatro formatos desde la vista de un escaneo.
 Figura 7. Desglose por herramienta de una ejecución del pipeline contra DVWA: duración y aporte de hallazgos de cada una.
 Figura 8. Hallazgos normalizados de un escaneo, con severidad homogeneizada y herramienta de origen.
 Figura 9. Workflow vulnscan-pipeline completo en n8n, ejecución exitosa de punta a punta (todos los nodos en verde).
 Figura 10. Informe de escaneo generado por el Servicio de Reportes en formato PDF, abierto en el visor de PDF del navegador.
 Figura 11. El mismo informe de escaneo en formato HTML, abierto desde el archivo descargado.

Figura 12. Respuesta cruda de GET /scans/{id}/findings contra el Backend, capturada en una terminal PowerShell (Invoke-RestMethod con el header X-API-Key).

Lista de Acrónimos

API — Application Programming Interface
CI/CD — Continuous Integration / Continuous Deployment
CIA (tríada) — Confidentiality, Integrity, Availability
CVE — Common Vulnerabilities and Exposures
CVSS — Common Vulnerability Scoring System
DAG — Directed Acyclic Graph (grafo dirigido acíclico)
DAST — Dynamic Application Security Testing
DVWA — Damn Vulnerable Web Application
EPSS — Exploit Prediction Scoring System
GVM — Greenbone Vulnerability Management
HTML — HyperText Markup Language
HTTP / HTTPS — HyperText Transfer Protocol (Secure)
ISO/IEC — International Organization for Standardization / International Electrotechnical Commission
JSON — JavaScript Object Notation
KEV — Known Exploited Vulnerabilities
NIST — National Institute of Standards and Technology
NVD — National Vulnerability Database
NVT — Network Vulnerability Test
ORM — Object-Relational Mapping
OWASP — Open Worldwide Application Security Project
PDF — Portable Document Format
SOAR — Security Orchestration, Automation and Response
SPA — Single Page Application
SQL — Structured Query Language
SSH — Secure Shell
TCP — Transmission Control Protocol
UUID — Universally Unique Identifier
VMDR — Vulnerability Management, Detection and Response
WMI — Windows Management Instrumentation
XML — eXtensible Markup Language
XSS — Cross-Site Scripting
ZAP — Zed Attack Proxy

1. Introducción

En la última década, la seguridad informática experimentó una transformación profunda, impulsada por la proliferación de servicios digitales, la adopción masiva de entornos en la nube y la interconexión de sistemas a escala global. Esa transformación amplió la superficie de ataque disponible para actores maliciosos y convirtió la gestión proactiva de vulnerabilidades en una necesidad operativa ineludible.

Las vulnerabilidades de software son una de las causas más frecuentes de incidentes de seguridad documentados a nivel mundial. Según el Informe de Investigaciones sobre Brechas de Datos de Verizon (2026), el 31 % de las brechas analizadas en el último año se originó en la explotación de una vulnerabilidad de software, frente al 20 % del informe anterior. Es la primera vez en los diecinueve años de la serie que este vector supera al robo de credenciales como vía

de acceso inicial. Gestionarlas sistemáticamente exige herramientas especializadas y profesionales calificados, escasos a nivel global: ISC2 (2024) estimó una brecha de 4,8 millones de profesionales necesarios y no cubiertos. A esa escasez se suma el costo de las plataformas comerciales: el precio de lista de Tenable Nessus Professional, una de las opciones de entrada del segmento, es de USD 4.790 anuales por escáner (Tenable, Inc., 2026), inaccesible para laboratorios académicos, pequeñas organizaciones e instituciones educativas.

El presente trabajo nace de esta necesidad en el contexto del Laboratorio de Redes y Seguridad de la Facultad Regional Mendoza de la Universidad Tecnológica Nacional. Se propone el diseño e implementación de una plataforma automatizada de análisis de vulnerabilidades, construida íntegramente sobre herramientas de código abierto. La plataforma se articula mediante una arquitectura de microservicios desacoplados que se comunican por contratos HTTP, coordinados por un pipeline de orquestación en n8n: descubrimiento de activos, ejecución de las herramientas de escaneo, normalización y clasificación de hallazgos, persistencia estructurada y generación automática de informes. El objetivo es reducir la carga operativa sobre los analistas, mejorar la consistencia del proceso y democratizar el acceso a prácticas profesionales de gestión de vulnerabilidades.

2. Planteo del Problema

A pesar de la disponibilidad creciente de herramientas de seguridad de código abierto con capacidades comparables a sus equivalentes comerciales, muchas organizaciones con recursos limitados enfrentan serias dificultades para implementar procesos de evaluación de vulnerabilidades eficientes, consistentes y reproducibles. Esta distancia entre disponibilidad teórica y adopción práctica no es una impresión de los autores: la revisión sistemática de Mohammed et al. (2025) identifica la integración fluida de herramientas de seguridad y la orquestación de cadenas de herramientas entre las brechas abiertas del área, y la señala como una de las dificultades recurrentes de adopción. Esa brecha define el problema central que motiva este trabajo, y se manifiesta en dos frentes.

El primero es operativo. Herramientas como Nmap, WhatWeb, Nikto, Nuclei y OWASP ZAP son eficaces en sus respectivos dominios, pero operan con independencia. Cada una genera resultados en formatos heterogéneos (XML, JSON con esquemas dispares) no interoperables entre sí. Esto obliga al analista a intervenir manualmente en cada transición: ejecutar cada herramienta, esperar activamente su finalización, interpretar y homogeneizar los resultados bajo un criterio de severidad uniforme, y elaborar un informe consolidado.

Ese proceso, según estimación de los propios autores a partir de su experiencia directa (no derivada de una medición sistemática, ver Sección 9.9), puede requerir del orden de una a dos horas de trabajo calificado por ciclo completo. Esta demanda de tiempo dificulta la ejecución periódica del proceso, especialmente en equipos de seguridad reducidos, y genera ventanas prolongadas de exposición sin remediar.

El segundo es pedagógico: los programas de formación en programación y ciberseguridad a nivel universitario tienen la responsabilidad de ofrecer una comprensión integrada del flujo completo de análisis de seguridad tal como se ejecuta en entornos profesionales reales, algo que la enseñanza basada en herramientas aisladas no logra transmitir. La literatura del área respalda esta lectura: tanto Książkowski et al. (2022), sobre un curso de seguridad web basado en desafíos prácticos, como Su (2024), sobre un entorno de aprendizaje web con ejercicios guiados, documentan que el valor formativo proviene de trabajar sobre el flujo completo y no sobre ejercicios desconectados entre sí.

Se identifica, por tanto, la necesidad de un sistema que automatice la ejecución coordinada de las herramientas y elimine la intervención manual entre etapas. Ese sistema debe normalizar y clasificar los resultados heterogéneos bajo un criterio uniforme, consolidarlos en un repositorio

estructurado y generar informes automáticamente en múltiples formatos. Debe, además, operar sobre software libre en una arquitectura reproducible y extensible, adaptada al contexto académico.

3. Justificación

La relevancia de este trabajo se sostiene en tres dimensiones convergentes. Técnicamente, automatizar el análisis de vulnerabilidades mediante una arquitectura de microservicios desacoplados, que permite incorporar nuevas herramientas o formatos sin modificar los componentes existentes, responde a una tendencia documentada en la literatura del área: Nasab et al. (2023) relevan las prácticas de seguridad efectivamente adoptadas en sistemas de microservicios reales, y Mohammed et al. (2025) sitúan la orquestación de herramientas entre las líneas activas de trabajo. Ambas se desarrollan en profundidad en las Secciones 7 y 8. Académicamente, el proyecto ofrece a los estudiantes de la Tecnicatura Universitaria en Programación un entorno práctico donde observar y reproducir un pipeline completo de análisis de seguridad, desde el descubrimiento de activos hasta la generación de informes. Esa visión integral es difícil de alcanzar mediante ejercicios de laboratorio con herramientas aisladas. El proyecto funciona además, en sí mismo, como objeto de estudio de ingeniería de software: materializa patrones de diseño (adaptador, registro, repositorio) y el paradigma SOAR que estructura el trabajo de los equipos de seguridad modernos. Económicamente, el uso exclusivo de herramientas de código abierto elimina las barreras de licenciamiento discutidas en la Sección 1. El sistema no busca equipararse funcionalmente a las soluciones comerciales en todos sus aspectos, cuestión no evaluada experimentalmente en este trabajo, sino demostrar que es posible automatizar el ciclo completo de análisis con herramientas gratuitas en un entorno académico. Finalmente, el proyecto desarrolla competencias transferibles al mercado laboral (arquitectura de microservicios, contenerización, automatización de procesos, diseño de bases de datos, elaboración de documentación técnica) alineadas con el perfil de egreso de la Tecnicatura Universitaria en Programación de la Universidad Tecnológica Nacional, Facultad Regional Mendoza.

4. Objetivos

4.1 Objetivo General

Diseñar e implementar una plataforma automatizada para la detección, análisis y reporte de vulnerabilidades en aplicaciones y servicios de red, integrando herramientas open source de amplia adopción en la industria (Nmap, WhatWeb, Nikto, Nuclei y OWASP ZAP) sobre una arquitectura de microservicios contenerizada, orquestada mediante la plataforma de workflows n8n y con persistencia en PostgreSQL. La plataforma persigue dos propósitos: eliminar la intervención manual durante la ejecución del proceso y generar informes de seguridad estructurados de manera completamente automática. A ello se suma un componente evaluativo: medir, mediante la descomposición interna de los tiempos del propio pipeline, el trabajo humano de consolidación y clasificación que la automatización elimina entre etapas (ver Sección 6, PM-1). La comparación cronometrada contra un proceso manual equivalente se ejecutó en dos corridas, una por cada autor, con el alcance y las limitaciones que declaran las Secciones 12.2 y 14.3.

4.2 Objetivos Específicos

1. Implementar un Servicio de Escaneo desacoplado que ejecute las herramientas de análisis mediante un patrón de adaptadores extensible y exponga su capacidad a través de un contrato HTTP uniforme. Cada herramienta —Nmap para descubrimiento de servicios, WhatWeb para fingerprinting de tecnologías, y Nikto, Nuclei y OWASP ZAP para vulnerabilidades de aplicación web— se integra como un adaptador independiente, sin modificar los componentes existentes.
2. Desarrollar un Backend en FastAPI que centralice la lógica de negocio: registro y validación de objetivos, gestión del ciclo de vida de los escaneos, normalización de resultados heterogéneos a una estructura común y clasificación de severidad según el estándar CVSS. Su arquitectura en capas —enrutadores, servicios, repositorios— separa la lógica de presentación, de negocio y de acceso a datos.
3. Diseñar e implementar un esquema relacional en PostgreSQL que registre de modo persistente y consultable el ciclo de vida completo de cada análisis. Ese ciclo comprende objetivos, ejecuciones del pipeline, tareas por herramienta, servicios y tecnologías descubiertos, hallazgos normalizados con sus métricas CVSS y referencias CVE asociadas. El esquema emplea claves primarias UUID y tipos enumerados nativos para los vocabularios cerrados.
4. Desarrollar un Servicio de Reportes que genere informes de seguridad en cuatro formatos (PDF, HTML, Markdown y JSON) a partir de una carga de datos autocontenida, sin acoplamiento a la base de datos ni a los demás servicios, automáticamente y en un plazo acotado desde la finalización del escaneo.
5. Orquestar el ciclo completo de análisis mediante un workflow de n8n que coordine, exclusivamente por llamadas HTTP a los servicios propios, las etapas de recepción del objetivo, reconocimiento, identificación de tecnologías, selección de herramientas, escaneo, normalización, persistencia y generación del reporte. La ejecución, normalización y clasificación residen en servicios propios versionados; n8n conserva la resolución de contexto y la selección de puerto.

6. Evaluar el desempeño del sistema automatizado mediante la descomposición de los tiempos de ejecución del pipeline entre orquestación y ejecución de herramientas, analizando la consistencia de los resultados entre ejecuciones sucesivas y la cobertura de vulnerabilidades detectadas en el entorno de laboratorio configurado. La comparación cronometrada contra un proceso manual equivalente se ejecutó en dos corridas, una por cada autor, con el alcance y las limitaciones que declaran las Secciones 12.2 y 14.3.

5. Preguntas de Investigación

El desarrollo del presente trabajo está orientado a responder las siguientes preguntas de investigación, formuladas de manera que guíen tanto el diseño del sistema como el diseño del estudio experimental de evaluación:

1. ¿Puede automatizarse el ciclo completo de escaneo de vulnerabilidades, desde el descubrimiento de activos hasta la generación de informes, sobre una arquitectura de microservicios de código abierto orquestada por n8n, y qué decisiones del proceso resisten esa delegación al orquestador?
2. ¿En qué medida la automatización reduce el tiempo total requerido para completar un ciclo de evaluación de vulnerabilidades en un entorno controlado, frente a la ejecución manual de las mismas etapas por un operador con conocimiento de las herramientas? Esta pregunta motivó el diseño experimental original y se responde con dos corridas cronometradas del brazo manual, una por cada autor (Sección 12.2 y Anexo K). Las salvedades metodológicas, en particular el sesgo de familiaridad del operador, se declaran en las Secciones 9.9 y 14.3.
3. ¿Es posible normalizar a una representación común las salidas heterogéneas de cinco herramientas de análisis (Nmap en XML, y WhatWeb, Nikto, Nuclei y ZAP en esquemas JSON distintos) dentro de un pipeline unificado, y qué información se pierde necesariamente en esa normalización?
4. ¿Qué proporción del tiempo total de una ejecución del pipeline corresponde a la coordinación entre etapas, incluida la generación del informe, y qué proporción a la ejecución de las herramientas que esa coordinación gobierna?
5. ¿Qué clases de vulnerabilidad del catálogo de verdad de referencia detecta el pipeline sobre los objetivos del laboratorio, cuáles quedan fuera de su alcance, y qué característica del diseño explica esa frontera?

6. Hipótesis

Sobre la base del análisis del problema, la revisión del estado del arte y el examen de la literatura académica disponible, el presente trabajo formula una hipótesis de trabajo contrastable, cuya validación es uno de los objetivos centrales del estudio experimental. El planteo original contenía tres enunciados; los otros dos se retiran del bloque por las razones que se detallan a continuación.

H2 (reformulada). La automatización produce resultados consistentes entre ejecuciones sucesivas contra el mismo objetivo —medido como variación del conjunto de hallazgos (cantidad total y distribución de severidad), no del tiempo de ejecución. Sobre las cinco corridas de la campaña A: el recuento total de hallazgos fue idéntico en las cinco (32) y la distribución de severidad fue idéntica en las cinco ({info: 21, low: 5, medium: 5, critical: 1}). H2 se sostiene en esa campaña, con 0 % de variación observada, pero no se sostiene como enunciado general: las dos réplicas de la Sección 12.6 —contra un segundo objetivo y sobre un segundo anfitrión— la refutan, y el detalle de cada caso se desarrolla en la Sección 12.6.2. La hipótesis debe leerse, en consecuencia, como verificada bajo la condición original y refutada al introducir variación de objetivo o de anfitrión.

Enunciados retirados del bloque. Los dos enunciados que siguen formaban parte del planteo original y se retiran del bloque de hipótesis por una razón común: ninguno de los dos enuncia una predicción que el diseño pudiera haber refutado.

PM-1 (pregunta de medición, antes H1). Tal como quedó formulada tras eliminar el umbral no derivado del planteo original, no enuncia una predicción que pudiera haber fallado —cualquier valor medido habría sido compatible con ella—, de modo que corresponde tratarla como pregunta de medición y no como hipótesis contrastable. Su contenido se conserva íntegro y se responde con dato en la Sección 12.3. La automatización elimina el trabajo humano de consolidación, clasificación y generación del informe entre etapas del pipeline —el componente de tiempo que un operador manual sí tendría que ejecutar y que las cinco herramientas de escaneo, corriendo con parámetros idénticos en cualquier condición, no requieren. Medido sobre 5 corridas automatizadas contra dwwa, este componente (“orquestación”) tiene una media de 0,85 s (CV 20,6%) de un tiempo total de pipeline de 259,06 s de media —es decir, el 99,7% del tiempo total es ejecución de herramientas, un piso que ninguna de las dos condiciones puede comprimir, aunque su magnitud sí varíe entre campañas: la Sección 12.6.3 documenta una caída del 35,4 % entre la A y la C sobre el mismo objetivo. El otro brazo de la comparación —el proceso manual cronometrado— se midió en dos corridas, una por cada autor y se reporta en la Sección 12.2. El contraste entre ambos figura en la Sección 12.4, acotado por el tamaño de la muestra y por el sesgo de familiaridad que la Sección 9.9 declara. La hipótesis H3 del planteo original, sobre el tiempo de generación de informes, se retira también de este bloque. Su umbral está garantizado por construcción, de modo que no es contrastable como hipótesis de investigación: se reformula como requisito no funcional del Servicio de Reportes (Sección 11.5).

PM-1 es de naturaleza cuantitativa y su evaluación se fundamenta en la medición sistemática y objetiva del componente de orquestación del pipeline automatizado, con independencia de las dos corridas manuales cronometradas que reporta la Sección 12.2 (ver Sección 14.3). El bloque conserva así una única hipótesis contrastable, H2, lo que es preferible a presentar como hipótesis

enunciados que el diseño no puede refutar. H2 es de naturaleza cuantitativa y se evalúa mediante la variación del conjunto de hallazgos entre ejecuciones sucesivas del pipeline automatizado. El detalle del procedimiento experimental y los criterios de validación adoptados se presentan en la sección de metodología.

7. Estado del Arte

Alcance de esta revisión. Lo que sigue es una revisión narrativa y no una revisión sistemática de la literatura: no se aplicó un protocolo de búsqueda exhaustivo con cadenas, bases y criterios de cribado preestablecidos, de modo que no puede afirmarse que el conjunto reunido agote los antecedentes del área. Se declara el criterio efectivamente empleado para seleccionarlos, que es el que permite juzgar lo que esta revisión cubre y lo que deja fuera. Se incorporaron trabajos de tres clases: los que evalúan empíricamente escáneres de vulnerabilidades web, porque fijan el punto de comparación de la capacidad de detección; los que abordan la automatización u orquestación de herramientas de seguridad, porque delimitan el problema que este trabajo aborda; y los que documentan entornos prácticos de enseñanza de seguridad de aplicaciones, porque sostienen la dimensión pedagógica de la justificación. A ellos se suman antecedentes producidos en la propia Universidad Tecnológica Nacional, incorporados por pertinencia institucional. Se privilegió la producción de los últimos cinco años, salvo en las obras de referencia metodológica y en las fuentes normativas, donde rige la versión vigente del estándar. La consecuencia de este alcance debe enunciarse sin rodeos: la afirmación de vacancia que cierra el capítulo se sostiene sobre los antecedentes efectivamente relevados y no sobre una búsqueda exhaustiva, y una revisión sistemática podría encontrar trabajos que este capítulo no recoge.

La gestión de vulnerabilidades como disciplina formal dentro de la ciberseguridad tiene sus raíces en los primeros programas de evaluación de seguridad desarrollados en el ámbito académico y gubernamental durante la década de 1990. El antecedente conceptual más citado de esa línea es el trabajo de Farmer y Venema (1993), que propuso examinar un sistema desde la perspectiva de un intruso potencial para exponer sus debilidades, y mostró que servicios de red aparentemente inocuos se vuelven instrumentos de reconocimiento aun operando exactamente como fueron diseñados. Poco después los mismos autores llevaron ese argumento a una herramienta, SATAN (Security Administrator Tool for Analyzing Networks), que automatizaba la comprobación de vulnerabilidades conocidas alcanzables por red. Sin embargo, fue la expansión masiva de Internet y la proliferación de servicios en línea durante los años 2000 la que impulsó el desarrollo de herramientas especializadas y, con el tiempo, plataformas integradas de gestión de vulnerabilidades orientadas al uso empresarial.

En el segmento de soluciones comerciales, Tenable Nessus se consolidó como referente del sector, sobre la base de un catálogo de pruebas de vulnerabilidad actualizado periódicamente y de capacidades de escaneo autenticado (Tenable, Inc., 2026). Con el tiempo, Tenable evolucionó su oferta hacia Tenable Vulnerability Management, un servicio basado en la nube que incorpora funcionalidades de gestión de riesgos, correlación de vulnerabilidades y priorización basada en inteligencia de amenazas. Qualys, una de las primeras soluciones del segmento concebidas por completo sobre infraestructura en la nube, ofrece capacidades similares con énfasis en la escalabilidad y la integración con procesos de DevSecOps mediante su plataforma VMDB (Vulnerability Management, Detection and Response). Rapid7 InsightVM, por su parte, destaca por sus capacidades de remediación guiada y su integración nativa con plataformas de gestión de incidentes como Jira y ServiceNow. El costo de licenciamiento de estas soluciones escala con el número de activos gestionados y parte de un piso que ya resulta prohibitivo para el destinatario de este trabajo: la licencia de entrada de Nessus Professional, limitada a un único escáner,

cuesta USD 4.790 anuales (Tenable, Inc., 2026). Las tres plataformas comercializan sus configuraciones para múltiples activos mediante cotización y no publican su precio de lista, de modo que este trabajo no consigna cifras para ellas.

Esta brecha impulsó el desarrollo y consolidación de un ecosistema de alternativas de código abierto especializadas por dominio. En el descubrimiento de red, Nmap se consolidó como el estándar de facto. En el análisis de vulnerabilidades de red, la referencia más extendida es OpenVAS. Su origen está documentado por quienes lo mantienen: en 2005 los desarrolladores de Nessus discontinuaron el trabajo bajo licencias de código abierto y adoptaron un modelo propietario, y en 2006 surgieron varias bifurcaciones del proyecto, de las cuales OpenVAS fue la única que conservó actividad sostenida (Greenbone Networks GmbH, 2024). Hoy se mantiene bajo la denominación Greenbone Vulnerability Management (GVM), con un catálogo que supera las cien mil pruebas de vulnerabilidad de red o NVT (Network Vulnerability Test). En el dominio específico de la seguridad de aplicaciones web, foco del presente trabajo, se ha consolidado un conjunto de herramientas complementarias. Nikto se orienta a la detección de configuraciones inseguras y archivos peligrosos en servidores web. WhatWeb se especializa en el fingerprinting de tecnologías. Nuclei, del proyecto ProjectDiscovery, ejecuta plantillas declarativas mantenidas por la comunidad para detectar vulnerabilidades conocidas, exposiciones y configuraciones incorrectas. OWASP ZAP, herramienta insignia de la OWASP Foundation para el análisis dinámico de aplicaciones web, rastrea la aplicación y ejecuta un escaneo activo que ataca cada parámetro descubierto en busca de vulnerabilidades del OWASP Top 10.

En paralelo al desarrollo de herramientas de análisis, el campo de la automatización de procesos de seguridad ha experimentado una evolución significativa. El paradigma SOAR (Security Orchestration, Automation and Response) designa la coordinación automatizada de flujos de trabajo de seguridad completos, y ha dado lugar a un mercado de plataformas comerciales que ofrecen ecosistemas de integraciones y playbooks preconfigurados —Splunk SOAR, Palo Alto Cortex XSOAR e IBM Security QRadar SOAR, entre otras— con las mismas barreras económicas que las soluciones comerciales de análisis de vulnerabilidades. Bridges et al. (2023) caracterizan el paradigma y lo evalúan en operación real, y su hallazgo principal interesa directamente al diseño del presente trabajo: los analistas más experimentados desconfían de la sobreautomatización, y las herramientas preferidas son las que equilibran la automatización con el apoyo a la decisión humana. En el ámbito de la integración continua, Marandi et al. (2023) documentan la incorporación de escaneo de seguridad automatizado a un pipeline DevSecOps de CI/CD, un antecedente conceptualmente análogo a la orquestación del presente sistema aunque aplicado a un contexto de integración continua de software en lugar de evaluación de aplicaciones desplegadas.

En el ámbito de la automatización de código abierto, herramientas como Apache Airflow, originalmente concebida para la orquestación de pipelines de datos, han sido adaptadas para coordinar flujos de trabajo de seguridad, y herramientas como Ansible se emplean para tareas de administración con implicaciones de seguridad. Unas y otras definen sus flujos en código, lo que supone en el operador competencias de programación que el destinatario de una plataforma de seguridad no necesariamente tiene. Las plataformas de automatización de workflows con interfaz visual de bajo código, como n8n, Prefect o Apache NiFi, desplazan esa exigencia hacia la composición gráfica del flujo. n8n publicó su primera versión en 2019 bajo un modelo de código

fuente disponible, con opción de despliegue en infraestructura propia (n8n GmbH, 2024). Desde entonces ganó una comunidad activa y un catálogo creciente de integraciones, que la hacen adecuada para proyectos de automatización de seguridad donde la privacidad de los datos es un requisito no negociable.

En la literatura académica reciente, diversos trabajos han abordado la automatización de procesos de seguridad utilizando herramientas de código abierto. Bridges et al. (2023) evaluaron herramientas SOAR (Security Orchestration, Automation and Response) con operadores de cuatro centros de operaciones de seguridad sobre cinco tipos de investigación estructurada, y encontraron que la sobreautomatización preocupaba a los analistas más experimentados: las herramientas preferidas fueron las que equilibraban la automatización con el apoyo a la decisión humana. Amankwah et al. (2020) compararon empíricamente ocho escáneres de vulnerabilidades web, comerciales y open source, entre ellos OWASP ZAP, una de las cinco herramientas integradas en el presente sistema. Los evaluaron contra WebGoat y DVWA, uno de los dos objetivos del laboratorio de este trabajo, y concluyeron que, si bien los escáneres comerciales son eficaces, algunos de código abierto —ZAP entre ellos— también lo son. Cierran recomendando mejorar la cobertura, elevar las tasas de detección y reducir los falsos positivos. Książkowski et al. (2022) y Su (2024) documentaron el valor pedagógico de entornos de laboratorio prácticos (hands-on) para la enseñanza de seguridad de aplicaciones web, en la misma línea que el uso de Juice Shop y DVWA en el presente trabajo.

El presente trabajo se inscribe dentro de esta línea de investigación aplicada, con dos particularidades que lo distinguen. La primera es su foco en el análisis de aplicaciones web mediante la integración de un conjunto de herramientas especializadas y complementarias (Nmap, WhatWeb, Nikto, Nuclei y ZAP) en lugar de depender de un único motor monolítico. La segunda es su enfoque de ingeniería: en vez de embeber la lógica de integración dentro de nodos de un orquestador, el sistema la encapsula en microservicios propios versionados y testeables, dejando a n8n el rol exclusivo de coordinación. Este diseño, adaptado a las necesidades y recursos de un entorno académico de nivel universitario de la República Argentina, es el aporte diferencial del trabajo.

Tabla 1

Antecedentes directos, su objeto de estudio y el alcance de su evaluación

Antecedente	Objeto de estudio	Entorno de evaluación	Qué mide y qué deja fuera
Amankwah et al. (2020)	Ocho escáneres web, comerciales y de código abierto	WebGoat y DVWA	Precisión y recall de detección; no aborda orquestación ni consolidación de salidas
Bridges et al. (2023)	Plataformas SOAR comerciales en operación real	Cuatro centros de operaciones de seguridad	Desempeño y preferencia de operadores; no evalúa herramientas de escaneo ni normalización
Marandi et al. (2023)	Escaneo de seguridad en	Pipeline DevSecOps de CI/CD	Automatización disparada por cambios en el código;

Antecedente	Objeto de estudio	Entorno de evaluación	Qué mide y qué deja fuera
	integración continua		no por objetivo desplegado
Nasab et al. (2023)	Prácticas de seguridad en microservicios	Diez sistemas de código abierto y 306 consultas técnicas	Catálogo de 28 prácticas validado con 74 profesionales; no aborda escaneo de vulnerabilidades
Este trabajo	Orquestación de cinco herramientas heterogéneas y normalización de sus salidas	Juice Shop y DVWA en laboratorio contenerizado	Descomposición del tiempo del pipeline, solapamiento entre herramientas y cobertura contra catálogo de verdad de referencia

Nota. Elaboración propia a partir de las fuentes citadas.

La comparación anterior permite enunciar el vacío que este trabajo aborda. **La literatura consultada evalúa herramientas de escaneo o evalúa plataformas de orquestación, pero no la unión de ambas cosas: la consolidación de las salidas de varias herramientas heterogéneas bajo un criterio de severidad único, y el costo y la cobertura que esa consolidación tiene.** Amankwah et al. comparan escáneres entre sí, pero cada uno reporta en su propio formato y con su propia escala, y el trabajo no aborda qué ocurre cuando hay que unificarlos. Bridges et al. evalúan plataformas que orquestan respuesta a incidentes, no ejecución de escaneo ni normalización de hallazgos. Marandi et al. automatizan escaneo dentro de una cadena de integración continua, donde el disparador es un cambio en el código y no un objetivo desplegado. Nasab et al. documentan prácticas de seguridad para sistemas de microservicios sin ocuparse del dominio de la evaluación de vulnerabilidades.

Ese hueco define las tres preguntas que este trabajo se plantea y responde con medición propia. La primera: **qué proporción del tiempo de un análisis automatizado corresponde efectivamente a la orquestación** y no a la ejecución de las herramientas. La segunda: **si un conjunto de herramientas heterogéneas aporta cobertura complementaria o redundante** sobre un mismo objetivo. La tercera: **en qué medida una severidad normalizada a partir de escalas dispares describe al objetivo analizado y en qué medida al criterio que la construyó.** Las tres se abordan en las Secciones 12 y 15.

7.1 Antecedentes en la propia universidad

Un trabajo final debe situarse también respecto de la producción de su propia institución. El Repositorio Institucional Abierto de la Universidad Tecnológica Nacional permite establecer ese encuadre con precisión, y el resultado conviene enunciarlo sin adornos: en la Facultad Regional Mendoza no existe un trabajo final previo sobre escaneo automatizado de vulnerabilidades. Los antecedentes institucionales pertinentes se ubican en dos planos distintos.

El primero es temático y se encuentra en otras facultades regionales. El antecedente más próximo es el de Barrionuevo et al. (2015), que proponen un sistema integrado de soporte para el análisis de vulnerabilidades en sistemas web. La misma línea de trabajo produjo antes una

revisión del estado del arte de las técnicas y herramientas disponibles para construirlo (Cuevas et al., 2016) y más tarde un análisis de vulnerabilidades sobre sistemas web en desarrollo y en producción (Cuevas et al., 2018). La diferencia con el presente trabajo es de alcance y no de tema: aquellos sistemas asisten a un analista humano que conduce el proceso y decide qué herramienta aplicar, mientras que aquí el ciclo completo —descubrimiento, selección de herramienta, ejecución, normalización, clasificación de severidad y emisión del informe— se ejecuta sin intervención entre la solicitud y la entrega. A ello se suman dos tesis de maestría de la misma universidad que abordan la gestión de vulnerabilidades desde ángulos complementarios: Giannone (2019) propone un método para incorporar hacking ético al proceso de testing de software, y Cifre (2020) modela la gestión de vulnerabilidades de servidores en nubes privadas. Ninguno de los cinco mide la descomposición temporal de un pipeline orquestado ni el solapamiento entre herramientas heterogéneas, que son las dos preguntas propias de este trabajo.

El segundo plano es institucional y local. En la Facultad Regional Mendoza, Rotella (2016) aborda la seguridad desde la evaluación económica de soluciones de gestión de identidades y accesos, un objeto distinto del presente pero que confirma la presencia de la línea de seguridad en la facultad. En cuanto al formato y al alcance esperables de un trabajo final de la carrera, sirven de referencia trabajos como Céspedes Ortega et al. (2023), que construyen un documentador automático de código fuente, y Ciperiani (2025), que implementa un laboratorio remoto de Internet de las cosas; ambos comparten con este trabajo la construcción de un artefacto de software verificable como núcleo de la contribución.

La conclusión de este encuadre es doble. El problema abordado tiene antecedente en la universidad pero no en la facultad, y el enfoque —orquestación de herramientas heterogéneas con normalización de sus salidas, medido sobre su propia descomposición temporal— no tiene antecedente en ninguna de las dos. Eso delimita el aporte sin sobreestimarlos: no se trata de un problema nuevo, sino de un tratamiento del problema que la producción institucional consultada no había abordado.

8. Marco Teórico

8.1 Vulnerabilidades de Seguridad Informática

En el ámbito de la seguridad informática, una vulnerabilidad se define como una debilidad, falla o deficiencia en el diseño, implementación, configuración u operación de un sistema, red, aplicación o componente de software que puede ser identificada y potencialmente aprovechada por un actor malicioso para comprometer las propiedades de seguridad del sistema afectado. Esta definición es coherente con la formulada por el NIST en el glosario de su Computer Security Resource Center (NIST, 2023b). Allí la vulnerabilidad se define como una debilidad en un sistema de información, en sus procedimientos de seguridad, en sus controles internos o en su implementación, susceptible de ser explotada o desencadenada por una fuente de amenaza. La formulación distingue la vulnerabilidad, condición inherente del sistema, de la amenaza, agente o evento capaz de explotarla; el riesgo es la función que relaciona ambas dimensiones con el impacto potencial de la explotación exitosa.

Las propiedades de seguridad que las vulnerabilidades ponen en riesgo se articulan habitualmente en torno a la tríada CIA (Kim y Solomon, 2018), del inglés Confidentiality, Integrity, Availability. La confidencialidad protege la información frente al acceso no autorizado. La integridad garantiza que no ha sido modificada de manera no autorizada. La disponibilidad garantiza que los sistemas y la información son accesibles cuando y para quienes deben serlo. A esta tríada, la literatura especializada contemporánea añade con frecuencia propiedades como la autenticidad, el no repudio y la responsabilidad (accountability) como dimensiones complementarias de un modelo de seguridad completo.

Las vulnerabilidades pueden originarse en múltiples factores que actúan de manera independiente o combinada. Los errores de programación son la categoría más documentada: desbordamientos de buffer, inyecciones de código SQL o de comandos del sistema, referencias directas a objetos inseguros, y mecanismos de deserialización inseguros (todos ellos presentes en el catálogo OWASP Top Ten (OWASP Foundation, 2021)). Las configuraciones incorrectas representan otra categoría prevalente: el uso de credenciales por defecto no modificadas, la exposición de servicios de administración remota a redes no confiables, la ausencia de cifrado en la transmisión de datos sensibles, o la asignación de privilegios excesivos. Finalmente, el software desactualizado sin parches de seguridad vigentes es quizás el vector de entrada más explotado en ataques reales, dado que las vulnerabilidades correspondientes son de dominio público y existen herramientas automatizadas para su explotación masiva.

Dado que el presente trabajo se enfoca en el análisis de aplicaciones web, cobra especial relevancia el catálogo OWASP Top Ten, que sistematiza las diez categorías de riesgo más críticas en aplicaciones web (entre ellas el control de acceso roto, los fallos criptográficos, la inyección, el diseño inseguro y la configuración de seguridad incorrecta). Las herramientas de escaneo dinámico integradas en la plataforma (Nikto, Nuclei y ZAP) apuntan precisamente a detectar manifestaciones observables de estas categorías desde la perspectiva de un cliente externo. Corresponde precisar cuál de sus ediciones adopta este trabajo, porque el catálogo se revisa periódicamente. Las categorías que se enumeran aquí, y la clasificación que el sistema aplica a sus hallazgos, corresponden a la edición 2021 (OWASP Foundation, 2021), que es la que se cita de forma uniforme en todo el documento. La OWASP Foundation publicó después

una edición 2025 que reorganiza y renumera el catálogo (OWASP Foundation, 2025). Las cinco categorías mencionadas en el párrafo anterior se conservan en ella, de modo que el análisis de este trabajo no queda invalidado. Una de sus incorporaciones merece mención aparte: los fallos en la cadena de suministro de software pasan a constituir una categoría propia, y eso coincide con un riesgo que este trabajo identificó por su cuenta y a partir de un incidente real, no por seguir el catálogo —la resolución en tiempo de construcción de las versiones de las herramientas de escaneo sin fijarlas, que las Secciones 13 y 14.1 analizan y que llegó a dejar al sistema sin su única herramienta capaz de emitir severidad crítica—. Reasignar la clasificación a la edición 2025 exigiría modificar el sistema y no el documento, por lo que queda fuera del estado del código que este trabajo describe y evalúa (Sección 20).

8.2 El Proceso de Escaneo de Vulnerabilidades

El escaneo de vulnerabilidades es el proceso técnico mediante el cual se analiza sistemáticamente un conjunto de sistemas informáticos con el objetivo de identificar debilidades de seguridad conocidas que podrían ser aprovechadas por actores maliciosos. Este proceso se diferencia del test de penetración (pentesting), distinción desarrollada en detalle por Engebretson (2013), en que no busca explotar activamente las vulnerabilidades identificadas, salvo en modalidades avanzadas de validación, sino únicamente identificarlas, clasificarlas y documentarlas para su posterior remediación por parte de los equipos responsables.

Un proceso completo de escaneo de vulnerabilidades comprende varias etapas bien diferenciadas que se ejecutan secuencialmente. La primera etapa es el descubrimiento de activos y servicios, que consiste en identificar qué sistemas están presentes y activos, y qué servicios y aplicaciones exponen, en qué puertos operan y qué versiones de software corresponden a cada uno. La segunda etapa, particularmente relevante en el análisis de aplicaciones web, es la identificación de tecnologías (fingerprinting), que determina el stack sobre el que se construye cada aplicación (servidor web, framework, lenguaje, bibliotecas) dado que esta información condiciona qué comprobaciones de vulnerabilidad son pertinentes. La tercera etapa es el análisis de vulnerabilidades propiamente dicho, en el que los servicios y aplicaciones detectados son sometidos a comprobaciones específicas (comparación contra bases de datos de vulnerabilidades conocidas, plantillas de detección, o ataques activos controlados sobre los parámetros de la aplicación) para identificar debilidades. Finalmente, la etapa de reporte consolida los resultados en documentos estructurados que permiten a los responsables de seguridad priorizar la remediación en función de su severidad, explotabilidad e impacto potencial.

Una tarea transversal y crítica en este proceso es la normalización: dado que cada herramienta produce resultados en un formato y con un vocabulario propios, es necesario traducirlos a una representación común y clasificar su severidad bajo un criterio uniforme antes de poder consolidarlos. Esta tarea, habitualmente realizada a mano por el analista, es una de las que el presente sistema automatiza explícitamente.

Análisis estático y análisis dinámico. La disciplina distingue dos formas de buscar vulnerabilidades, según lo que cada una observa. El análisis estático examina el código fuente o el binario sin ejecutarlo, y alcanza los defectos visibles en la escritura del programa. El análisis dinámico examina la aplicación en ejecución e interactúa con ella desde fuera, de modo que

alcanza los defectos que se manifiestan en su comportamiento. Ninguno de los dos subsume al otro: el estático ve código que quizá nunca se ejecute, y el dinámico ve efectos cuyo origen en el código desconoce. Qadir et al. (2025) evalúan nueve herramientas de ambas familias, cuatro estáticas y cinco dinámicas, sobre setenta y cinco aplicaciones web reales.

El presente trabajo se sitúa por completo del lado dinámico. Las cinco herramientas que integra analizan una aplicación desplegada y en ejecución, sin acceso a su código fuente. Esa decisión responde al objetivo de evaluar objetivos ya desplegados y no a una preferencia metodológica. Su consecuencia debe declararse igual: las clases de vulnerabilidad que solo el análisis estático alcanza quedan fuera del sistema por diseño, no por una carencia de la implementación.

8.3 Nmap: Descubrimiento de Red y Servicios

Nmap (Network Mapper) es una herramienta de código abierto desarrollada originalmente por Gordon Lyon en 1997 (Lyon, 2009) y mantenida actualmente por una comunidad global de contribuidores. A lo largo de más de dos décadas de evolución, Nmap se convirtió en el estándar de facto para el descubrimiento de hosts y la detección de servicios en redes informáticas. La utilizan tanto administradores de sistemas, en tareas de gestión y auditoría, como profesionales de la seguridad ofensiva y defensiva en contextos de evaluación autorizada.

La herramienta opera enviando paquetes de red especialmente contruidos a los hosts objetivo y analizando las respuestas recibidas para inferir información sobre el estado de los puertos (abierto, cerrado, filtrado o sin filtrar), los servicios en ejecución y sus versiones, el sistema operativo del host y otras características de la configuración de red. En el contexto del presente sistema, Nmap se emplea en su modalidad de detección de versiones de servicios (-sV), que permite identificar qué servicio y qué versión concreta opera en cada puerto abierto. Esta información alimenta la etapa de selección de herramientas: a partir de los servicios HTTP/HTTPS descubiertos, el pipeline decide sobre qué puerto ejecutar las herramientas de análisis de aplicaciones web.

Una característica de particular relevancia para el presente trabajo es la capacidad de Nmap para exportar sus resultados en formato XML mediante el parámetro -oX, que preserva toda la riqueza informativa de los resultados de manera estructurada y parseable. En el sistema implementado, el adaptador de Nmap invoca la herramienta con la salida XML dirigida a la salida estándar (-oX -) y aplica sobre ella un parser propio que extrae, para cada puerto abierto, su número, protocolo, nombre de servicio, producto y versión, produciendo la lista de servicios que alimenta las etapas posteriores.

8.4 Herramientas de Análisis de Aplicaciones Web: WhatWeb, Nikto, Nuclei y OWASP ZAP

A diferencia del descubrimiento de servicios de red, el análisis de vulnerabilidades de aplicaciones web requiere herramientas especializadas capaces de interactuar con el protocolo HTTP y con la lógica específica de las aplicaciones. El sistema integra cuatro herramientas complementarias, cada una con un dominio de detección diferenciado.

WhatWeb es una herramienta de fingerprinting de tecnologías web que identifica el stack sobre el que se construye una aplicación: servidor web, lenguaje, framework, sistema de gestión de

contenidos, bibliotecas JavaScript, cabeceras de seguridad y otros componentes. No detecta vulnerabilidades directamente, sino que produce el contexto tecnológico que orienta el análisis posterior. En el sistema implementado, WhatWeb se invoca con salida JSON estructurada (`--log-json=-`) en modo silencioso y sin color; estas dos últimas opciones son necesarias, no cosméticas, porque, sin ellas, WhatWeb intercala su reporte legible coloreado en el mismo flujo de salida estándar que el JSON, corrompiéndolo. Sus resultados alimentan la tabla de tecnologías del sistema.

Nikto (Sullo y Lodge, 2023) es un escáner de servidores web orientado a la detección de configuraciones inseguras, archivos y scripts peligrosos, versiones de software desactualizadas y otros problemas de configuración a nivel de servidor. Ejecuta miles de comprobaciones sobre el servidor objetivo. En el sistema, Nikto se invoca solicitando su reporte en formato JSON escrito a un archivo (no a la salida estándar): se confirmó empíricamente que su modo de salida a stdout no produce un JSON válido de forma fiable, quedando la ejecución bloqueada sin generar resultados utilizables hasta que finaliza el escaneo. Adicionalmente, se acota su duración mediante el parámetro `-maxtime`, dado que su suite completa de comprobaciones puede extenderse mucho más de lo útil para un objetivo de laboratorio. Nikto no aporta, por sí mismo, una puntuación de severidad ni referencias CVE, por lo que sus hallazgos reciben un tratamiento conservador en la etapa de clasificación (ver Sección 11).

Nuclei, desarrollada por ProjectDiscovery, es una herramienta de escaneo basada en plantillas (templates): definiciones declarativas en formato YAML, mantenidas por una comunidad activa, que describen cómo detectar una vulnerabilidad, exposición o configuración incorrecta concreta. Cada coincidencia de una plantilla lleva asociada una etiqueta de severidad (info, low, medium, high, critical) y, cuando corresponde, una clasificación que puede incluir identificadores CVE y métricas CVSS. En el sistema, Nuclei se invoca con salida en formato JSON Lines (`-jsonl`), en modo silencioso y con la comprobación de actualizaciones deshabilitada (`-duc`); las plantillas se descargan una única vez durante la construcción de la imagen del contenedor, de modo que ninguna ejecución de escaneo paga el costo de una actualización en caliente. Nuclei es la principal fuente de hallazgos con severidad nativa y referencias CVE del sistema.

OWASP ZAP (Zed Attack Proxy) es la herramienta de referencia de la OWASP Foundation para el análisis dinámico de aplicaciones web. A diferencia de las anteriores, ZAP realiza un análisis activo: primero rastrea (spider) la aplicación para descubrir sus recursos y parámetros, y luego ejecuta un escaneo activo que ataca cada parámetro descubierto con cargas construidas para revelar vulnerabilidades como inyección, cross-site scripting y otras del OWASP Top 10. En el sistema, ZAP se ejecuta en su modo de línea de comandos de una sola pasada (`-cmd -quickurl ... -quickout ...`), que lanza el proceso, realiza el análisis y escribe el reporte JSON, en lugar de operar como un demonio de larga duración gobernado por su API. Esta decisión mantiene a ZAP consistente con el modelo de “un proceso por escaneo” del resto de los adaptadores. Cada alerta de ZAP incluye un riskcode numérico (0 a 3) que el sistema mapea directamente a su escala de severidad. Por la naturaleza de su análisis activo, ZAP es la herramienta más lenta del pipeline, un escaneo completo puede tomar varios minutos incluso sobre una aplicación pequeña, comportamiento inherente a su técnica y no un defecto.

La combinación de estas cuatro herramientas con Nmap proporciona una cobertura estratificada: descubrimiento de servicios (Nmap), contexto tecnológico (WhatWeb), configuración del servidor (Nikto), vulnerabilidades conocidas por plantilla (Nuclei) y vulnerabilidades de lógica de aplicación mediante ataque activo (ZAP).

Conviene, sin embargo, tratar esa cobertura estratificada como una **hipótesis sobre el diseño y no como una propiedad demostrada del conjunto de herramientas**. La afirmación contiene en realidad dos supuestos distintos, y ambos son verificables. El primero es que cada herramienta cubre una capa que las demás no cubren, es decir, que su aporte es complementario y no redundante. El segundo es que la suma de esas capas alcanza efectivamente las clases de vulnerabilidad que interesan, en particular las de lógica de aplicación, que son las que un catálogo de verdad de referencia registra.

Ninguno de los dos supuestos se sostiene por construcción. Dos herramientas pueden estar reportando el mismo hallazgo bajo nombres distintos, en cuyo caso la estratificación es aparente. Una herramienta de análisis activo puede no alcanzar la superficie donde reside la vulnerabilidad que en teoría le corresponde detectar, en cuyo caso la capa existe en el diseño pero no en el resultado. **Este trabajo somete ambos supuestos a medición**, mediante el análisis de solapamiento entre herramientas y la anotación del conjunto de verdad de referencia, y encuentra que se cumplen de forma desigual según el objetivo y la condición de acceso. Las Secciones 12.5 y 15.2 desarrollan ese resultado; se lo anticipa aquí para que la noción de cobertura estratificada no se lea, en lo que sigue, como una premisa aceptada.

El alcance de la observación de caja negra. Las cuatro herramientas anteriores comparten un supuesto operativo que conviene hacer explícito, porque delimita por construcción lo que el conjunto puede encontrar. Todas infieren la presencia de una vulnerabilidad a partir de la respuesta que el servidor devuelve a una petición construida por ellas mismas. Ese modo de observación alcanza lo que el servidor expone por sí solo: cabeceras ausentes, archivos accesibles, versiones declaradas o parámetros que responden de forma anómala a una carga. No alcanza aquello cuya evidencia aparece solo después de recorrer un flujo de la aplicación.

Tres condiciones colocan a una vulnerabilidad del otro lado de esa frontera. La primera es el estado de sesión: si la superficie vulnerable exige autenticación, un escaneo no autenticado no llega a ella. La segunda es la interacción de varios pasos, como subir un archivo y después invocarlo, o enviar un formulario cuyo efecto solo se observa en otra página. La tercera es la superficie construida en el cliente: cuando la interfaz se genera mediante JavaScript, un rastreador que no ejecuta ese código no descubre los recursos que debería analizar (OWASP Foundation, 2024).

La consecuencia metodológica pesa más que la técnica. Si el instrumento observa un subconjunto de la superficie, el recall que mida no describe la capacidad del sistema frente a la aplicación, sino frente a la porción de la aplicación que el instrumento alcanza. Toda cifra de cobertura obtenida por esta vía debe leerse con ese condicionamiento declarado, y no como una medida de la seguridad del objetivo. Las Secciones 12.5 y 15.5 retoman este punto con medición propia.

8.5 Estándares CVE y CVSS

La gestión efectiva de vulnerabilidades requiere un lenguaje común y estandarizado que permita identificar, clasificar y comunicar las debilidades detectadas sin ambigüedad. Con este propósito, la comunidad de seguridad ha desarrollado dos estándares complementarios que se han convertido en referencias universales del sector: CVE y CVSS.

El CVE (Common Vulnerabilities and Exposures) es una base de datos pública mantenida por la organización MITRE Corporation (MITRE Corporation, 2024), que asigna identificadores únicos y estandarizados a vulnerabilidades de seguridad conocidas en software y hardware. Cada entrada CVE contiene un identificador único con formato CVE-AÑO-NÚMERO (por ejemplo, CVE-2021-44228, la célebre vulnerabilidad Log4Shell), una descripción de la naturaleza de la vulnerabilidad, y referencias a fuentes adicionales de información. Este sistema de identificación unívoca permite que distintas herramientas, plataformas, organizaciones e investigadores se refieran a la misma vulnerabilidad sin ambigüedades. El National Vulnerability Database (NVD) del NIST enriquece el catálogo CVE con puntuaciones CVSS, clasificaciones CWE y otras métricas. En el presente sistema, las referencias CVE se extraen principalmente de las plantillas de Nuclei que las declaran, y se almacenan en una tabla propia dado que un mismo hallazgo puede asociarse a varios CVE.

El CVSS (Common Vulnerability Scoring System), mantenido actualmente por el FIRST (FIRST.Org, 2019), es un estándar abierto para la evaluación cuantitativa de la severidad de las vulnerabilidades. En su versión 3.1, calcula una puntuación numérica entre 0 y 10 basándose en métricas de base, temporales y ambientales. Las puntuaciones base se clasifican en rangos cualitativos: ninguna/informativa (0,0), baja (0,1-3,9), media (4,0-6,9), alta (7,0-8,9) y crítica (9,0-10,0). El sistema implementado adopta exactamente estos umbrales en su módulo de clasificación de severidad: cuando una herramienta reporta una puntuación CVSS numérica, el sistema la traduce al nivel cualitativo correspondiente según estos rangos. Cuando reporta una etiqueta textual (como Nuclei) o un código numérico propio (como el riskcode de ZAP), el sistema la mapea a la misma escala unificada. De este modo, hallazgos provenientes de herramientas heterogéneas quedan clasificados bajo un criterio común y comparable. Cabe señalar que CVSS no es el único estándar de priorización disponible. El Exploit Prediction Scoring System (EPSS, Jacobs et al., 2021) complementa la severidad técnica de CVSS con una estimación probabilística de explotación dentro de los primeros 12 meses posteriores a la divulgación pública. El catálogo Known Exploited Vulnerabilities (KEV) de CISA añade evidencia de explotación confirmada. El presente sistema no incorpora estas señales en esta ronda, limitación que se retoma en la Sección 14.2.

Ahora bien, adoptar CVSS como criterio de clasificación no es una decisión neutral, y conviene explicitar la discusión que la rodea antes de construir un sistema sobre ella. **CVSS mide severidad técnica intrínseca, no riesgo.** Su puntuación base describe cuán grave sería la explotación de una vulnerabilidad si llegara a producirse, y deliberadamente no incorpora cuán probable es que se produzca. La consecuencia práctica es conocida y motivó el desarrollo de alternativas: una vulnerabilidad puntuada 9,8 que ningún actor explota ordena por delante de una puntuada 5,3 que se explota de forma activa, cuando para decidir qué remediar primero el orden debería ser el inverso.

Es esa insuficiencia la que da origen al Exploit Prediction Scoring System (EPSS, Jacobs et al., 2021), que complementa la severidad técnica con una estimación probabilística de explotación

real dentro de los doce meses posteriores a la divulgación, y al catálogo Known Exploited Vulnerabilities (KEV) de CISA, que aporta evidencia de explotación confirmada en lugar de una estimación. Ninguno de los dos reemplaza a CVSS: lo corrigen en la dimensión que CVSS declara explícitamente no cubrir. El sistema aquí presentado no incorpora estas señales, decisión de alcance que se retoma en la Sección 14.2, y que implica que su clasificación ordena por gravedad potencial y no por urgencia de remediación.

Existe además una segunda dificultad, más específica del problema que este trabajo aborda, y que la literatura de estándares no resuelve porque no se le plantea. **CVSS supone una fuente única que puntúa según un método conocido, y un sistema que consolida cinco herramientas heterogéneas no está en esa situación.** Nuclei declara una etiqueta cualitativa propia, ZAP un código numérico de cuatro niveles, Nikto ninguna señal de severidad, y Nmap y WhatWeb no producen hallazgos de seguridad en sentido estricto. Homogeneizar esas salidas bajo una escala común exige decisiones que ningún estándar dicta: qué hacer con una herramienta que no puntúa, cómo proyectar una escala de cuatro niveles sobre una de cinco, y qué prioridad asignar cuando una misma herramienta reporta a la vez una etiqueta y una puntuación numérica que discrepan.

La consecuencia conceptual merece enunciarse con claridad, porque condiciona la lectura de los resultados de este trabajo: **una severidad normalizada no es una propiedad observada del objetivo, sino una construcción del sistema que la normaliza.** Las reglas concretas adoptadas aquí se detallan en la Sección 11.4, y sus efectos sobre la distribución obtenida se cuantifican y discuten en las Secciones 12.5 y 15.3. Presentar esa distribución como si describiera únicamente al objetivo analizado, y no en parte al criterio con que se lo clasificó, sería un error de interpretación que este trabajo procura no cometer.

8.6 Automatización de Procesos de Seguridad

La automatización de procesos de seguridad informática ha evolucionado desde la simple programación de tareas mediante scripts y cron jobs hacia el desarrollo de plataformas sofisticadas de orquestación que permiten integrar decenas de herramientas y servicios dentro de flujos de trabajo complejos, con capacidad de toma de decisiones condicionales, manejo de errores y ramificaciones dinámicas. Este campo se enmarca dentro del concepto de Security Orchestration, Automation and Response (SOAR), acuñado por Gartner para describir plataformas capaces de integrar y coordinar herramientas de seguridad heterogéneas, automatizar respuestas a incidentes y gestionar flujos de trabajo de investigación y remediación.

Los sistemas SOAR permiten definir playbooks (secuencias predefinidas de acciones automatizadas que se desencadenan en respuesta a determinados eventos, alertas o condiciones) que coordinan la acción de múltiples herramientas de seguridad. Un principio de diseño central en las implementaciones maduras de este paradigma es la separación entre la orquestación y la lógica de negocio: el orquestador decide qué se ejecuta y en qué orden, pero la lógica concreta de cada acción reside en componentes especializados e independientes. Este principio es precisamente el que estructura la arquitectura del presente sistema, en el que n8n coordina el flujo pero no contiene ninguna lógica de escaneo, normalización o clasificación: todas ellas viven en servicios propios versionados. La literatura reciente respalda la pertinencia de este

enfoque: la revisión sistemática de Mohammed et al. (2025) sitúa la orquestación de cadenas de herramientas y su integración fluida entre las brechas abiertas del área, y Marandi et al. (2023) documentan la incorporación de escaneo de seguridad automatizado a un pipeline de integración continua, disparado por cambios en el código antes que por objetivo desplegado.

8.7 Orquestación mediante n8n

n8n es una plataforma de automatización de workflows de código fuente disponible (source-available), lanzada en 2019 (n8n GmbH, 2024). A diferencia de plataformas como Zapier o Make, n8n puede ser desplegada en infraestructura propia (self-hosted) mediante Docker, lo que garantiza que todos los datos procesados, incluyendo resultados de escaneos de seguridad, permanezcan dentro del entorno controlado de la organización. Este requisito es fundamental en contextos de seguridad donde la exfiltración de información sobre vulnerabilidades a servicios de terceros es inaceptable.

La arquitectura de n8n se basa en el concepto de nodo, que representa una unidad funcional capaz de ejecutar una acción específica. Los nodos se conectan entre sí formando grafos dirigidos acíclicos (DAGs) que definen la lógica del workflow. La plataforma gestiona automáticamente la transferencia y serialización de datos entre nodos y el manejo de errores.

En el presente trabajo, y en coherencia con el principio de separación entre orquestación y lógica descrito en la sección anterior, el tipo de nodo de mayor relevancia es el nodo **HTTP Request**: cada etapa del pipeline se implementa como una llamada HTTP a uno de los servicios propios (el Backend o el Servicio de Escaneo), que son los que contienen la lógica real. El sistema utiliza además dos tipos de trigger complementarios —un nodo **Webhook** que actúa como punto de entrada de producción, invocado por el Backend, y un nodo **Form** que permite ejecutar el pipeline manualmente con fines de demostración— y nodos de tipo **Code/Set** para tareas menores de resolución de contexto y selección del puerto HTTP sobre el que operar. Es importante subrayar que n8n no ejecuta directamente ninguna herramienta de escaneo mediante nodos de comando del sistema operativo: toda ejecución de Nmap, WhatWeb, Nikto, Nuclei o ZAP ocurre dentro del Servicio de Escaneo, invocado por HTTP.

8.8 Arquitectura de Microservicios y Contenedores

La arquitectura de microservicios es un estilo de diseño de software en el que una aplicación se estructura como un conjunto de servicios pequeños, independientes y débilmente acoplados, cada uno responsable de una capacidad de negocio específica y comunicados entre sí mediante contratos bien definidos, típicamente APIs HTTP/REST. Frente al estilo monolítico, este enfoque favorece el desarrollo, despliegue y escalado independiente de cada componente, la resiliencia ante fallos localizados y la posibilidad de emplear tecnologías distintas en cada servicio. Estudios empíricos recientes respaldan esta elección de estilo arquitectónico desde la perspectiva de la seguridad: Nasab et al. (2023) relevan prácticas de seguridad efectivamente adoptadas en sistemas de microservicios reales, y Mills et al. (2023) realizaron una evaluación longitudinal de seguridad basada en riesgo sobre imágenes de contenedor Docker reales, la misma tecnología de contenerización que emplea la plataforma (Docker Inc., 2023), relevante tanto para las decisiones de diseño de la plataforma (Docker, servicios separados) como para el análisis de riesgos de la Sección 13.

El presente sistema adopta este estilo de manera deliberada. Cada responsabilidad del pipeline (centralización de la lógica de negocio y persistencia (Backend), ejecución de las herramientas (Servicio de Escaneo), generación de documentos (Servicio de Reportes) y orquestación (n8n)) reside en un servicio independiente. Dos de estos servicios (Escaneo y Reportes) se diseñan explícitamente sin acoplamiento a la base de datos: no la consultan ni dependen de su esquema, lo que los hace triviales de probar de forma aislada y de razonar. Conviene ser preciso en la formulación: el Servicio de Reportes no es estrictamente sin estado, ya que persiste los archivos que renderiza en un volumen propio y los sirve en solicitudes posteriores (ver Sección 11.5). Lo que efectivamente se logra, y es lo que sostiene la simplicidad de ambos servicios, es la ausencia de acoplamiento a la base de datos. El Backend es el único servicio con estado y la única superficie de API pública del sistema.

El despliegue reproducible de esta arquitectura se apoya en la contenerización mediante Docker y su orquestación local mediante Docker Compose. Cada servicio se empaqueta en una imagen de contenedor con sus dependencias, y un único archivo de definición (`docker-compose.yml`) describe el conjunto completo de servicios, sus redes, volúmenes persistentes, dependencias de arranque y comprobaciones de salud (healthchecks). Esta aproximación permite levantar la totalidad del sistema con un único comando y sin pasos de configuración manuales, garantizando que el entorno sea idéntico en cualquier máquina que disponga de Docker —una propiedad de particular valor en un contexto académico donde la reproducibilidad del entorno de evaluación es un requisito metodológico.

Complementariamente, el sistema aplica el patrón de **adaptador** para integrar las herramientas de escaneo, y un **registro** (registry) que asocia el nombre de cada herramienta con su adaptador, de modo que agregar una herramienta nueva no requiere modificar el código existente sino únicamente añadir una clase adaptadora y registrarla. El mismo patrón se replica en la capa de normalización. Este uso de patrones de diseño es uno de los aportes de ingeniería de software del trabajo, y se describe en detalle en la Sección 11.

8.9 La Medición de la Cobertura: Recall, Precisión y Catálogo de Verdad de Referencia

El capítulo de resultados evalúa la cobertura del sistema mediante recall, precisión y F1 contra un catálogo de verdad de referencia. Conviene fundamentar ese instrumento antes de aplicarlo, porque las tres métricas son menos transparentes de lo que su aritmética sugiere y porque la lectura correcta de las cifras del Capítulo 12 depende de entender qué miden exactamente.

Las tres se construyen sobre la clasificación de cada hallazgo producido por el escáner respecto de un conjunto de referencia previamente establecido. Un verdadero positivo es un hallazgo que se corresponde con una entrada del catálogo; un falso positivo, uno que no se corresponde con ninguna; un falso negativo, una entrada del catálogo que el escáner no reportó. El recall es la proporción de entradas del catálogo efectivamente detectadas y responde a la pregunta por la cobertura; la precisión es la proporción de hallazgos reportados que se corresponden con una entrada y responde a la pregunta por el ruido; F1 es su media armónica, y se emplea cuando ninguna de las dos por separado describe adecuadamente el desempeño. Amankwah et al. (2020) construyen sobre ese mismo aparato su comparación empírica de ocho escáneres web, lo que permite situar los valores de este trabajo en una escala conocida.

Tres advertencias condicionan su interpretación, y las tres son consecuencia de una misma propiedad: ninguna de las métricas mide una capacidad absoluta del escáner, sino su desempeño relativo a un catálogo construido por alguien. La primera es que el catálogo fija el denominador del recall. Un recall de 0,27 no significa que el sistema detecte el 27 % de las vulnerabilidades de la aplicación, sino el 27 % de las entradas que el catálogo enumera; catálogos distintos sobre el mismo objetivo producen recalls distintos sin que el instrumento haya cambiado. Por esa razón este trabajo construye sus catálogos desde las fuentes oficiales de cada aplicación objetivo y publica el procedimiento (Anexo E), de modo que el denominador quede expuesto a inspección.

La segunda es que la correspondencia entre un hallazgo y una entrada del catálogo no es un hecho observable sino una decisión, y admite criterios de distinta exigencia. Emparejar por identificador CVE exacto, por tipo de vulnerabilidad y ubicación, o por coincidencia de palabra clave en el título, produce recuentos crecientes de verdaderos positivos sobre exactamente los mismos datos. Un criterio laxo infla el recall sin que el escáner haya detectado nada adicional. En consecuencia, un recall reportado sin su criterio de correspondencia no es interpretable, y este trabajo reporta el suyo desagregado por nivel de confianza en la Sección 12.5.

La tercera es que la precisión depende de qué se acepte como verdadero positivo, y ese umbral es discutible para los hallazgos de carácter informativo, que constituyen una fracción considerable de la salida de varias de las herramientas integradas. Un hallazgo informativo correcto que no corresponde a ninguna entrada del catálogo se contabiliza como falso positivo aunque no sea un error de la herramienta, sino una diferencia de propósito entre lo que la herramienta reporta y lo que el catálogo enumera. Las cifras de precisión de la Sección 12.5 deben leerse con esa salvedad.

La consecuencia metodológica converge con la que el apartado 8.4 deriva del alcance de la observación de caja negra, y conviene enunciarla una sola vez: lo que estas métricas miden es la relación entre dos construcciones —la porción de superficie que el instrumento alcanza y la porción de vulnerabilidades que el catálogo enumera— y no una propiedad de la aplicación analizada. Es una limitación de constructo en los términos de Wohlin et al. (2012), y el trabajo la trata como tal: declara el catálogo, declara el criterio de correspondencia y reporta las métricas desagregadas por herramienta y por nivel de confianza, en lugar de presentar un valor único cuya composición quedaría oculta.

9. Metodología

9.1 Enfoque y Diseño del Proyecto

El presente trabajo adopta un enfoque metodológico de tipo experimental-aplicado, enmarcado en el paradigma cuantitativo de investigación científica. Se trata de una investigación orientada al desarrollo de un artefacto tecnológico concreto, la plataforma automatizada de análisis de vulnerabilidades, cuyo desempeño se evalúa objetivamente mediante pruebas controladas en un entorno de laboratorio reproducible. Esta clasificación es coherente con la taxonomía propuesta por Wieringa (2014), que distingue entre investigación de diseño orientada a la construcción de artefactos y la investigación empírica orientada a la evaluación de su comportamiento.

El diseño del proyecto se estructura en torno a un ciclo iterativo de construcción, integración y evaluación inspirado en los principios del desarrollo ágil de software, y organizado en módulos incrementales. En cada módulo se implementa un componente del sistema (el esquema de base de datos, el laboratorio de objetivos, el Backend, el Servicio de Escaneo, la capa de normalización, la orquestación en n8n, el Servicio de Reportes), se verifica su funcionamiento correcto de manera aislada, y a continuación se integra con los componentes ya existentes manteniendo en todo momento un sistema desplegable mediante docker compose up -d. Este enfoque incremental permite detectar y resolver problemas de compatibilidad e integración de manera temprana.

Conviene denominar el diseño por lo que efectivamente se ejecutó y no por lo que se previó, porque la diferencia entre ambos es de naturaleza y no sólo de magnitud. Lo realizado es una investigación de diseño en los términos de Wieringa (2014) —construcción de un artefacto y evaluación empírica de su comportamiento— acompañada de dos observaciones cronometradas del proceso manual de referencia, que funcionan como punto de comparación y no como brazo de control con capacidad inferencial. El diseño originalmente previsto sí era de tipo cuasi-experimental, de comparación entre dos condiciones aplicadas al mismo objeto de laboratorio: una condición de control (proceso manual) y una condición experimental (sistema automatizado), a cargo en ambos casos de los autores del sistema. La ausencia de aleatorización no responde a la naturaleza determinista del entorno, sino a la existencia de una sola unidad de análisis y de un conjunto de operadores reducido al equipo autor: no hay una población entre la que sortear la asignación de condiciones. De ahí que el planteo original hablara de diseño cuasi-experimental y no de experimento puro (la Sección 9.8 declara la unidad de análisis, el tamaño y el criterio de selección). En esta ronda de evaluación se instanciaron ambas condiciones, aunque con tamaños que no admiten el mismo tratamiento: la experimental con quince corridas del sistema automatizado repartidas en tres campañas, y la de control con dos corridas del proceso manual, ejecutadas por los propios autores (Sección 9.6). Esa asimetría es la razón por la que las cifras comparativas de la Sección 12.4 se reportan como razón entre medias de un par de observaciones y no como una diferencia con fuerza estadística, y por la que el trabajo no reclama para ellas el estatus de resultado experimental. La validez interna del estudio se apoya en el control de las condiciones del entorno de laboratorio dentro de cada campaña y en la definición precisa del protocolo de evaluación descrito en la sección 9.6; entre campañas, en cambio, cuatro variables quedaron fuera de control, como declaran la Sección 9.3 y la Sección 12.6.3 (ver también Sección 9.9, amenazas a la validez).

9.2 Tipo de Investigación y Paradigma Epistemológico

Desde el punto de vista epistemológico, el presente trabajo se inscribe en el paradigma positivista-empirista que caracteriza a la investigación cuantitativa en ingeniería de sistemas. El conocimiento producido se basa en la medición objetiva de variables operacionalizadas (tiempos de ejecución, número de vulnerabilidades detectadas, variabilidad entre ejecuciones) y en el análisis comparativo de los datos obtenidos mediante procedimientos replicables. Este paradigma es el predominante en la investigación aplicada en ingeniería de software y sistemas, donde la validación de soluciones tecnológicas mediante evidencia empírica cuantificable es el estándar metodológico de referencia.

La dimensión aplicada del proyecto implica que el conocimiento generado no es de naturaleza puramente teórica, sino que está orientado a resolver un problema concreto con implicaciones prácticas para la organización académica en la que se enmarca. En este sentido, el proyecto se alinea con la tradición de la investigación de diseño en ingeniería, en la que el proceso de construcción y evaluación del artefacto tecnológico es en sí mismo una fuente de conocimiento sobre el dominio de aplicación. Wieringa (2014) formaliza esta posición al distinguir entre los problemas de diseño, que se resuelven construyendo un artefacto, y las preguntas de conocimiento, que se responden mediante investigación empírica. El presente trabajo contiene ambos tipos y los mantiene deliberadamente separados, como muestra la distinción entre el componente constructivo y el componente evaluativo del objetivo general (Sección 4.1).

9.3 Variables del Estudio

Con el propósito de evaluar objetivamente el desempeño del sistema desarrollado y contrastar las hipótesis formuladas, se definen con precisión las siguientes variables del estudio:

Variable independiente principal: el método de análisis de vulnerabilidades empleado, que toma dos valores posibles según el diseño original. (a) Proceso manual, ejecutado por un operador humano con conocimiento de las herramientas involucradas. (b) Sistema automatizado, implementado en el presente trabajo. Esta variable es el factor experimental cuyo efecto sobre las variables dependientes se busca cuantificar. En esta ronda de evaluación se instanciaron ambos valores: el sistema automatizado en quince ejecuciones repartidas en tres campañas, y el proceso manual en dos corridas cronometradas, una por cada autor (Sección 12.2 y Anexo K), cuyo alcance declara la Sección 14.3.

Variables dependientes. (a) Tiempo total de ejecución del ciclo completo de análisis, medido en minutos desde el inicio del descubrimiento de servicios hasta la disponibilidad del informe generado. (b) Tiempo de generación del informe de seguridad, medido en segundos desde la finalización del escaneo hasta la disponibilidad del documento. (c) Recall contra un catálogo de verdad de referencia construido desde las fuentes oficiales de Juice Shop y DVWA (Anexo E), como medida de cobertura, y número de hallazgos por ejecución como medida adicional de consistencia. El número de hallazgos no es, por sí solo, una medida válida de cobertura: un escáner que reporta cien falsos positivos no cubre más que uno que reporta diez hallazgos correctos. (d) Coeficiente de variación del tiempo total entre ejecuciones sucesivas, como medida de la consistencia y reproducibilidad del proceso. (e) Estado de completitud de cada ejecución, indicando si el pipeline completó todas sus etapas sin errores.

Variable de control: dentro de cada campaña se mantienen constantes el entorno de laboratorio contenerizado, las imágenes de contenedor de las aplicaciones objetivo y sus versiones, los parámetros de invocación de cada herramienta y las plantillas de Nuclei embebidas en la imagen del Servicio de Escaneo. Entre campañas, en cambio, cuatro variables quedan fuera de control y conviene enumerarlas: la máquina anfitriona, las versiones de las herramientas de escaneo — que la imagen del Servicio de Escaneo resuelve en tiempo de construcción sin fijarlas—, las imágenes de las aplicaciones objetivo y del orquestador, que se resolvían por la etiqueta latest, y el código del propio sistema, ya que la rama principal avanzó entre una captura y la siguiente. La variación observada entre las campañas A y C se documenta en las Secciones 12.6.3 y 14.1, y la correspondencia entre cada campaña y el estado del código, en la Sección 20.

9.4 Entorno de Laboratorio

El sistema fue implementado y evaluado en un entorno de laboratorio contenerizado construido específicamente para el presente trabajo, diseñado para proporcionar un conjunto conocido y reproducible de aplicaciones web deliberadamente vulnerables. A diferencia de un laboratorio basado en máquinas virtuales, el entorno se define íntegramente mediante Docker Compose, de modo que la totalidad de los objetivos y los servicios de la plataforma se levantan con un único comando sobre cualquier equipo que disponga de Docker, sin instalación ni configuración manual de las aplicaciones objetivo.

El laboratorio se compone de dos aplicaciones web vulnerables ampliamente reconocidas en la formación en seguridad, desplegadas como contenedores en una red Docker segmentada (lab-network) aislada del resto de la plataforma:

Tabla 2

Aplicaciones web vulnerables del entorno de laboratorio

Aplicación objetivo	Imagen	Puerto host	Propósito
OWASP Juice Shop	bkimminich/juice-shop:latest	3000	Aplicación web moderna (Node.js/Angular) con vulnerabilidades del OWASP Top 10 deliberadas. Proyecto oficial de la OWASP Foundation.
DVWA	vulnerables/web-dvwa:latest	3001	Aplicación PHP/MySQL clásica de entrenamiento en vulnerabilidades web (inyección SQL, XSS, CSRF, etc.), con niveles de dificultad configurables.

Nota. Elaboración propia, a partir de docs/lab.md.

Ambas imágenes son proyectos de código abierto creados específicamente para la formación en seguridad ofensiva; ninguna contiene datos reales ni se conecta a servicios externos para operar. OWASP Juice Shop no requiere pasos de configuración: queda operativa apenas su contenedor supera la comprobación de salud. DVWA, en cambio, requiere que su base de datos MySQL se cree explícitamente —normalmente mediante un botón en su página setup.php—; para preservar el requisito de despliegue sin pasos manuales, el sistema incluye un contenedor auxiliar de inicialización de una sola ejecución (dvwa-init) que reproduce automáticamente la petición HTTP de creación de la base de datos tras el primer arranque, con reintentos hasta confirmar el éxito.

El entorno de máquina anfitriona sobre el que se ejecutaron las mediciones se documenta en la siguiente tabla:

Tabla 3

Especificación del entorno de la máquina anfitriona

Componente	Especificación
Máquina anfitriona	CPU AMD Ryzen 7 5700U with Radeon Graphics / RAM 31,4 GB / Disco (partición del sistema) 475,7 GB
Sistema operativo	Microsoft Windows 11 Home Single Language 10.0.26200
Motor de contenedores	Docker version 29.0.1, build eedd969 / Docker Compose v2.40.3-desktop.1
Máquina anfitriona B/C	CPU Apple M5 (10 núcleos) / RAM 16,0 GB / Disco (partición del sistema) 460 GB
Sistema operativo B/C	macOS 26.6 (arquitectura arm64)
Motor de contenedores B/C	Docker version 29.4.0, build 9d7ad9f / Docker Compose v5.1.2

Nota. Elaboración propia, capturado programáticamente durante las campañas de medición. Las filas "B/C" corresponden al anfitrión de las campañas B y C y se capturaron el 2026-08-24 (Sección 12.6); el resto, al de la campaña A, capturado el 2026-08-11.

9.5 Herramientas y Tecnologías Utilizadas

Tabla 4

Herramientas y tecnologías utilizadas, con versión y licencia

Herramienta / Tecnología	Versión	Función en el Sistema	Licencia
n8n	latest (self-hosted)	Orquestación del pipeline mediante llamadas HTTP	Sustainable Use License
Nmap	7.95	Descubrimiento de servicios y detección de versiones	GPL v2
WhatWeb	0.5.5	Fingerprinting de tecnologías web	GPL v3

Herramienta / Tecnología	Versión	Función en el Sistema	Licencia
Nikto	2.6.0 (LW 2.5)	Detección de configuraciones inseguras en servidores web	GPL v2
Nuclei (ProjectDiscovery)	v3.11.0 (motor) / v10.4.7 (plantillas)	Escaneo de vulnerabilidades basado en plantillas	MIT
OWASP ZAP	2.17.0	Análisis dinámico activo de aplicaciones web (DAST)	Apache 2.0
Python	3.12	Lenguaje de los servicios propios	PSF License
FastAPI	0.115	Framework web de Backend, Scanner y Reports	MIT
SQLAlchemy	2.0	ORM y capa de acceso a datos	MIT
Alembic	1.13.2	Migraciones de esquema de base de datos	MIT
PostgreSQL	16	Almacenamiento estructurado de resultados	PostgreSQL License
Jinja2 + WeasyPrint	3.1 / 62.3	Renderizado de reportes HTML/PDF	BSD / BSD
Docker / Docker Compose	Docker 29.0.1 / Docker Compose v2.40.3-desktop.1	Contenerización y orquestación local	Apache 2.0

Nota. Elaboración propia, versiones capturadas en vivo contra el contenedor Scanner (2026-08-11). Las versiones consignadas son las vigentes en la campaña A (2026-08-11). En las campañas B y C (2026-08-24) dos de ellas habían cambiado sin intervención, por efecto de la resolución en tiempo de construcción: Nikto 2.6.1 y Nuclei v3.11.1 (motor). La Sección 12.6.3 analiza la consecuencia de ese cambio sobre las cifras de cobertura.

Versiones capturadas en vivo contra el contenedor Scanner en ejecución (docker compose exec scanner <tool> --version), no contra los pines del Dockerfile: el binario de Nuclei y su set de plantillas se descargan de la release latest de GitHub en build-time, así que la única versión reproducible es la que efectivamente corrió durante esta campaña de medición (2026-08-11).

9.6 Procedimiento de Evaluación y Protocolo Experimental

El protocolo experimental de evaluación fue diseñado para garantizar la comparabilidad de las mediciones entre el proceso manual y el sistema automatizado, minimizando las fuentes de

variabilidad que podrían sesgar los resultados. El procedimiento se estructura en cuatro fases secuenciales.

Fase 1 — Establecimiento de la línea base: Antes de iniciar las mediciones, se levanta el entorno completo mediante `docker compose up -d` y se verifica, a través de las comprobaciones de salud de cada servicio, que la totalidad de la plataforma y los objetivos del laboratorio estén operativos. Se documenta la versión de cada herramienta y el estado de las plantillas de Nuclei embebidas. Esta verificación se realiza inmediatamente antes de cada sesión de medición para garantizar la invarianza de las condiciones.

Fase 2 — Medición del proceso manual: el protocolo, documentado en detalle en el Anexo K, se ejecutó dos veces, el 26 de agosto y el 23 de septiembre de 2026, sobre el mismo objetivo (dvwa) y el mismo anfitrión y versiones de herramienta que la campaña C (Sección 12.6). Cada uno de los autores ejecutó el proceso manualmente por separado, cronometrando etapa por etapa. (a) Ejecución de Nmap sobre el objetivo y lectura de sus resultados. (b) Ejecución de WhatWeb e interpretación del fingerprint. (c) Ejecución de Nikto, Nuclei y ZAP sobre el servicio HTTP, con espera hasta su finalización. (d) Lectura y consolidación manual de los resultados de las tres herramientas de detección. (e) Clasificación manual de la severidad de cada hallazgo. (f) Elaboración manual del informe de seguridad. Los resultados se reportan en la Tabla 10 (Sección 12.2). El sesgo de familiaridad de que ambos operadores fueran autores del sistema se declara y se discute en la Sección 9.9.

Fase 3 — Medición del sistema automatizado: El pipeline de n8n se ejecuta en cinco ocasiones consecutivas sobre el mismo objetivo del laboratorio, bajo condiciones idénticas. Los tiempos de ejecución de cada herramienta quedan registrados automáticamente por el Backend (tabla `scan_tasks`) en cada corrida del pipeline, y el tiempo total del ciclo se mide desde la activación del trigger hasta la disponibilidad del reporte generado. Se registran además el número de hallazgos identificados y su distribución por severidad. El recuento de servicios y tecnologías por corrida y el tiempo aislado de generación del informe no quedaron capturados por este esquema y se consignan N/D en la Tabla 11, como advierte la nota metodológica del Capítulo 12. Como fuente de verdad complementaria a los logs de n8n, se consulta directamente la base de datos (tablas `scans`, `scan_tasks`, `findings`) que registra las marcas temporales de inicio y fin de cada etapa.

Fase 4 — Análisis de los datos: los datos de tiempo del sistema automatizado se organizan en tablas y se analizan mediante estadísticas descriptivas (media aritmética, rango de variación y coeficiente de variación = $\text{desviación estándar} / \text{media} \times 100$). La fórmula de reducción porcentual originalmente prevista, $\text{Reducción (\%)} = ((\text{Tiempo manual promedio} - \text{Tiempo automatizado promedio}) / \text{Tiempo manual promedio}) \times 100$, se aplica con reservas: el brazo manual se cronometró dos veces, de modo que existe un promedio, pero dos observaciones no sostienen una reducción porcentual con fuerza estadística. La comparación se reporta, en consecuencia, como razón entre la media manual y la media automatizada, y no como una reducción porcentual con intervalo (Tabla 13, Sección 12.4). A ella se suma la descomposición interna del pipeline automatizado entre ejecución de

herramientas y orquestación, que no depende de un operador humano y es la comparación con mayor fuerza metodológica de las dos.

9.7 Criterios de Validez y Confiabilidad

La validez interna del estudio es asegurada mediante el control estricto de las variables de confusión potenciales: el entorno de laboratorio contenerizado se mantiene invariante dentro de cada campaña, con la salvedad de que las versiones de las herramientas de escaneo no se fijan en la imagen y variaron entre las campañas A y C (Sección 12.6.3). Los parámetros de invocación de cada herramienta son idénticos en las quince ejecuciones del sistema automatizado. En la condición de control (proceso manual, Sección 12.2), el operador siguió en todo momento la secuencia de pasos documentada en el Anexo K.

La confiabilidad de las mediciones se refuerza mediante la repetición de la condición experimental (sistema automatizado) en cinco ejecuciones sucesivas por campaña. La condición de control (proceso manual) se ejecutó dos veces, una por cada autor, de modo que sus cifras admiten una comparación entre operadores pero no un coeficiente de variación con fuerza estadística y se reportan como observación puntual (Sección 12.2). El coeficiente de variación calculado sobre esas mediciones repetidas (4,1% para el tiempo total del pipeline, Sección 12.3) es descriptivamente bajo, pero con $n=5$ no aporta por sí solo evidencia estadística de estabilidad. Wohlin et al. (2012) advierten precisamente contra esta forma de amenaza a la validez de conclusión: extraer inferencias sobre la relación entre tratamiento y resultado a partir de muestras cuyo tamaño no sostiene la prueba estadística invocada. Por esa razón el coeficiente se reporta como estadístico descriptivo y no se adopta ningún umbral porcentual como criterio de aceptación, dado que ninguna fuente metodológica consultada sostiene un valor de corte válido para un tamaño muestral tan reducido (ver Sección 9.9 sobre las amenazas a la validez que esto implica).

La validez externa del estudio, es decir, la generalizabilidad de los resultados a entornos distintos del laboratorio, es limitada. Las características específicas del entorno —número y tipo de aplicaciones objetivo, capacidad de cómputo de la máquina anfitriona— pueden diferir de las de producción en órdenes de magnitud, tanto en número de objetivos como en la superficie que expone cada uno. Esta limitación se aborda con mayor detalle en la Sección 14.

9.8 Población, Muestra y Muestreo

El universo de este estudio son las aplicaciones web deliberadamente vulnerables utilizables como objetivo de un pipeline automatizado de escaneo en un laboratorio contenerizado y reproducible. La unidad de análisis es una ejecución completa del pipeline (un escaneo) contra un objetivo de laboratorio, con el operador, humano o automatizado, y el protocolo de invocación mantenidos constantes entre ejecuciones.

El muestreo es no probabilístico, de tipo intencional. Se trata de una elección estándar en los estudios de caso en ingeniería de software, donde la selección de los objetos de estudio responde a su pertinencia respecto de la pregunta de investigación y no a un procedimiento aleatorio (Runeson y Höst, 2009). No existe aquí una población de objetivos de laboratorio de la cual extraer una muestra aleatoria, sino una selección deliberada de dos aplicaciones: OWASP Juice Shop y DVWA. Se eligieron por tres criterios explícitos. (a) Ser proyectos de código abierto ampliamente reconocidos en la formación en seguridad ofensiva. (b) Contar con un catálogo público de vulnerabilidades documentadas, condición necesaria para construir la

verdad de referencia del Anexo E. (c) No contener datos reales ni conectarse a servicios externos, preservando el aislamiento del laboratorio (Sección 10.2).

El tamaño de la muestra es de quince ejecuciones del pipeline automatizado, repartidas en tres campañas de cinco corridas consecutivas cada una, diseñadas de modo que cada una aísla una variable distinta. La campaña A (2026-08-11, DVWA, anfitrión con procesador AMD Ryzen 7 5700U bajo Windows 11) establece la línea de base. La campaña B (2026-08-24, OWASP Juice Shop, anfitrión con procesador Apple M5 bajo macOS) introduce un objetivo distinto y permite evaluar si los resultados dependen de la superficie analizada. La campaña C (2026-08-24, DVWA, mismo anfitrión que la B) conserva el objetivo de la campaña A, de modo que la diferencia entre ambas no proviene de la superficie analizada; no aísla, sin embargo, una variable única, porque entre las dos capturas cambiaron también el anfitrión, las versiones de herramienta, las imágenes del laboratorio y el código del sistema (Secciones 9.3 y 20). Ninguna de las tres es acumulable con las otras, y el alcance de cada comparación se detalla en la Sección 12.6. A ello se suman ejecuciones puntuales de verificación empleadas en el análisis de cobertura (Sección 9.3(c), Anexo E). La condición de control (proceso manual) se ejecutó dos veces, el 26 de agosto y el 23 de septiembre de 2026, una por cada autor, bajo el mismo anfitrión y las mismas versiones de herramienta que la campaña C, para que la comparación con el sistema automatizado no quede además confundida por esas dos variables (Sección 12.6). El tamaño de cinco repeticiones no responde a un cálculo de potencia estadística (no es esa la pretensión de este estudio, ver Sección 9.9) sino a un balance entre la duración de cada corrida (del orden de cuatro minutos, dominada por Nikto en la campaña A y por Nuclei en la C, según detalla la Sección 12.6.1) y la necesidad de observar la consistencia del sistema entre ejecuciones repetidas.

9.9 Amenazas a la Validez

Además de las limitaciones generales discutidas en la Sección 14, cinco amenazas específicas al diseño experimental de esta evaluación merecen tratamiento explícito en esta sección metodológica. Se las enuncia siguiendo la clasificación de amenazas a la validez de Wohlin et al. (2012), que distingue entre validez interna, externa, de constructo y de conclusión. El sesgo de espera activa y el de familiaridad del operador son amenazas de validez interna, y la confusión de variables de la condición autenticada lo es de constructo. El tamaño muestral compromete la validez de conclusión, y el alcance del laboratorio afecta a la validez externa. Sesgo de espera activa. El brazo manual se cronometró registrando por separado el tiempo atendido —en el que el operador ejecuta una acción— y el tiempo total de reloj, que incluye la espera a que cada herramienta termine (Sección 12.2). Contabilizar esa espera íntegramente contra el operador manual habría inflado artificialmente la condición manual frente a la automatizada, porque ese tiempo de ejecución es un piso común a ambas condiciones: ningún operador, humano o automatizado, puede acelerar a Nmap, WhatWeb, Nikto, Nuclei o ZAP. La comparación de la Sección 12.4 se construye, por eso, sobre el tiempo atendido y no sobre el tiempo total. Este riesgo motivó, en parte, reformular PM-1 (Sección 6) para medir solo el componente de orquestación del sistema automatizado en aislamiento, sin necesidad de esa comparación cronometrada.

Sesgo de familiaridad del operador. Las dos corridas de la condición de control las ejecutaron los propios autores del sistema, con conocimiento profundo de las herramientas involucradas. Repetir la medición con un segundo autor aumenta el tamaño de la muestra, pero no elimina este sesgo: ambos operadores construyeron la herramienta que cronometran. El sesgo que eso introduce es de dirección indeterminada. Por un lado, la familiaridad pudo reducir el tiempo manual respecto al de un operador sin experiencia, comprimiendo la diferencia observable. Por otro, un analista con práctica rutinaria en un contexto profesional real podría ejecutar el mismo proceso todavía más rápido que el registrado por un autor-operador. El diseño de este estudio

no controla esa fuente de sesgo: haría falta repetir la cronometría con operadores externos al equipo, como propone la Sección 17.

Tamaño muestral reducido. Las quince ejecuciones del sistema automatizado, repartidas en tres campañas de cinco corridas no acumulables entre sí (Sección 9.8), no alcanzan el tamaño necesario para inferencia estadística paramétrica ni para el cálculo de intervalos de confianza (ver Sección 14.3). Los estadísticos descriptivos reportados en la Sección 12 deben interpretarse como observaciones preliminares, no como evidencia con fuerza probatoria independiente.

Validez externa acotada al laboratorio. El entorno de laboratorio comprende dos aplicaciones deliberadamente vulnerables y dos máquinas anfitrionas, según la campaña, sobre un total de quince ejecuciones. Esas características limitan la generalizabilidad de los resultados a entornos de producción reales, con mayor diversidad de objetivos y de capacidad de cómputo (ver Sección 9.7).

Confusión de variables en la condición autenticada. El modo de escaneo autenticado descrito en la Sección 12.5 no se limita a iniciar sesión: el procedimiento implementado autentica contra DVWA y, a continuación, fija su nivel de seguridad en su valor más permisivo antes de invocar las herramientas. La condición autenticada difiere entonces de la no autenticada en dos variables simultáneas, presencia de sesión y nivel de seguridad de la aplicación, de modo que la mejora de cobertura observada no puede atribuirse limpiamente a la autenticación. Fijar el nivel de seguridad es necesario para que los módulos vulnerables se comporten como el catálogo de verdad de referencia describe, pero esa necesidad no elimina el problema de diseño: lo medido es el efecto conjunto de ambas intervenciones. Aislarlas exigiría una tercera condición, autenticada y con el nivel de seguridad inalterado, no ejecutada en esta ronda.

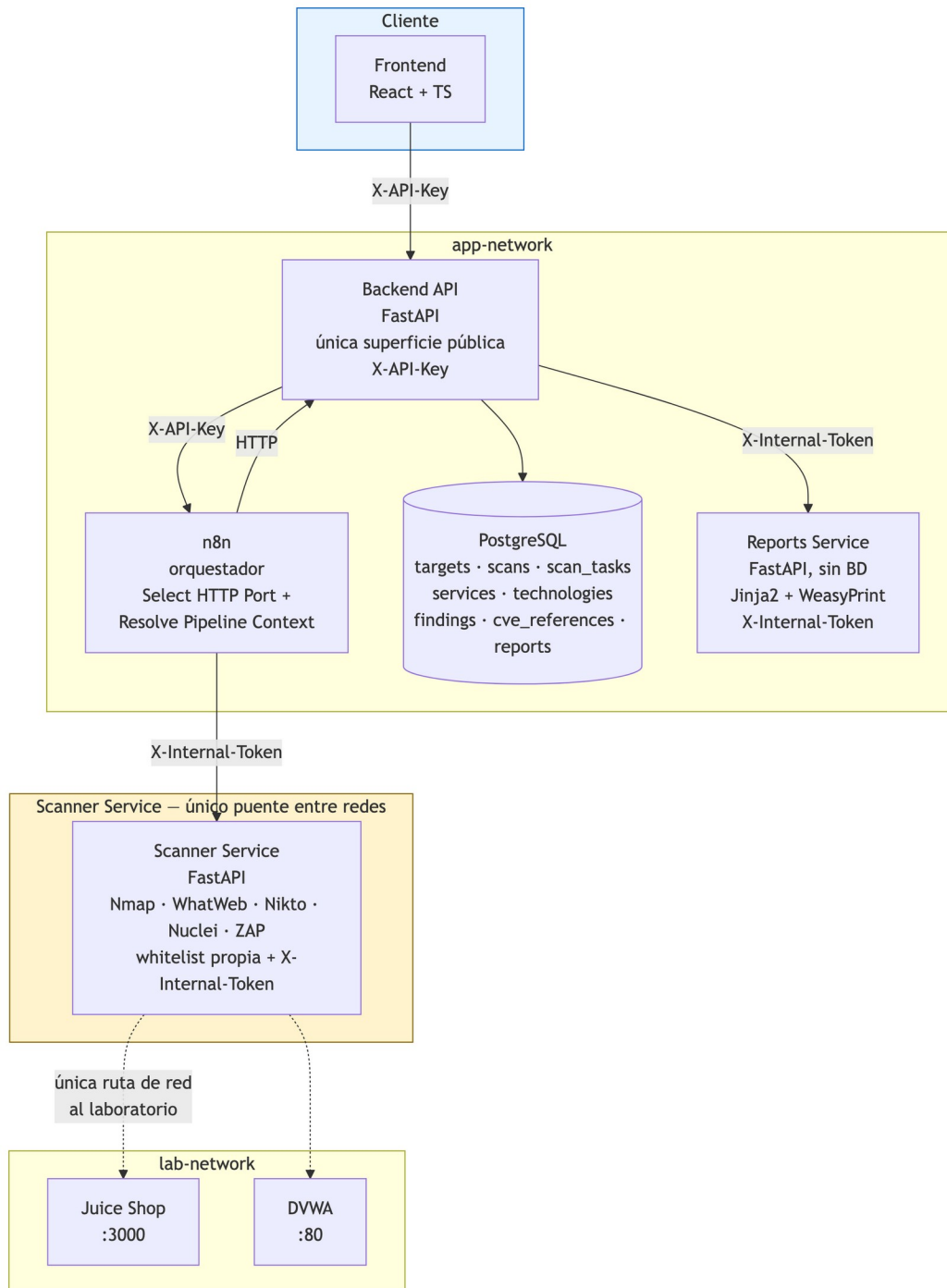
10. Arquitectura del Sistema

10.1 Descripción General

La arquitectura del sistema, denominado VulnScan Platform, está concebida como un conjunto de microservicios desacoplados que colaboran para ejecutar el ciclo completo de análisis de vulnerabilidades, desde el descubrimiento de servicios hasta la generación del informe. Cada servicio cumple una responsabilidad específica y bien delimitada, se comunica con los demás exclusivamente mediante contratos HTTP, y puede evolucionar o sustituirse sin afectar al resto del sistema. La Figura 1 resume el flujo general:

Figura 1

Diagrama de componentes (C4 nivel 2) de VulnScan Platform



Nota. Elaboración propia.

Los componentes son:

Frontend (React + TypeScript): tablero web servido por Nginx, única interfaz gráfica del sistema. Consume la API pública del Backend con el header X-API-Key (inlineada en tiempo de build) para registrar objetivos, disparar pipelines, visualizar hallazgos con filtro por severidad y descargar reportes en los cuatro formatos.

- **Backend API (FastAPI):** única superficie de API pública y único servicio con estado además de la base de datos. Centraliza la lógica de negocio: registro y validación de objetivos, gestión del ciclo de vida de los escaneos, ingesta y normalización de los resultados, clasificación de severidad y orquestación de la generación de reportes.
- **Scanner Service (FastAPI, sin estado):** ejecuta las herramientas de escaneo mediante un patrón de adaptadores. Es el único servicio que se conecta simultáneamente a la red de la plataforma (app-network) y a la red del laboratorio (lab-network); ningún otro componente tiene ruta hacia los objetivos vulnerables.
- **Reports Service (FastAPI, sin acoplamiento a la base de datos):** genera los informes en cuatro formatos a partir de una carga de datos autocontenida que recibe del Backend. No accede a la base de datos ni conoce a los demás servicios, aunque sí persiste los archivos que renderiza en un volumen propio (ver Sección 8.8).
- **n8n (orquestador): coordina el pipeline mediante llamadas HTTP a los servicios anteriores. La lógica de ejecución, normalización y clasificación reside en servicios propios; n8n conserva la resolución de contexto y la selección de puerto.**
- **PostgreSQL + migrador Alembic:** persistencia estructurada del ciclo de vida completo. Un contenedor migrador de una sola ejecución aplica el esquema al arranque.
- **Laboratorio (lab-network): las aplicaciones objetivo (Juice Shop, DVWA) y el inicializador de DVWA.**

10.2 Segmentación de Red y Modelo de Seguridad (Defensa en Profundidad)

Una decisión arquitectónica central es la separación en dos redes Docker. La red app-network conecta entre sí a los servicios de la plataforma (Backend, Scanner, Reports, n8n, base de datos). La red lab-network alberga exclusivamente las aplicaciones objetivo. El único componente conectado a ambas redes es el Scanner Service: en consecuencia, ni el Backend, ni n8n, ni la base de datos, ni un eventual frontend tienen ruta de red alguna hacia los objetivos vulnerables.

La plataforma aplica la whitelist de hosts de laboratorio en dos puntos independientes: el Backend, al registrar un target (POST /targets, administrativo, una sola vez); y el Scanner Service, en cada invocación de escaneo (POST /scan/{tool_name}, hasta 5 veces por pipeline). El segundo control es el que efectivamente importa, porque el Scanner es el único componente con ruta de red hacia lab-network —un chequeo único en el Backend, que ni siquiera puede alcanzar el laboratorio, no constituiría defensa en profundidad real por sí solo. Ambos puntos leen el mismo valor de configuración (BACKEND_ALLOWED_LAB_HOSTS/SCANNER_ALLOWED_LAB_HOSTS, una sola política mantenida en .env), aplicado con independencia en dos capas de código distintas —el principio de defensa en profundidad, ahora real. Cabe destacar una decisión técnica deliberada: la red del laboratorio no se marca como internal, pese a que ello impediría además la salida a internet de los contenedores objetivo, porque se comprobó empíricamente que Docker omite la publicación de puertos hacia el host para redes marcadas como internas, dejando las aplicaciones inaccesibles desde el navegador.

10.3 Componentes del Sistema y Puertos

Tabla 5

Componentes del sistema y sus puertos internos

Componente	Puerto interno	Función Principal
Backend API	TCP 8000 (HTTP)	API pública, lógica de negocio, ingesta y normalización
Scanner Service	TCP 8100 (HTTP)	Ejecución de las herramientas de escaneo (adaptadores)
Reports Service	TCP 8200 (HTTP)	Generación de reportes PDF/HTML/Markdown/JSON
n8n	TCP 5678 (HTTP)	Orquestación del pipeline
PostgreSQL	TCP 5432	Almacenamiento estructurado y persistente
OWASP Juice Shop	TCP 80	Aplicación objetivo del laboratorio
DVWA	TCP 80 (host 3001)	Aplicación objetivo del laboratorio

Nota. Elaboración propia.

Cada servicio declara una comprobación de salud (healthcheck) en la definición de Docker Compose, y las dependencias de arranque se expresan explícitamente: el Backend espera a que la base de datos esté sana y a que el migrador haya finalizado con éxito; n8n espera a que el Backend esté sano. Esto garantiza un orden de arranque correcto sin intervención manual.

10.4 Flujo de Datos

El flujo de datos sigue una secuencia determinista. El proceso se inicia cuando el Backend recibe una solicitud de ejecución del pipeline para un objetivo registrado (POST /targets/{id}/pipeline): el Backend crea una fila de scan en la base de datos y notifica al trigger Webhook de n8n con el identificador del escaneo, el del objetivo y su host, respondiendo de inmediato sin esperar a que el pipeline termine.

A partir de allí, n8n coordina las etapas invocando por HTTP al Scanner Service y al Backend. Para el reconocimiento, solicita al Scanner la ejecución de Nmap sobre el objetivo y remite el resultado al Backend, que lo normaliza a filas de la tabla services. Con la misma mecánica ejecuta WhatWeb, cuyos resultados alimentan la tabla technologies. A continuación, un paso de selección determina, a partir de los servicios descubiertos por Nmap, el puerto HTTP sobre el que operarán las herramientas de aplicación web. Sobre ese puerto se ejecutan secuencialmente Nikto, Nuclei y ZAP; el resultado de cada una se envía al Backend, que en una única transacción crea la fila de scan_task correspondiente y, cuando el resultado trae salida estructurada, la normaliza a filas de findings —con su severidad clasificada— y cve_references. Finalizados los escaneos, n8n solicita al Backend marcar el escaneo como completado y generar el reporte en

PDF, que el Backend produce reuniendo los datos y delegando el renderizado al Servicio de Reportes. Un nodo final consolida el resultado de la ejecución.

La comunicación es asíncrona respecto del cliente que dispara el pipeline: dado que el Backend responde en cuanto notifica a n8n, el progreso se consulta posteriormente mediante los endpoints GET /scans/{id} y GET /scans/{id}/findings.

11. Implementación Técnica

11.1 Configuración del Entorno de Despliegue

El sistema se despliega mediante Docker Compose, que define la totalidad de los servicios en un único archivo, con sus redes, volúmenes persistentes, dependencias de arranque y comprobaciones de salud. Este enfoque cumple un requisito de diseño explícito del proyecto: `docker compose up -d` debe dejar el sistema completo operativo, sin ningún paso de configuración manual posterior. Las únicas variables que el operador debe proveer son las del archivo `.env` (credenciales de base de datos, configuración de n8n, lista blanca de hosts de laboratorio y URLs internas de los servicios), que se excluye del repositorio mediante `.gitignore` y del que se provee una plantilla `.env.example`.

El código fuente completo del sistema (los cinco servicios, la configuración de despliegue y los artefactos referenciados en el Anexo F y siguientes) está publicado en https://github.com/lauroCast/Ciberseguridad_Prog2026. El estado descrito en este trabajo corresponde al commit 0357df2 de la rama `main`, marcado con la etiqueta `tesis-entrega`; los datos crudos de las mediciones se publican bajo la etiqueta `tesis-datos`, según detalla la Sección 20.

Un principio transversal en la configuración de los servicios es la validación estricta al arranque (`fail-fast`). Los ajustes sensibles o críticos para el funcionamiento no tienen valor por defecto: la URL de la base de datos, la lista blanca de hosts de laboratorio, la URL del webhook de n8n y la del Servicio de Reportes. Si faltan o están vacíos, el servicio correspondiente falla al arrancar con un mensaje explicativo, en lugar de arrancar aparentemente sano y fallar después en cada solicitud sin causa evidente. Esta decisión previene una clase completa de errores de configuración silenciosos.

El esquema de base de datos se aplica mediante un contenedor migrador de una sola ejecución, construido a partir del directorio `database/`, que ejecuta las migraciones de Alembic (`alembic upgrade head`) y finaliza. Los servicios que dependen del esquema declaran esta dependencia explícitamente, de modo que no arrancan hasta que la migración ha concluido con éxito. Es relevante notar que el Backend reutiliza las mismas clases de modelo ORM que Alembic migra, no redefine el esquema,: por ello, la imagen del Backend se construye desde la raíz del repositorio, incorporando tanto su propio código como los modelos compartidos.

11.2 El Backend y su Arquitectura en Capas

El Backend, desarrollado en FastAPI, es la única superficie de API pública y concentra la lógica de negocio del sistema. Su código se organiza en capas con una dependencia estrictamente unidireccional:

- **Enrutadores (routers):** capa HTTP. Solo traducen solicitudes y respuestas; no contienen lógica de negocio.
- **Servicios (services):** reglas de negocio. Aquí viven, por ejemplo, la aplicación de la lista blanca de laboratorio, la orquestación de la ingesta y normalización, y la generación de reportes.
- **Repositorios (repositories):** consultas a la base de datos, sin reglas.
- **Esquemas (schemas):** modelos Pydantic de solicitud y respuesta.

Las excepciones de dominio que la capa de servicios levanta (por ejemplo, “objetivo no encontrado”, “host no permitido por la whitelist”, “conflicto de nombre de objetivo”) se traducen a respuestas HTTP en un único lugar, manejadores de excepción registrados una sola vez en el punto de entrada de la aplicación, en lugar de repetirse en cada enrutador. Todo recurso nuevo reutiliza este mecanismo simplemente levantando sus propias excepciones. Esta organización refleja directamente los principios de separación de responsabilidades expuestos en el marco teórico y es uno de los aportes de ingeniería del trabajo.

Los endpoints principales expuestos por el Backend son:

Tabla 6
Endpoints principales del Backend

Método	Ruta	Descripción
GET	/health	Liveness + verificación de conectividad con la base de datos
GET / POST	/targets	Listado y registro de objetivos (el host debe estar en la whitelist)
GET / PATCH / DELETE	/targets/{id}	Consulta, actualización y baja de un objetivo
POST	/targets/{id}/scans	Inicia un escaneo sobre un objetivo
POST	/targets/{id}/pipeline	Dispara el pipeline completo de n8n; responde 202 de inmediato
GET	/scans/{id}	Consulta el estado de un escaneo
POST	/scans/{id}/complete	Marca un escaneo como completado/fallido
POST	/scans/{id}/tasks	Ingesta el resultado de una herramienta y lo normaliza en una transacción
GET	/scans/{id}/findings	Lista los hallazgos normalizados de un escaneo
POST / GET	/scans/{id}/reports	Genera y lista reportes de un escaneo
GET	/reports/{id}/download	Descarga el archivo del reporte (proxy al Servicio de Reportes)

Nota. Elaboración propia.

11.3 El Servicio de Escaneo y el Patrón de Adaptadores

El Servicio de Escaneo es un servicio FastAPI sin estado cuya responsabilidad es ejecutar las herramientas de escaneo y devolver su salida cruda y, cuando es posible, estructurada. Su diseño se organiza en torno al patrón de adaptadores: cada herramienta se implementa como una clase que hereda de una interfaz común (ScannerAdapter) e implementa dos métodos — build_command(), que construye la línea de comandos a ejecutar, y parse_output(), que estructura la salida cruda—. Un registro (registry) asocia el nombre de cada herramienta con su instancia de adaptador. El único endpoint de escaneo, POST /scan/{tool_name}, es genérico: despacha al adaptador correspondiente a través del registro y no necesita modificarse para agregar una herramienta nueva. La lógica compartida de ejecución de subprocessos (medición de tiempos, gestión de timeouts, captura de salida por stdout o por archivo) reside una única vez en un ejecutor común (scan_runner), no duplicada por adaptador.

La siguiente tabla resume, para cada herramienta, la invocación real que construye su adaptador y su rol en el pipeline:

Tabla 7

Invocación real de cada herramienta de escaneo por su adaptador

Herramienta	Invocación (resumida)	Salida	Rol
Nmap	nmap -sV -T4 -oX -	XML por stdout	Descubrimiento de servicios y versiones
WhatWeb	whatweb --log-json=- -q --color=never -a <n>	JSON por stdout	Fingerprinting de tecnologías
Nikto	nikto.pl -Format json -output <archivo> -maxtime <t>	JSON por archivo	Configuraciones inseguras del servidor
Nuclei	nuclei -u <url> -jsonl -silent -duc [-severity/-tags]	JSON Lines por stdout	Vulnerabilidades por plantilla + CVE
OWASP ZAP	zap.sh -cmd -quickurl <url> -quickout <archivo>.json	JSON por archivo	Análisis dinámico activo (DAST)

Nota. Elaboración propia.

Dos de los adaptadores (Nikto y ZAP) escriben su reporte a un archivo en lugar de a la salida estándar; la interfaz común contempla este caso mediante un indicador (uses_output_file) que instruye al ejecutor a crear un archivo temporal, pasárselo a la herramienta y leer de él la salida. Cada uno de estos detalles de invocación responde a un comportamiento verificado empíricamente durante la construcción del servicio, documentado en el código. WhatWeb exige suprimir el color para no corromper el JSON, y Nikto no emite JSON válido por stdout. Las plantillas de Nuclei, por su parte, se descargan al construir la imagen para no penalizar la primera ejecución.

11.4 Normalización y Clasificación de Severidad

El punto en que un resultado específico de una herramienta se convierte en datos estructurados y homogéneos es el endpoint de ingesta del Backend (POST /scans/{id}/tasks). La normalización se organiza, nuevamente, mediante un registro que asocia el nombre de cada herramienta con una función normalizadora. Cada normalizador recibe la salida ya parseada de una herramienta y produce únicamente estructuras de datos planas (dataclasses) de tres tipos (servicios, tecnologías y hallazgos). La conversión de esas estructuras a filas de la base de datos ocurre en un único lugar, lo que mantiene a los normalizadores libres de cualquier dependencia de base de datos y fácilmente testeables de forma aislada. Agregar soporte de normalización para una herramienta nueva consiste en añadir una función y una línea al registro.

El sistema es tolerante a fallos de normalización. Si el normalizador de una herramienta arroja un error ante una salida con una forma inesperada, la ingesta no falla: el resultado crudo se conserva, con el error anexado al registro de la tarea. La salida cruda tiene valor por sí misma aun cuando no pueda estructurarse. Asimismo, una herramienta sin normalizador registrado no interrumpe el flujo: su tarea se registra igualmente, simplemente sin producir filas de hallazgos hasta que se añada su normalizador.

La clasificación de severidad es específica por herramienta, dado que cada una ofrece una señal distinta, pero todas convergen a la misma escala unificada (info, low, medium, high, critical) mediante un módulo compartido:

- **Nuclei** aporta su propia etiqueta de severidad por coincidencia de plantilla, que se adopta directamente.
- **OWASP ZAP** aporta un riskcode numérico (0 a 3) por alerta, que se mapea directamente a la escala en lugar de intentar interpretar la cadena de texto legible.
- **Nikto** no aporta señal alguna de severidad ni CVSS: todos sus hallazgos se clasifican de forma conservadora como severidad baja, marcados para triaje manual en lugar de pre-clasificados por riesgo.
- **Nmap** y **WhatWeb** no producen hallazgos: alimentan las tablas de servicios y tecnologías respectivamente (datos de reconocimiento, no vulnerabilidades en sí mismas).

Cuando una herramienta reporta una puntuación CVSS numérica, el sistema la traduce al nivel cualitativo según los rangos de FIRST.org descritos en la Sección 8.5. El mapeo de CVE solo se realiza para las herramientas que los reportan (fundamentalmente Nuclei, a partir de la clasificación de sus plantillas), generando una fila por CVE en la tabla de referencias.

La clasificación de severidad no trata a las tres herramientas de detección por igual. La causa no está en el objetivo analizado sino en la política de mapeo de cada herramienta. ZAP aporta un riskcode de cuatro niveles (0-3), que se mapea a los cuatro inferiores de la escala de cinco del sistema: ningún hallazgo de ZAP puede clasificarse como crítico. Nikto no reporta severidad ni CVSS, de modo que todos sus hallazgos se fuerzan a “baja”, por decisión conservadora y deliberada. Sólo Nuclei, cuya plantilla declara severidad nativa, puede producir un hallazgo crítico. En consecuencia, la fila “Crítica” de la Sección 12.5 depende exclusivamente de qué encontró Nuclei, no de la severidad real agregada del objetivo. Cuando una herramienta reporta simultáneamente una etiqueta de severidad y una puntuación CVSS numérica, caso habitual en las plantillas de Nuclei, el sistema da precedencia incondicional a la etiqueta nativa de la herramienta. La puntuación CVSS se conserva con fines informativos y de trazabilidad, pero no participa en la clasificación de severidad.

11.5 El Servicio de Reportes

El Servicio de Reportes es un microservicio FastAPI sin acoplamiento a la base de datos que, dada una carga autocontenida (objetivo, escaneo y hallazgos) y un formato, renderiza el reporte a un archivo y devuelve su nombre. No tiene base de datos, no conoce identificadores de escaneo más allá de lo que recibe en la solicitud, y no invoca a ningún otro servicio. El Backend es su único cliente: reúne todos los datos por adelantado (reutilizando sus propios servicios de objetivos, escaneos y hallazgos) y los envía en una sola solicitud (un modelo de “empuje”, no de “consulta”), lo que mantiene al servicio simple de probar de forma aislada.

Requisito no funcional (Servicio de Reportes): el sistema genera un informe de seguridad en los cuatro formatos soportados (PDF, HTML, Markdown, JSON) en menos de 5 segundos desde que recibe el payload autocontenido, sin intervención manual. No se plantea como hipótesis contrastable (ver H3 retirada de la Sección 6) porque el mecanismo (una plantilla Jinja2 renderizada sobre algunas decenas de hallazgos, convertida a PDF con WeasyPrint) hace que el umbral esté garantizado por construcción: medirlo tiene valor como verificación de que el sistema cumple su contrato, no como evidencia de investigación.

El servicio soporta cuatro formatos, cada uno con su renderizador: HTML y PDF comparten una plantilla Jinja2 (el PDF se produce convirtiendo el HTML mediante WeasyPrint), mientras que Markdown y JSON tienen los suyos. Un detalle de seguridad relevante es el tratamiento del autoescapeado: los renderizadores de HTML y PDF activan el autoescapeado de Jinja2, porque los títulos y descripciones de los hallazgos provienen directamente de la salida de las herramientas de escaneo —un hallazgo literalmente titulado `<script>alert(1)</script>` es un resultado plausible de una plantilla orientada a XSS— y reflejarlo sin escapar en un reporte HTML/PDF constituiría una vulnerabilidad de XSS autoinfligida en el visor del reporte. El renderizador de Markdown, en cambio, no escapa, dado que el Markdown es texto plano y un literal escapeado sería incorrecto, no más seguro. Adicionalmente, el endpoint de descarga valida que el nombre de archivo solicitado, entrada controlada por el cliente, se resuelva dentro del directorio de reportes, de modo que un intento de path traversal se rechaza con un 404 en lugar de leer fuera del directorio previsto.

11.6 Orquestación en n8n

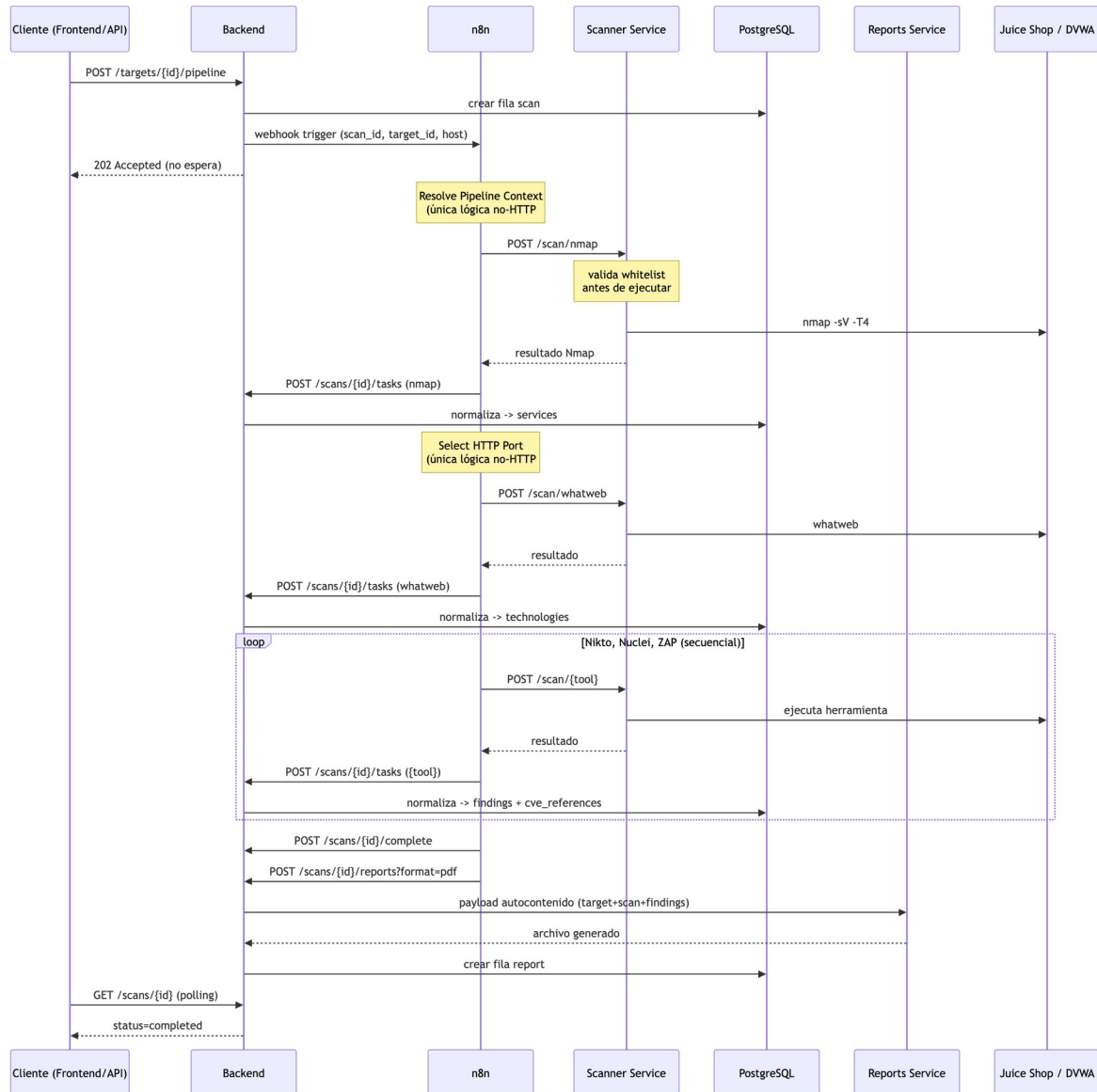
El workflow de n8n implementa el pipeline coordinando, exclusivamente por HTTP, a los servicios anteriores. La arquitectura conceptual del proceso se organiza en doce etapas, varias de las cuales se materializan en un único nodo porque los módulos previos ya las implementan como una operación atómica. El pipeline admite dos puntos de entrada que convergen en la misma cadena: un trigger **Webhook** —el camino de producción, invocado por el Backend, que ya validó el objetivo y creó la fila de escaneo— y un trigger **Form** —un camino manual para demostraciones, que ejecuta por sí mismo la validación y creación del escaneo dado que no dispone de uno precreado—. Ambas ramas se unifican en un nodo de resolución de contexto, tras el cual el resto de la cadena es idéntica.

La selección de la herramienta a aplicar se resuelve en un nodo que lee directamente la respuesta de Nmap y elige el primer puerto abierto cuyo servicio parezca HTTP(S). Conviene declarar el alcance de esa heurística: cuando el objetivo expone más de un servicio web —por ejemplo, una aplicación en el puerto 80 y un panel de administración en el 8080—, el nodo selecciona únicamente el primero que devuelve Nmap y los restantes quedan sin analizar, sin

que el sistema lo advierta. En el laboratorio esta limitación no se manifiesta, porque cada objetivo expone un solo servicio web, pero sobre un objetivo real implicaría una pérdida de cobertura silenciosa. Iterar sobre todos los puertos HTTP detectados, en lugar de sobre el primero, queda identificado como trabajo futuro (Sección 17). Las cuatro herramientas de aplicación (WhatWeb, Nikto, Nuclei, ZAP) se ejecutan secuencialmente, no en ramas paralelas: esto mantiene el workflow como una cadena lineal, sencilla de leer y depurar desde el editor, y es adecuado dado que la ejecución secuencial de las herramientas domina el tiempo total de todos modos. Conviene señalar que la medición de la Sección 12.3 matiza el supuesto original de este apartado: contra DVWA es Nikto, y no ZAP, la herramienta que más tiempo consume, por razones de configuración que allí se detallan. La generación del reporte se ejecuta con tolerancia a fallos (continueOnFail): un fallo en el reporte no debe enmascarar el hecho de que el escaneo en sí ya finalizó correctamente. Una consideración de despliegue relevante es que n8n no importa automáticamente el workflow al arrancar; el workflow, versionado como JSON en el repositorio, debe importarse una vez por volumen de datos de n8n y activarse desde la interfaz. La Figura 2 representa la secuencia completa de las doce etapas y el orden en que se encadenan.

Figura 2

Diagrama de secuencia de las 12 etapas del pipeline orquestado por n8n

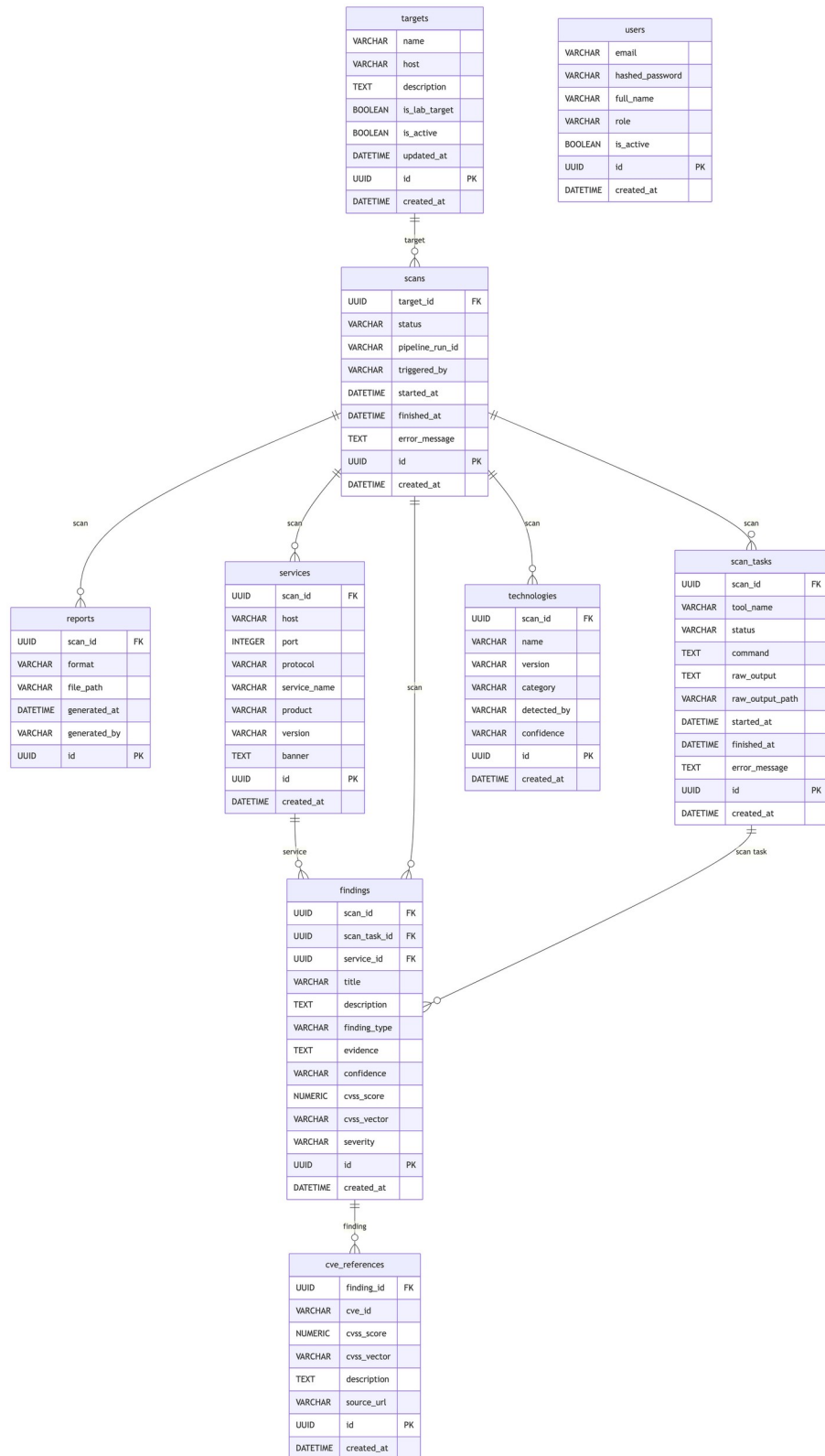


Nota. Elaboración propia.

11.7 Esquema de Base de Datos

El esquema de PostgreSQL persiste el ciclo de vida completo de un análisis: objetivo registrado → ejecución del pipeline → tareas por herramienta → servicios y tecnologías descubiertos → hallazgos normalizados → CVE asociados → reportes generados. Está definido mediante modelos de SQLAlchemy 2.0 (uno por tabla) y versionado con Alembic. La Figura 3 resume su estructura mediante un diagrama entidad-relación:

Figura 3
Diagrama entidad-relación del esquema de PostgreSQL



Nota. Elaboración propia, generado por introspección del código (database/models/*.py).

Tabla 8

Tablas del esquema de base de datos y su responsabilidad

Tabla	Responsabilidad
targets	Sistemas registrados para análisis. El flag is_lab_target materializa la whitelist de laboratorio.
scans	Una ejecución del pipeline sobre un objetivo. pipeline_run_id correlaciona la fila con la ejecución de n8n.
scan_tasks	Una invocación de una herramienta concreta dentro de un escaneo. tool_name es texto libre.
services	Puertos y servicios descubiertos durante el reconocimiento (Nmap).
technologies	Stack tecnológico identificado (WhatWeb).
findings	Hallazgo normalizado: forma común a la que se traduce la salida de cualquier herramienta. finding_type es texto libre.
cve_references	CVE asociados a un hallazgo (relación 0..N).
reports	Metadatos de un reporte generado; el archivo reside en un volumen propio del Servicio de Reportes.
users	Placeholder para una futura capa de autenticación multiusuario; no consumido en esta etapa.

Nota. Elaboración propia, generado por introspección de database/models/*.py.

Las decisiones de diseño más relevantes del esquema son cuatro. (a) **Claves primarias UUID** generadas por la base de datos (gen_random_uuid(), nativo de PostgreSQL, sin extensión), para no filtrar el volumen de filas a través de la API. (b) **Tipos enumerados nativos** de PostgreSQL únicamente para vocabularios cerrados y estables (scan_status, scan_task_status, severity_level, report_format, user_role). Todo lo que debe poder ampliarse sin migración (tool_name, finding_type) es una cadena de texto. (c) cve_references como **tabla propia** en lugar de una columna en findings, dado que un mismo hallazgo puede asociarse a varios CVE. (d) **Borrado en cascada** desde targets hacia abajo, coherente con un laboratorio donde los objetivos pueden recrearse. El detalle completo de columnas se incluye en el Anexo A.

11.8 Verificación Automatizada

El sistema se acompaña de una suite de 347 pruebas automatizadas distribuidas por servicio: 177 en el Backend, 76 en el Servicio de Escaneo, 38 en el Servicio de Reportes y 56 en el Frontend. A ellas se suman ocho pruebas de la utilidad de verdad de referencia (scripts/ground_truth), que quedan fuera de la ruta de descubrimiento declarada en pytest.ini y

se ejecutan por separado. Se ejecutan dentro de los propios contenedores mediante el script `scripts/test-all.sh`, y la integración continua definida en `.github/workflows/ci.yml` las corre en cada push y en cada pull request del repositorio.

El recuento de pruebas por sí solo dice poco sobre qué porción del código ejercita. La cobertura de sentencias, medida con `coverage.py` sobre los tres servicios en Python, permite acotarlo.

Tabla 9

Pruebas automatizadas y cobertura de sentencias por componente

Componente	Pruebas	Sentencias	Cobertura
Backend	177	901	97 %
Servicio de Escaneo	76	297	98 %
Servicio de Reportes	38	200	98 %
Frontend	56	—	—
Total	347	1.398	—

Nota. Elaboración propia, medida el 2026-09-22 sobre el commit 0357df2 (etiqueta tesis-entrega) ejecutando `pytest` con `coverage.py` dentro de cada contenedor. La cobertura del Frontend no se reporta porque su suite se ejecuta con Vitest sobre el anfitrión y emplea un instrumental distinto, no comparable con el de los servicios en Python.

Conviene interpretar estas cifras con la misma cautela que el resto del trabajo aplica a sus mediciones. La cobertura de sentencias indica qué proporción del código se ejecuta durante la suite, no que su comportamiento haya sido verificado: una sentencia cubierta por una prueba que no comprueba su resultado cuenta igual que una verificada con rigor. Es, por tanto, una cota superior de la verificación efectiva y no una medida de ella.

Dos decisiones de esta suite merecen mención porque responden a limitaciones reales del entorno de prueba y no a cobertura nominal. La primera: la suite rápida se ejecuta sobre SQLite en memoria, que no aplica los anchos de columna VARCHAR que sí aplica PostgreSQL, de modo que una clase entera de error, salida de herramienta que excede el ancho de una columna acotada, resultaría estructuralmente invisible. Por esa razón existe un marcador específico que ejecuta contra la base de datos efectiva las pruebas que requieren semántica real del motor, y que se omite automáticamente cuando esa base no está disponible. La segunda: existe además una prueba de integración de extremo a extremo, invocada mediante `scripts/test-e2e.sh`, que ejercita el pipeline completo contra el laboratorio real con el workflow de n8n importado y activo. La integración continua la ejecuta en un job separado por su duración.

La verificación automatizada es, en un trabajo de esta naturaleza, parte del resultado y no un detalle de implementación: es lo que permite afirmar que el comportamiento descrito en las secciones anteriores es el que el sistema efectivamente presenta, y no solo el que se pretendió construir.

11.9 Interfaz Web de Operación

La interfaz de operación es una aplicación React con TypeScript servida por Nginx, y es el único consumidor de la API del Backend además de n8n. No incorpora lógica de negocio propia:

registra objetivos, dispara el pipeline, sigue el avance de un escaneo y descarga los informes, delegando toda decisión al Backend. Su diseño responde a una restricción del dominio que conviene explicitar, porque condiciona la interacción: un escaneo completo tarda entre cuatro y seis minutos, de modo que la interfaz debe comunicar progreso durante una espera larga sin bloquear al operador. El seguimiento se resuelve con sondeo periódico sobre el estado del escaneo, y no con notificación desde el servidor, porque el Backend no mantiene conexión persistente con el cliente.

Figura 4

Pantalla de objetivos: registro y listado de las aplicaciones del laboratorio habilitadas por la whitelist

Objetivos 2 registrados

Aplicaciones del laboratorio habilitadas para escaneo. La whitelist del servidor define qué hosts pueden registrarse.

REGISTRAR OBJETIVO

Nombre: Host: juice-shop dvwa Descripción opcional:

Únicos hosts habilitados por la whitelist

REGISTRAR

NOMBRE	HOST	ESTADO	ÚLTIMO ESCANEO	
Juice Shop OWASP Juice Shop (SPA Angular)	juice-shop	activo	✓ 23-año, 08:03 p. m.	Ver detalle
DVWA Damn Vulnerable Web Application	dvwa	activo	✓ 22-año, 02:57 a. m.	Ver detalle

Nota. Elaboración propia.

La pantalla de objetivos (Figura 4) refleja en la interfaz la restricción de seguridad descrita en la Sección 10.2: el campo de host no admite texto libre, sino que ofrece únicamente los hosts declarados en la whitelist. Esta es una comodidad de la interfaz y no un control de seguridad. La validación efectiva ocurre en el Backend al registrar el objetivo y, sobre todo, en el Servicio de Escaneo antes de invocar cualquier herramienta, que es el único componente con ruta de red hacia el laboratorio.

Figura 5

Detalle de un objetivo: datos de registro, disparo del pipeline e historial de ejecuciones

DVWA • activo [Eliminar](#)

Damn Vulnerable Web Application

DATOS DEL OBJETIVO

Host: **dvwa**
Laboratorio: **sí - en whitelist**
Identificador: 1283e9b4-1e4e-4097-86c6-ed748bc99adc
Registrado: 22-año, 02:42 a. m.

EJECUTAR PIPELINE

Lanza las doce etapas contra dvwa: reconocimiento, fingerprinting, escaneo con las cinco herramientas, normalización y persistencia.

Tarda entre **4 y 6 minutos**. Podés cerrar la pestaña – sigue en segundo plano.

Correr pipeline

HISTORIAL DE ESCANEOS 2 ejecuciones

ESTADO	INICIO	DURACIÓN	DETALLE
✓ completado	22-año, 02:57 a. m.	2:18	—
✓ completado	22-año, 02:44 a. m.	1:23	—

Nota. Elaboración propia.

El detalle de un objetivo (Figura 5) concentra las dos operaciones que el sistema expone sobre él: lanzar una ejecución del pipeline y revisar el historial de las anteriores. La duración estimada se anuncia antes de disparar la ejecución, y el historial registra el estado y la duración efectiva de cada corrida, que son los datos sobre los que se construyó la medición de la Sección 12.3.

Figura 6

Generación de informes en los cuatro formatos desde la vista de un escaneo



Nota. Elaboración propia.

La generación de informes (Figura 6) se solicita desde la vista del escaneo y se resuelve contra el Servicio de Reportes por intermedio del Backend. Los cuatro formatos se producen a partir de la misma carga de datos, de acuerdo con el diseño descrito en la Sección 11.5, y la descarga se proxee a través del Backend porque los dos servicios no comparten sistema de archivos.

12. Resultados Obtenidos

Nota metodológica. Los valores numéricos de esta sección provienen de la ejecución real del pipeline sobre el entorno de laboratorio descrito en la Sección 9. Corresponden a tres campañas de cinco corridas cada una —A contra dvwa el 2026-08-11, y B contra juice-shop y C contra dvwa el 2026-08-24— y a las dos corridas manuales cronometradas, del 2026-08-26 y del 2026-09-23, que reporta la Sección 12.2. Los datos del pipeline se obtuvieron de las marcas temporales registradas en las tablas scans, scan_tasks y findings de la base de datos, expuestas por la API pública del Backend (GET /scans/{id}/tasks). No se reportan aquí valores estimados o hipotéticos: donde el esquema de medición no capturó un dato (recuento de servicios/tecnologías por corrida, tiempo aislado de generación del informe), la celda correspondiente se marca N/D en vez de completarse con una estimación, en coherencia con las consideraciones de integridad académica del trabajo. El criterio de redondeo adoptado es uniforme: los tiempos se reportan con dos decimales cuando se trata de totales o desviaciones, y con uno en los desgloses por herramienta; las proporciones y los coeficientes de variación, con un decimal. Las métricas de recall, precisión y F1 se reportan con dos, por derivar de denominadores de una o dos cifras. Los redondeos se aplican al presentar y no al calcular, de modo que las sumas de valores redondeados pueden diferir del total en el último decimal, como se señala en la Sección 12.3. El proceso manual de comparación (Sección 12.2) se ejecutó dos veces, el 26 de agosto y el 23 de septiembre de 2026, una vez por cada autor y con los mismos criterios de redondeo aplicados al resto de la sección.

12.1 Diseño de la Evaluación

Con el objetivo de contrastar el desempeño del sistema automatizado respecto del proceso manual equivalente, se definió un esquema de evaluación basado en la ejecución controlada del pipeline sobre el entorno de laboratorio contenerizado. El diseño previsto contemplaba comparar el tiempo requerido para completar el análisis manualmente, ejecutando y consolidando a mano las cinco herramientas, frente al tiempo del pipeline automatizado sobre el mismo objetivo y bajo condiciones idénticas. Esa comparación se ejecutó en esta ronda con el alcance que la Sección 9.9 declara: el proceso automatizado, en las cinco ejecuciones consecutivas de la campaña C contra el mismo objetivo de laboratorio (Sección 12.6), y el proceso manual, en dos corridas cronometradas, una por cada autor (Sección 12.2), ambos bajo el mismo objetivo, anfitrión y versiones de herramienta.

12.2 Resultados del Proceso Manual

Tabla 10

Resultados del proceso manual por etapa, en dos corridas, una por autor

Etapas del Proceso Manual	Atendido (A)	Reloj (A)	Atendido (B)	Reloj (B)	Dato registrado
Ejecución de Nmap e interpretación	17 s (rc)	24 s (rc)	20 s	24 s	Puerto HTTP: 80

Etapas del Proceso Manual	Atendido (A)	Reloj (A)	Atendido (B)	Reloj (B)	Dato registrado
Ejecución de WhatWeb e interpretación	1 min 20 s	2 min 03 s	1 min 12 s	1 min 59 s	4 tecnologías identificadas
Ejecución de Nikto	15 min 32 s	17 min 15 s	14 min 42 s	16 min 52 s	14 hallazgos
Ejecución de Nuclei	18 min 03 s	20 min 04 s	17 min 42 s	19 min 56 s	20 hallazgos
Ejecución de OWASP ZAP (escaneo activo)	9 min 32 s	11 min 15 s	10 min 01 s	11 min 50 s	12 hallazgos
Consolidación y clasificación de severidad	18 min 12 s	18 min 59 s	19 min 36 s	19 min 50 s	45 hallazgos consolidados (se descarta 1 alerta de ZAP)
Elaboración del informe de seguridad	28 min 58 s	29 min 03 s	26 min 47 s	28 min 21 s	—
TOTAL	1 h 31 min 54 s	1 h 39 min 03 s	1 h 30 min 20 s	1 h 39 min 12 s	—

Nota. Elaboración propia. Condiciones. Dos corridas del mismo protocolo, una por cada autor: la corrida A el 2026-08-26 y la corrida B el 2026-09-23, ambas sobre el mismo anfitrión (Apple M5 / macOS 26.6) y sobre la misma imagen del Servicio de Escaneo empleada en las campañas B y C, construida el 2026-08-21 y no reconstruida entre ambas fechas: Nmap 7.95, WhatWeb 0.5.5, Nikto 2.6.1 (LW 2.5), Nuclei v3.11.1 con plantillas v10.4.7 y OWASP ZAP 2.17.0 (Sección 12.6). A diferencia del resto de las mediciones de este capítulo, estos tiempos provienen de un cronómetro operado por cada autor y no de marcas temporales registradas por el sistema. Protocolo completo en el Anexo K; la transcripción de ambas hojas, con la evidencia del entorno, se publica en docs/manual-run-sheets del repositorio declarado en la Sección 20.

Concordancia entre operadores. Las dos corridas coincidieron en el dato registrado de cada etapa, de modo que la última columna vale para ambas: el mismo puerto, las mismas cuatro tecnologías, los mismos recuentos por herramienta y los mismos 45 hallazgos tras consolidar, incluido el descarte de una misma alerta de ZAP durante la consolidación. Esa coincidencia documenta la reproducibilidad del procedimiento y no constituye una réplica independiente: el segundo operador conocía el resultado de la primera corrida, ya publicado, de modo que la concordancia del producto no puede leerse como confirmación ciega.

(rc) Recronometraje. La hoja de registro original de la corrida A consignaba para la etapa de Nmap el mismo tiempo atendido que para la de Nikto, valor que no resistió la verificación. Esa etapa se volvió a cronometrar sobre el mismo objetivo, el mismo anfitrión y el mismo procedimiento del Anexo K, en dos sesiones posteriores del operador de la corrida A: el tiempo atendido el 2026-09-23, que arrojó 17 s, y el tiempo de reloj el 2026-09-25, que arrojó 24 s. El resto de la columna proviene de la sesión original del 2026-08-26. Ambos valores son mediciones de cronómetro, de modo que esta tabla no contiene ninguna cifra estimada ni reconstruida. Los totales de la tabla y las razones de la Sección 12.4 incorporan ambas correcciones.

Composición del tiempo de reloj. Comprende, además de la ejecución de la herramienta, el arranque de los contenedores, la espera del modo demonio de OWASP ZAP y la lectura de la salida por pantalla, lo que explica que supere en todas las filas la duración que la campaña C mide para la misma herramienta.

El brazo manual de comparación se ejecutó en esta ronda (Sección 9.6, Anexo K) y sus resultados se reportan en la Tabla 10. La reformulación de PM-1 (Sección 6) mide, independientemente de esas corridas, el componente de orquestación del propio pipeline

automatizado, que es la comparación con mayor fuerza metodológica de las dos: no depende de un operador humano ni del sesgo de “espera activa” que la corrida manual sí debe declarar.

Coincidencia entre operadores. Las dos corridas produjeron el mismo conjunto de hallazgos: los mismos recuentos por herramienta, los mismos 45 tras consolidar y el mismo criterio al descartar una alerta de ZAP durante la anotación. Esa coincidencia importa para interpretar los tiempos. Descarta que la diferencia entre operadores se explique porque uno haya hecho menos trabajo que el otro, y deja la comparación de la Sección 12.4 apoyada en dos ejecuciones equivalentes en producto.

Conviene precisar qué no autoriza a concluir. Los dos operadores son autores del sistema, de modo que la coincidencia habla de la reproducibilidad del procedimiento y no de su independencia respecto del sesgo de familiaridad que la Sección 9.9 declara. Dos personas que construyeron la misma herramienta tienden a leer su salida del mismo modo, y ese es precisamente el sesgo en cuestión.

12.3 Resultados del Sistema Automatizado

Datos de cinco corridas reales contra dvwa (campana A, 2026-08-11; los artefactos crudos se identifican en la nota de la Tabla 11). Desglose de la ejecución de herramientas por herramienta (media de las 5 corridas): Nikto 120,1 s, Nuclei 60,3 s, ZAP 52,1 s, WhatWeb 18,9 s, Nmap 6,9 s. Estas medias están redondeadas a un decimal. Su suma (258,3 s) más la orquestación (0,85 s) arroja 259,15 s, frente a los 259,06 s de la media del tiempo total: la diferencia de 0,09 s es artefacto del redondeo y no un tiempo sin contabilizar. La Figura 7 muestra ese mismo desglose tal como lo presenta la interfaz web de operación, con la duración y el aporte de cada herramienta.

Tabla 11

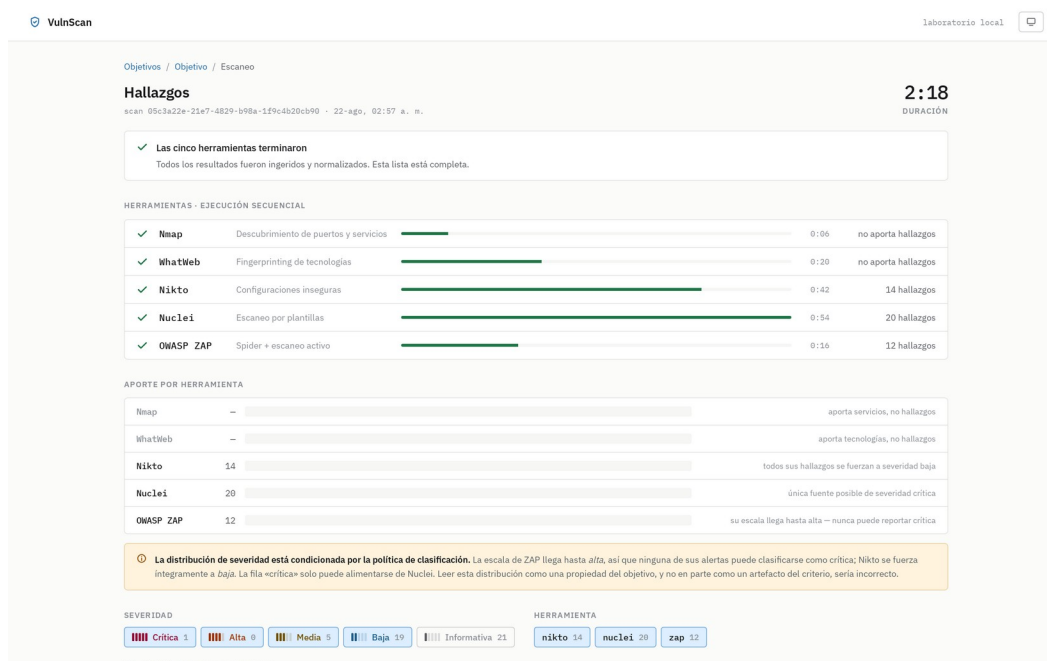
Resultados del sistema automatizado por ejecución

Ejecución	Tiempo total (min)	Servicios	Tecnologías	Hallazgos	T. informe (seg)	Estado
1	4,23 min (253,58 s)	N/D ¹	N/D ¹	32	N/D ¹	completed
2	4,63 min (277,53 s)	N/D ¹	N/D ¹	32	N/D ¹	completed
3	4,32 min (259,33 s)	N/D ¹	N/D ¹	32	N/D ¹	completed
4	4,22 min (253,19 s)	N/D ¹	N/D ¹	32	N/D ¹	completed
5	4,19 min (251,64 s)	N/D ¹	N/D ¹	32	N/D ¹	completed
Promedio	4,32 min (259,06 s)	N/D ¹	N/D ¹	32,0	N/D ¹	—
Desv. estándar	0,18 min (10,73 s)	N/D ¹	N/D ¹	0,0	N/D ¹	—
Coef. de variación	4,1%	N/D ¹	N/D ¹	0,0%	N/D ¹	—

Nota. Elaboración propia sobre los datos de la campaña A (2026-08-11). ¹ Las celdas “N/D” corresponden a columnas cuyo dato no forma parte del esquema capturado por el script de medición en esta ronda: el recuento de servicios y de tecnologías y el tiempo aislado de generación del informe. Este último está incluido dentro de “orquestración” sin desagregar, y se verificó por separado en menos de 1 segundo por formato.

Figura 7

Desglose por herramienta de una ejecución del pipeline contra DVWA: duración y aporte de hallazgos de cada una



Nota. Elaboración propia.

La Figura 7 expone el mismo desglose por herramienta sobre el que se construyó esta medición, lo que permite verificar la descomposición del tiempo en cada ejecución concreta y no sólo en el agregado de la campaña. La corrida allí mostrada es ilustrativa de la vista y no forma parte de las cinco corridas medidas, cuyos valores son los de la Tabla 11.

Un hallazgo no anticipado. Contra DVWA es Nikto, y no OWASP ZAP, la herramienta que domina el tiempo de ejecución: consume el 46,5 % del tiempo de herramientas. La causa no está en la herramienta sino en su invocación: el adaptador la llama con un presupuesto fijo de 120 segundos, de modo que su duración es una decisión de configuración y no una propiedad del objetivo. La Sección 11.6 asumía lo contrario. La Sección 12.6 mide después ambos objetivos y muestra que el supuesto original era correcto contra Juice Shop, donde ZAP consume el 51,5 %.

12.3.1 Origen de la variabilidad del pipeline

El coeficiente de variación del tiempo total, 4,1 %, describe la estabilidad del pipeline pero no explica de dónde proviene. Descomponer esa dispersión por herramienta, sobre las mismas cinco corridas, produce un resultado que la media por sí sola oculta.

Tabla 12

Dispersión del tiempo de ejecución por herramienta sobre cinco corridas contra DVWA

Herramienta	Media (s)	Desv. est. (s)	Coef. variación	% del tiempo	% de la varianza
Nikto	120,05	0,01	0,0 %	46,5 %	0,0 %
Nuclei	60,29	0,69	1,1 %	23,3 %	0,4 %
OWASP ZAP	52,10	10,52	20,2 %	20,2 %	96,1 %
WhatWeb	18,86	0,14	0,7 %	7,3 %	0,0 %
Nmap	6,91	0,05	0,7 %	2,7 %	0,0 %

Nota. Elaboración propia sobre el artefacto scripts/measurement_campaign_results/20260811T004104Z.json del repositorio. El porcentaje de la varianza se calcula sobre la varianza del tiempo total del pipeline (115,20 s²).

La lectura es inequívoca dentro de esta campaña, y sólo dentro de ella: la Sección 12.6 muestra que la herramienta responsable cambia incluso sin cambiar de objetivo. En la campaña A, contra DVWA, el escaneo activo de OWASP ZAP concentra el 96,1 % de la varianza del tiempo total, pese a representar sólo el 20,2 % de su duración media. Su tiempo oscila entre 45,60 s y 70,45 s según la corrida, un rango de 24,85 s que explica casi por completo el rango de 25,89 s observado en el tiempo total; la correlación entre ambos es de 0,998. Excluido ZAP del cómputo, el coeficiente de variación del pipeline cae de 4,14 % a 0,34 %.

Las otras cuatro herramientas son, a efectos prácticos, deterministas en duración. El caso de Nikto merece mención aparte porque su desviación estándar es de 0,01 s sobre cinco corridas, con valores entre 120,04 s y 120,07 s: no se trata de una herramienta excepcionalmente estable, sino de que el adaptador la invoca con un presupuesto fijo de 120 segundos y Nikto lo agota siempre. Su duración no es una medición del objetivo, es una constante de configuración, y esto confirma cuantitativamente lo que la Sección 15.1 argumenta sobre su aparente dominio del tiempo total.

Este resultado da contenido empírico a H2 en la condición evaluada. La consistencia observada entre ejecuciones contra DVWA no es una propiedad difusa del sistema: es consecuencia de que cuatro de sus cinco herramientas tienen duración fija o casi fija, y de que la única fuente real de variación está aislada e identificada. La Sección 12.6.2 muestra que esa consistencia no se reproduce ni contra el segundo objetivo ni sobre el segundo anfitrión. En las condiciones de esta campaña, una intervención dirigida a reducir la dispersión del pipeline tendría que actuar sobre la configuración del escaneo activo de ZAP. La Sección 12.6.1 muestra que esa conclusión no se traslada a las otras dos campañas, donde la dispersión se concentra en Nuclei y una intervención sobre ZAP no tendría efecto.

12.4 Análisis Comparativo

Tabla 13

Comparación entre la condición manual y la condición automatizada

Dimensión	Proceso manual	Sistema automatizado	Diferencia
Tiempo atendido /	1 h 31 min 07 s de	0,24 s (media,	el pipeline requiere el

Dimensión	Proceso manual	Sistema automatizado	Diferencia
de orquestación	media (5.467 s): 5.514 s y 5.420 s	campana C)	0,004 % del tiempo atendido que exige el proceso manual
Tiempo total de reloj	1 h 39 min 08 s de media (5.947,5 s): 5.943 s y 5.952 s	167,25 s (2,79 min, media, campana C)	el proceso manual insumió aproximadamente 36 veces el tiempo de reloj del pipeline
Hallazgos obtenidos	45 en las dos corridas (14 Nikto + 20 Nuclei + 12 ZAP, menos 1 alerta de ZAP descartada al consolidar)	45,6 de media (46, 46, 46, 44 y 46)	equivalentes: cuatro de las cinco corridas produjeron 46 hallazgos y una 44; la diferencia respecto de las dos corridas manuales es consistente con el descarte de una alerta de ZAP durante la consolidación manual
Condición de comparación	Dos corridas separadas, una por autor, sobre el mismo anfitrión y las mismas versiones de herramienta que la campana C (Nikto 2.6.1, Nuclei v3.11.1)	Media de cinco corridas, campana C (2026-08-24)	—

Nota. Elaboración propia sobre el artefacto 20260824T193554Z.json de la campana C y sobre las dos hojas de cronometraje transcritas en docs/manual-run-sheets, ambos publicados en el repositorio (Sección 20). La media de 167,25 s de la campana C tiene una desviación estándar de 69,56 s, es decir un coeficiente de variación del 41,6 %: cuatro de las cinco corridas se agrupan en torno a 134 s y una alcanzó 291,54 s. La razón de 36 veces debe leerse, por tanto, como un orden de magnitud y no como un factor estable: calculada sobre la corrida más rápida da 44 veces y sobre la más lenta, 20.

PM-1 (Sección 6) queda respondida con medición directa: 0,85 s de media sobre 259,06 s de pipeline total (99,7% del tiempo es ejecución de herramientas, un piso común a cualquier operador). H2 (reformulada) se sostuvo en la campana original contra DVWA, con 0 % de variación observada en el conjunto de hallazgos, y no se sostiene en ninguna de las dos réplicas: contra Juice Shop una corrida produjo 31 hallazgos frente a 44, y contra el mismo DVWA sobre otro anfitrión una produjo 44 frente a 46 (Sección 12.6.2). La consistencia perfecta se observó en una de tres campañas. La hipótesis, por tanto, queda refutada como enunciado general y acotada a la condición en que se verifica. La descomposición de la Sección 12.3.1 añade el mecanismo que explica esa estabilidad: cuatro de las cinco herramientas tienen duración fija o casi fija, y en esa campana el 96,1 % de la varianza temporal se concentra en el escaneo activo de ZAP, sin que ello altere el conjunto de hallazgos producido. H3 se retiró del bloque de hipótesis (Sección 6) y pasó a requisito no funcional del Servicio de Reportes (Sección 11.5).

12.5 Distribución de Vulnerabilidades por Severidad

Tabla 14

Distribución de hallazgos por nivel de severidad

Nivel de Severidad (CVSS 3.1)	Cantidad	Porcentaje	Ejemplos representativos
Crítica (9,0 – 10,0)	1	3,1%	DVWA Default Login (Nuclei)
Alta (7,0 – 8,9)	0	0,0%	Ninguno registrado en estas corridas (ver Sección 11.4 sobre el sesgo estructural de la escala)
Media (4,0 – 6,9)	5	15,6%	Content Security Policy (CSP) Header Not Set; Directory Browsing; HTTP Only Site (ZAP)
Baja (0,1 – 3,9)	5	15,6%	Cookie No HttpOnly Flag; Cookie without SameSite Attribute (ZAP)
Informativa (0,0)	21	65,6%	Authentication Request Identified; Session Management Response Identified; User Agent Fuzzer (ZAP/Nuclei)
TOTAL	32	100%	—

Nota. Elaboración propia, agregado de la campaña de medición contra dvwa (2026-08-11).

Cobertura real contra los catálogos de verdad de referencia (Anexo E), sobre un scan real de cada target:

DVWA (scan f9760239-..., 32 hallazgos, catálogo de 11 entradas):

Tabla 15

Recall, precisión y F1 por herramienta contra el catálogo de DVWA

Herramienta	Hallazgos	Verd. positivos	Falsos pos.	Precisión	Recall	F1
Nuclei	20	14	6	0,70	0,27	0,39
ZAP	12	11	1	0,92	0,27	0,42
Nikto	0	—	—	—	—	—

Nota. Elaboración propia, scripts/ground_truth/match_findings.py contra el catálogo de DVWA (Anexo E), sobre el escaneo f9760239 del 2026-08-01. Sus hallazgos y tareas se publican en scripts/ground_truth/reported-in-report del repositorio (Sección 20). Sobre la versión del comparador empleada, ver la nota de la Tabla 19.

Juice Shop (scan c2421b99-..., 28 hallazgos, catálogo de 52 entradas):

Tabla 16

Recall, precisión y F1 por herramienta contra el catálogo de Juice Shop

Herramienta	Hallazgos	Verd.	Falsos	Precisión	Recall	F1
-------------	-----------	-------	--------	-----------	--------	----

		positivos	pos.			
ZAP	6	5	1	0,83	0,08	0,14
Nikto	9	1	8	0,11	0,02	0,03
Nuclei	13	0	13	0,00	0,00	0,00

Nota. Elaboración propia, scripts/ground_truth/match_findings.py contra el catálogo de Juice Shop (Anexo E), sobre el escaneo c2421b99 del 2026-08-11. Sus hallazgos y tareas se publican en scripts/ground_truth/reported-in-report del repositorio (Sección 20). Sobre la versión del comparador empleada, ver la nota de la Tabla 19.

La medición de recall, precisión y F1 contra los catálogos públicos de Juice Shop y DVWA (tablas arriba) no encontró, en ninguno de los dos laboratorios, un solo emparejamiento de alta confianza (CVE exacto o tipo más ubicación). Todos los emparejamientos obtenidos en ambos objetivos son del nivel más débil: coincidencia de palabra clave. Contra DVWA, la causa es identificable y corregible: el pipeline escanea sin autenticar y las 11 páginas vulnerables del catálogo exigen sesión, por lo que el recall medido por emparejamiento léxico es 0% en su nivel de alta confianza; la anotación manual presentada más abajo corrige esa lectura. Contra Juice Shop, sin ese impedimento de acceso, el recall de alta confianza también fue nulo, lo que apunta a una limitación del propio instrumento de medición (desalineamiento entre el vocabulario de los catálogos oficiales y el de los hallazgos de las herramientas) más que a una ausencia real de detección. Esta es, en sí, una limitación metodológica legítima (ver Sección 14.1). Construir verdad de referencia contra aplicaciones deliberadamente vulnerables sin CVE asignados exige un criterio de emparejamiento semántico más robusto que la coincidencia de texto libre. La alternativa es anotar manualmente qué hallazgos de cada herramienta corresponden a qué desafío del catálogo, trabajo realizado para DVWA autenticado y reportado en la Tabla 18.

La distribución se obtiene directamente mediante una consulta de agregación sobre la tabla findings agrupando por la columna severity. Es idéntica en las 5 corridas de la campaña de medición (pipeline determinista contra un dvwa sin cambios entre corridas). Desagregada por herramienta: ZAP (3 info, 5 low, 4 medium, 0 critical), Nuclei (18 info, 0 low, 1 medium, 1 critical), Nikto (0 hallazgos parseables en las cinco corridas sin autenticar de esta campaña; ver más abajo la condición autenticada, donde aporta 15, y el Anexo E sobre el muro de autenticación de DVWA). Esta desagregación es necesaria, no opcional: la fila “Crítica” depende exclusivamente de qué encontró Nuclei (ver Sección 11.4).

Para acotar esta limitación se implementó un modo de escaneo autenticado opcional (options.authenticated=true en POST /scan/{tool} del Scanner Service, ver Anexo E) que ejecuta el login de DVWA y fija su nivel de seguridad en “low” antes de invocar Nikto, Nuclei y ZAP contra el mismo target. Este modo es opt-in: no es el comportamiento por defecto del workflow de n8n, que sigue escaneando sin autenticar, sin cambios, para no alterar el comportamiento ya medido de la campaña original.

Tabla 17

Recall, precisión y F1 por herramienta contra el catálogo de DVWA (escaneo autenticado)

Herramienta	Hallazgos	Verd. positivos	Falsos pos.	Precisión	Recall	F1
Nikto	15	7	8	0,47	0,27	0,34
Nuclei	19	15	4	0,79	0,27	0,41
ZAP	22	16	6	0,73	0,36	0,48

Nota. Elaboración propia, scripts/ground_truth/_run_authenticated_dvwa_scan.py + scripts/ground_truth/match_findings.py contra el catálogo de DVWA (Anexo E), corrida única con options.authenticated=true: el escaneo 0a605f82 del 2026-08-20, ejecutado con la implementación del modo autenticado que se registró ese mismo día en el commit 5b2b8be. Sus hallazgos y tareas se publican en scripts/ground_truth/reported-in-report del repositorio (Sección 20). Sobre la versión del comparador empleada, ver la nota de la Tabla 19.

Nota sobre las Tablas 15 a 17. Todas las métricas de estas tres tablas derivan exclusivamente de emparejamientos por coincidencia de palabra clave, el más débil de los tres niveles definidos en la Sección 9.3: ningún hallazgo alcanzó los niveles de CVE exacto ni de tipo más ubicación. Bajo ese criterio, “verdadero positivo” significa que el hallazgo comparte vocabulario con una entrada del catálogo, no que se haya verificado que corresponda a ella. Los valores se reportan con dos decimales por esa razón, y deben leerse como indicativos del orden de magnitud y no como medidas de precisión conocida. La Tabla 18 presenta, en cambio, correspondencias verificadas manualmente una por una.

El recall de alta confianza (CVE exacto o tipo+ubicación) permanece en 0% incluso autenticado: los 38 matches de esta corrida son, igual que en la medición sin autenticar, de nivel de confianza más débil (coincidencia de palabra clave) — la limitación del instrumento de matching descrita más arriba no se resuelve autenticando. Lo que sí cambia es la cobertura: Nikto pasó de 0 hallazgos parseables a 15 (3 de las 11 entradas del catálogo cubiertas), ZAP mejoró de 3 a 4 entradas cubiertas (recall 0,27 → 0,36), y Nuclei se mantuvo en las mismas 3 entradas (recall sin cambios, 0,27 en ambas condiciones — buena parte de sus plantillas no requieren sesión). Las 8 entradas del catálogo que exigen completar un flujo de la aplicación (inyección SQL ciega, ejecución de comandos, inclusión de archivos, subida de archivos, identificadores de sesión débiles y las tres variantes de XSS) siguen sin cobertura incluso autenticado: el muro de acceso era una causa real y ahora corregible, pero no la única.

Anotación manual del conjunto de verdad. Para superar la limitación del emparejamiento léxico descrita más arriba, se anotó manualmente la correspondencia entre las once entradas del catálogo de DVWA y los 56 hallazgos de la corrida autenticada, evaluando cada par contra el título, el tipo, la descripción y la evidencia del hallazgo.

Tabla 18

Cobertura del catálogo de DVWA bajo anotación manual (corrida autenticada)

Entrada del catálogo	Clase	Estado	Herramienta que la detecta
DVWA-csp	Configuración	Detectada	ZAP, Nikto
DVWA-csrf	Configuración	Detectada	ZAP
DVWA-sqli	Inyección	Detectada	ZAP
DVWA-sqli_blind	Inyección	No detectada	—
DVWA-exec	Inyección	No detectada	—
DVWA-fi	Inyección	No detectada	—
DVWA-upload	Configuración	No detectada	—
DVWA-weak_id	Configuración	No detectada	—
DVWA-xss_d	XSS	No detectada	—
DVWA-xss_r	XSS	No detectada	—
DVWA-xss_s	XSS	No detectada	—

Nota. Elaboración propia. Anotación manual sobre la corrida autenticada; el criterio y la justificación por entrada se detallan en el Anexo E.

El recall del sistema es de **0,27** (tres de once entradas), esta vez con correspondencia verificada y no por coincidencia de vocabulario. La distribución de lo detectado y lo no detectado no es aleatoria y admite una lectura clara. **El pipeline detecta lo observable en la respuesta del servidor:** cabeceras ausentes, formularios sin token anti-CSRF, inyección SQL alcanzable desde un parámetro. **No detecta, en cambio, lo que exige completar un flujo de la aplicación:** subir un archivo y verificar su ejecución, inyectar carga en un formulario y observar el resultado, o recolectar identificadores de sesión para analizar su predictibilidad.

La ausencia de las tres variantes de cross-site scripting merece mención aparte, por tratarse de la clase de vulnerabilidad que un escáner de aplicaciones web debería cubrir por definición. La explicación más plausible es de alcance del rastreo y no de capacidad de la herramienta: OWASP ZAP se invoca en modo de una sola pasada, cuyo rastreador clásico descubre poca superficie sobre una aplicación de navegación autenticada, y la inyección de carga exige alcanzar primero el formulario objetivo. Incorporar el rastreador AJAX o una fase de rastreo autenticada extendida queda identificado como la línea de trabajo futuro con mayor impacto potencial sobre la cobertura (Sección 17).

Conviene enunciar el alcance de esta anotación con precisión: se realizó sobre una corrida única de un solo objetivo, la efectuaron los propios autores del sistema, lo que introduce un sesgo de dirección indeterminada al decidir qué cuenta como correspondencia, y no fue validada por un tercero independiente. Es suficiente para sostener la descripción de qué clases de vulnerabilidad cubre el pipeline y cuáles no; no lo es para una comparación con las tasas de detección reportadas en la literatura.

Figura 8

Hallazgos normalizados de un escaneo, con severidad homogeneizada y herramienta de origen

25 de 46 21 ocultos por los filtros

SEVERIDAD	TÍTULO	TIPO	ORIGEN	CVSS
CRÍTICA	DVWA Default Login	http	Nuclei	—
MEDIA	Content Security Policy (CSP) Header Not Set	web_vulnerability	OWASP ZAP	—
MEDIA	Missing Anti-clickjacking Header	web_vulnerability	OWASP ZAP	—
MEDIA	HTTP Only Site	web_vulnerability	OWASP ZAP	—
MEDIA	Directory Browsing	web_vulnerability	OWASP ZAP	—
MEDIA	Sensitive Configuration Files Listing - Detect	http	Nuclei	5.3
BAJA	X-Content-Type-Options Header Missing	web_vulnerability	OWASP ZAP	—
BAJA	Server Leaks Version Information via "Server" HTTP Response Header Field	web_vulnerability	OWASP ZAP	—
BAJA	In Page Banner Information Leak	web_vulnerability	OWASP ZAP	—
BAJA	Cookie without SameSite Attribute	web_vulnerability	OWASP ZAP	—
BAJA	Cookie No HttpOnly Flag	web_vulnerability	OWASP ZAP	—
BAJA	The X-Content-Type-Options header is not set. This could allow the user agent to render the content of the site in a different fashion to the MIME type.	web_misconfiguration	Nikto	—
BAJA	Directory indexing found.	web_misconfiguration	Nikto	—
BAJA	Suggested security header missing: permissions-policy.	web_misconfiguration	Nikto	—
BAJA	Apache/2.4.25 appears to be outdated (current is at least 2.4.68).	web_misconfiguration	Nikto	—
BAJA	Suggested security header missing: strict-transport-security.	web_misconfiguration	Nikto	—
BAJA	Suggested security header missing: content-security-policy.	web_misconfiguration	Nikto	—
BAJA	Suggested security header missing: referrer-policy.	web_misconfiguration	Nikto	—
BAJA	Suggested security header missing: x-content-type-options.	web_misconfiguration	Nikto	—
BAJA	X-Frame-Options header is deprecated and was replaced with the Content-Security-Policy HTTP header with the frame-ancestors directive.	web_misconfiguration	Nikto	—
BAJA	.gitignore file found. It is possible to grasp the directory structure.	web_misconfiguration	Nikto	—
BAJA	Admin login page/section found.	web_misconfiguration	Nikto	—
BAJA	Apache default file found.	web_misconfiguration	Nikto	—
BAJA	Directory indexing found.	web_misconfiguration	Nikto	—
BAJA	Configuration information may be available remotely.	web_misconfiguration	Nikto	—

Nota. Elaboración propia.

La vista de hallazgos (Figura 8) muestra el resultado de la normalización descrita en la Sección 11.4: hallazgos de tres herramientas con vocabularios y escalas distintas presentados bajo un esquema único, con la herramienta de origen siempre visible junto a cada uno. Esa columna no es decorativa: es la que permite al operador reconstruir a qué política de clasificación responde cada severidad, y es la razón por la cual la distribución se reporta siempre desagregada por herramienta.

12.5.1 Estabilidad de la precisión entre objetivos

Las Tablas 15 a 17 reportan cada condición por separado. Léidas en conjunto, sobre la misma columna de precisión, exponen una diferencia entre herramientas que ninguna de las tres muestra por sí sola.

Tabla 19

Precisión por herramienta y proporción de hallazgos sin correspondencia, según condición

Herramienta	DVWA	DVWA autenticado	Juice Shop
OWASP ZAP	0,92	0,73	0,83
Nuclei	0,70	0,79	0,00

Herramienta	DVWA	DVWA autenticado	Juice Shop
Nikto	—	0,47	0,11
Hallazgos sin correspondencia	21,9 %	32,1 %	78,6 %

Nota. Elaboración propia sobre los informes de emparejamiento de scripts/ground_truth/. La última fila expresa el porcentaje de hallazgos producidos por el pipeline que no corresponden a ninguna entrada del catálogo de verdad de referencia. Nikto no produjo hallazgos parseables en la condición no autenticada. Todas las cifras derivan de emparejamiento por palabra clave y son indicativas, no comparables con mediciones basadas en correspondencia validada. Versión del comparador. Las cifras de cobertura de esta sección —Tablas 15, 16, 17 y 19— se calcularon con la versión de scripts/ground_truth/match_findings.py y de los catálogos vigente en la fecha de cada corrida. Ambos se revisaron el 2026-08-24, y la versión revisada, aplicada a los mismos hallazgos, produce valores distintos: contra DVWA sin autenticar, precisiones de 0,50 para OWASP ZAP y 0,10 para Nuclei, con el 75,0 % de hallazgos sin correspondencia; contra DVWA autenticado, 0,36, 0,05 y 0,00 para ZAP, Nuclei y Nikto, con el 83,9 %; contra Juice Shop, los mismos valores que aquí se reportan. La diferencia no procede de los hallazgos, que son los mismos y están publicados, sino del criterio con que se decide si un hallazgo corresponde a una entrada del catálogo. Es la situación que la Sección 8.9 anticipa al enunciar que un recall no es interpretable sin declarar su criterio de correspondencia, y la razón por la cual este trabajo apoya su conclusión de cobertura en la anotación manual de la Tabla 18, que no depende del comparador automático, y no en estas cifras. Los hallazgos crudos de las tres corridas se publican en scripts/ground_truth/reported-in-report para que cualquiera de las dos versiones pueda recalcularse.

OWASP ZAP es la única herramienta cuya precisión se mantiene en el mismo orden de magnitud a través de las tres condiciones, entre 0,73 y 0,92. Las otras dos se degradan al cambiar de objetivo: Nuclei pasa de 0,79 contra DVWA autenticado a 0,00 contra Juice Shop, donde emite trece hallazgos y ninguno corresponde a una entrada del catálogo; Nikto cae de 0,47 a 0,11. Es decir que la utilidad de estas dos herramientas, en este laboratorio, depende del objetivo, mientras que la de ZAP no.

La última fila mide algo distinto y de consecuencia práctica directa: qué proporción de lo que el sistema entrega no corresponde a nada del catálogo. Crece de 21,9 % contra DVWA a 78,6 % contra Juice Shop. Sobre el objetivo de mayor superficie, casi cuatro de cada cinco hallazgos exigen triaje humano antes de poder descartarlos o accionarlos. Ese trabajo de triaje no está automatizado por el sistema y no fue medido. Es el residuo de intervención humana que sobrevive a la automatización del ciclo, y conviene enunciarlo con esa precisión en lugar de presentar el pipeline como un reemplazo completo del análisis manual.

Ambos resultados convergen con el de la Sección 12.3.1 en una caracterización única del conjunto de herramientas. En la campaña A, ZAP es simultáneamente la fuente dominante de variabilidad temporal del pipeline, con el 96,1 % de su varianza, y la única herramienta cuya precisión resiste el cambio de objetivo. Sólo la segunda de esas dos propiedades se sostiene en las tres campañas: la primera vale para la A y no para las réplicas (Sección 12.6.1). Su costo y

su valor tienen el mismo origen: es la única que realiza análisis activo dependiente de la superficie concreta que descubre.

12.5.2 Procedencia de la severidad asignada

La distribución de severidad reportada más arriba admite una pregunta que las tablas anteriores no responden: de dónde proviene cada etiqueta. El sistema puede asignarla a partir de una puntuación CVSS emitida por la propia herramienta, o derivarla de su política de normalización cuando la herramienta no aporta esa señal. Contar ambos casos sobre los 116 hallazgos de las tres condiciones permite medir cuánto de la severidad reportada describe al objetivo y cuánto describe al sistema que la clasifica.

Tabla 20

Severidad asignada y procedencia de la puntuación, por herramienta, sobre las tres condiciones

Herramienta	n	Crít.	Alta	Media	Baja	Info.	Con CVSS	Sev. máx.
OWASP ZAP	40	0	1	15	13	11	0 / 40	Alta
Nuclei	52	2	0	3	1	46	3 / 52	Crítica
Nikto	24	0	0	0	24	0	0 / 24	Baja
Total	116	2	1	18	38	57	3 / 116	—

Nota. Elaboración propia sobre los hallazgos crudos de scripts/ground_truth/, unidos a su herramienta de origen por scan_task_id. “Con CVSS” cuenta los hallazgos que traen una puntuación CVSS emitida por la herramienta; en los restantes, la severidad la asigna la política de normalización del Backend (Sección 8.5). “Sev. máx.” es el nivel más alto efectivamente alcanzado por esa herramienta.

Tres de ciento dieciséis hallazgos traen puntuación CVSS propia. En el 97,4 % restante la severidad es una decisión del sistema de normalización, no una medida emitida por la herramienta que detectó el hallazgo. Ese dato reencuadra por completo la lectura de la distribución de severidad: no es una descripción del riesgo del objetivo, es en su casi totalidad una proyección de la política de clasificación descrita en la Sección 8.5.

El desglose por herramienta muestra de qué manera. Los veinticuatro hallazgos de Nikto están clasificados como severidad baja, sin excepción, porque la herramienta no emite señal de severidad y la política los asigna a ese nivel por defecto. Su etiqueta de severidad no transporta información alguna sobre el hallazgo. Los cuarenta hallazgos de OWASP ZAP no traen ninguna puntuación CVSS y su escala de cuatro niveles no alcanza el nivel crítico del sistema, de modo que ZAP no puede aportar un solo hallazgo a esa fila por construcción. En consecuencia, los dos únicos hallazgos críticos de todo el conjunto provienen de Nuclei, que es también la única herramienta que emite CVSS, y lo hace en tres de sus cincuenta y dos hallazgos.

La implicación práctica es directa y conviene que quede escrita antes que la formule un lector: comparar herramientas por la distribución de severidad de sus hallazgos, dentro de este sistema, no compara capacidad de detección sino posición de cada herramienta dentro de la política de normalización. Por esa razón toda distribución de severidad de este trabajo se reporta

desagregada por herramienta y nunca como propiedad del objetivo analizado. La Sección 15.3 desarrolla las consecuencias de este resultado.

12.6 Replicación en un segundo anfitrión y un segundo objetivo

Los resultados anteriores provienen de una única campaña, ejecutada el 11 de agosto de 2026 contra DVWA sobre un anfitrión con procesador AMD Ryzen 7 5700U bajo Windows 11. Para evaluar en qué medida dependen de esas condiciones, se ejecutaron el 24 de agosto de 2026 dos campañas adicionales de cinco corridas cada una, sobre un anfitrión distinto: un procesador Apple M5 de arquitectura ARM bajo macOS 26.6. La campaña B corrió contra OWASP Juice Shop e introduce un objetivo nuevo. La campaña C corrió contra DVWA y conserva el objetivo original, lo que descarta la superficie analizada como explicación de sus diferencias con la A; no aísla en cambio una variable única, por las razones que detalla la Sección 12.6.3. Entre ambas fechas las versiones de dos herramientas cambiaron sin intervención, por efecto de la resolución en tiempo de construcción que la Sección 14.1 describe: Nikto pasó de 2.6.0 a 2.6.1 y el motor de Nuclei de v3.11.0 a v3.11.1.

Conviene declarar de entrada la consecuencia metodológica: la campaña B difiere de la A en objetivo, anfitrión, arquitectura, sistema operativo y versiones de herramienta simultáneamente, de modo que sus diferencias no son atribuibles a una causa aislada. La campaña C reduce ese conjunto, porque comparte objetivo con la A, pero sigue moviendo cuatro variables a la vez: anfitrión, versiones de herramienta, imágenes del laboratorio y código del sistema (Secciones 9.3 y 20). Ninguna de las tres es acumulable con las otras. Lo que sí permiten es evaluar la estabilidad de las conclusiones de las Secciones 12.3.1 y 12.5.1 fuera de las condiciones en que se obtuvieron, que es el propósito de esta subsección.

Tabla 21

Composición del tiempo y de la varianza por herramienta en las tres campañas

Herram.	% t (A)	% var (A)	% t (B)	% var (B)	% t (C)	% var (C)
Nmap	2,7 %	0,0 %	4,4 %	0,0 %	3,7 %	0,0 %
WhatWeb	7,3 %	0,0 %	1,9 %	0,0 %	11,5 %	0,0 %
Nikto	46,5 %	0,0 %	25,7 %	0,1 %	25,0 %	0,0 %
Nuclei	23,3 %	0,4 %	16,5 %	101,0 %	50,5 %	98,6 %
OWASP ZAP	20,2 %	96,1 %	51,5 %	0,0 %	9,3 %	0,0 %

Nota. Elaboración propia sobre los artefactos 20260811T004104Z.json, 20260824T172108Z.json y 20260824T193554Z.json del directorio scripts/measurement_campaign_results del repositorio (Sección 20). Campaña A: 2026-08-11, DVWA, AMD Ryzen 7 5700U / Windows 11, Nikto 2.6.0 y Nuclei v3.11.0. Campañas B y C: 2026-08-24, Apple M5 / macOS 26.6, Nikto 2.6.1 y Nuclei v3.11.1, contra OWASP Juice Shop y DVWA respectivamente. Las tres de cinco corridas consecutivas. Los porcentajes de tiempo se calculan sobre el tiempo de ejecución de herramientas y los de varianza sobre la varianza del tiempo total del pipeline. La participación de Nuclei en la varianza de la campaña B supera el 100 %. Las varianzas de los componentes no suman la del total cuando existe covarianza negativa entre ellos: el porcentaje se calcula sobre la varianza del tiempo total y no sobre la suma de las

varianzas componentes. Debe leerse, por tanto, como que la dispersión del pipeline se explica enteramente por esa herramienta.

12.6.1 Lo que se sostiene y lo que no

El resultado de la Sección 12.3 se sostiene sin matices y se refuerza. La orquestación consume 0,846 s sobre 259,06 s en la campaña A y 0,277 s sobre 252,83 s en la B: 0,33 % y 0,11 % respectivamente. Que el costo de coordinación sea despreciable frente al de ejecución no depende del objetivo, del anfitrión ni de la arquitectura de procesador.

El resultado de la Sección 12.3.1, en cambio, debe acotarse más de lo que su enunciado original sugería. La atribución del 96,1 % de la varianza al escaneo activo de OWASP ZAP no describe el comportamiento del pipeline contra DVWA: describe el de la campaña A. En la campaña B, contra Juice Shop, la varianza de ZAP es del 0,0 % y la dispersión pasa a Nuclei, con un coeficiente de variación del 43,3 %. Y en la campaña C, contra el mismo DVWA de la campaña A, ocurre exactamente lo mismo: ZAP aporta el 0,0 % de la varianza y Nuclei el 98,6 %, arrastrado por una corrida en la que se extendió a 207,8 s frente a los 51 a 59 s de las otras cuatro. Lo que se mantiene en las tres campañas es la forma del resultado y no su contenido: la dispersión del pipeline se concentra casi por completo en una sola herramienta, pero cuál es esa herramienta cambia incluso sin cambiar de objetivo. La consecuencia práctica invierte una recomendación que la Sección 12.3.1 formulaba y conviene enunciarla sin rodeos: no existe un componente sobre el cual intervenir para reducir la dispersión del pipeline, porque el que la domina en una campaña es irrelevante en la siguiente. Este trabajo no dispone de evidencia para predecir cuál será la herramienta dominante en una condición que no haya medido.

Una segunda observación corrige una afirmación que el propio documento había dejado pendiente. La Sección 11.6 asumía que el escaneo activo de ZAP dominaría el tiempo total, y la campaña A lo desmintió: contra DVWA domina Nikto, con el 46,5 %. La Sección 15.1 conjeturaba que el supuesto original sería correcto contra Juice Shop, cuya superficie es mayor, pero advertía que no se había medido. La campaña B lo mide y lo confirma: contra Juice Shop, ZAP consume el 51,5 % del tiempo de herramientas y Nikto el 25,7 %, exactamente la relación inversa. Ambas afirmaciones eran correctas, cada una en su objetivo, y ninguna de las dos generaliza.

12.6.2 H2 no se sostiene contra el segundo objetivo

El resultado de mayor consecuencia de la campaña B afecta a H2. La hipótesis sostiene que la automatización produce resultados consistentes entre ejecuciones sucesivas contra el mismo objetivo, medida como invariancia del conjunto de hallazgos. En la campaña A la evidencia era categórica: treinta y dos hallazgos con distribución de severidad idéntica en las cinco corridas. Las dos réplicas la desmienten. En la campaña B, contra Juice Shop, los recuentos fueron 44, 44, 44, 31 y 44. En la campaña C, contra el mismo DVWA de la campaña A pero sobre el otro anfitrión, fueron 46, 46, 46, 44 y 46. La consistencia perfecta se observó, por tanto, en una de las tres campañas.

La cuarta corrida produjo trece hallazgos menos que las otras cuatro. La descomposición identifica su origen sin ambigüedad. Los trece ausentes son exactamente los que Nuclei aporta en las demás corridas: doce informativos y uno de severidad media. En esa ejecución Nuclei

corrió durante 10,8 s, frente a los 40,5 a 53,8 s de las restantes, sin emitir un solo hallazgo. Las contribuciones de ZAP y de Nikto fueron idénticas en las cinco.

El escaneo, sin embargo, se registró como completado. Ni el Servicio de Escaneo ni el orquestador distinguen entre una herramienta que no encuentra nada y una que no llega a ejecutarse correctamente: el endpoint devolvió código HTTP 200 en las cinco invocaciones, la tarea quedó sin mensaje de error y el pipeline continuó con normalidad. El sistema es, por tanto, susceptible de fallos parciales silenciosos, y en la condición evaluada la tasa observada fue de uno en cinco, o de dos en quince sobre el conjunto de las tres campañas. La Sección 12.6.4 muestra que sobre OWASP Juice Shop la proporción es bastante mayor, de modo que esa tasa describe la condición evaluada y no acota el fenómeno. Es una limitación del diseño que la evaluación reveló y que no había sido anticipada.

Debe consignarse el alcance de esta observación con precisión. El fallo no pudo reproducirse de forma aislada: tres invocaciones directas al endpoint del Servicio de Escaneo contra el mismo objetivo, posteriores a la campaña, completaron con normalidad y devolvieron diecinueve, veinte y diecinueve hallazgos respectivamente. Su causa raíz no queda establecida por esta evaluación, y afirmar lo contrario excedería la evidencia disponible. Lo que sí queda establecido es el hecho observado —una corrida de cinco con conjunto de hallazgos distinto— y la incapacidad del sistema para señalarlo.

La consecuencia sobre la hipótesis es directa: H2 se sostuvo en la campaña A y no se sostiene en las otras dos, con la particularidad de que la campaña C corrió contra el mismo objetivo que la A. La discrepancia no depende entonces del objetivo elegido, y el componente responsable tampoco es siempre el mismo: en la campaña B fue Nuclei, que no aportó ningún hallazgo, y en la campaña C fue OWASP ZAP, que en una corrida emitió un hallazgo informativo en lugar de tres. La Sección 14.3 advierte que la formulación de H2 tiene poder falsador débil sobre un pipeline determinista evaluado contra un objetivo invariante. Introducir un segundo objetivo constituye la fuente de variación que esa advertencia reclamaba, y bajo esa condición la hipótesis efectivamente falla. Un resultado negativo obtenido de este modo es más informativo que la confirmación previa, porque identifica la condición bajo la cual la consistencia deja de verificarse y el componente responsable de que así sea.

Corresponde por último señalar lo que esta réplica no resuelve. Sigue tratándose de cinco corridas por condición, sobre dos objetivos de laboratorio y dos anfitriones, y sin brazo manual ejecutado en paralelo a estas dos réplicas: las dos corridas de la condición de control son posteriores y reproducen la condición de la campaña C, no la de la A ni la de la B (Sección 12.2). Las cifras conservan su carácter descriptivo y no admiten inferencia estadística ni extrapolación a entornos de producción (Sección 14.3).

12.6.3 Reproducibilidad de la medición sobre el mismo objetivo

La campaña C permite la comparación más informativa del conjunto, porque conserva el objetivo de la campaña A. Lo que cambia entre ambas no es, sin embargo, una sola variable: cambian el anfitrión, las versiones de herramienta, las imágenes del laboratorio y el código del sistema. Es, en los términos de la Sección 14.1, la comprobación empírica de un argumento que hasta ahora el documento sostenía sólo de forma conceptual.

El mismo comando, contra el mismo objetivo, trece días después, produjo un resultado materialmente distinto. El tiempo total medio pasó de 259,06 s a 167,25 s, una reducción del 35,4 %. El recuento de hallazgos pasó de 32 a 45,6 de media, un aumento del 42,5 %. La distribución de severidad se modificó en consecuencia: la fila de severidad baja pasó de 5 hallazgos a 19, porque Nikto —que en la campaña A no había producido ningún hallazgo parseable— aporta ahora catorce en cada corrida.

Las causas candidatas son cuatro y el diseño de esta réplica no permite separarlas. La primera es el cambio de anfitrión. La segunda, el cambio de versiones de herramienta. La tercera, el cambio de la imagen del objetivo, que en el commit declarado se resolvía por la etiqueta latest tanto para DVWA y OWASP Juice Shop como para el propio orquestador n8n (Tabla 2 y Anexo F) y sólo quedó fijada por digest el 2026-09-23. La cuarta es el código del propio sistema: entre la captura de la campaña A y la de la C la rama principal avanzó ochenta y dos commits, con seis cambios en los normalizadores de hallazgos, de modo que las dos campañas no midieron la misma versión del Servicio de Escaneo ni de la capa de normalización (Sección 20). Hay un indicio que apuntaba a la segunda: la duración de Nikto cayó de 120,1 s a 41,7 s, es decir que dejó de agotar el presupuesto fijo de 120 segundos con que su adaptador lo invoca, y pasó además a producir hallazgos que antes no producía. Una herramienta que antes se detenía por límite de tiempo y ahora termina por cuenta propia parecía más compatible con un cambio en su conjunto de comprobaciones —de 2.6.0 a 2.6.1— que con un cambio de procesador. La Sección 12.6.4 acota ese indicio con datos del propio repositorio: una campaña posterior contra el mismo objetivo, sobre el anfitrión de la campaña A y con Nikto 2.6.0, produjo once hallazgos de Nikto por corrida sin dejar de agotar el presupuesto. El cambio de versión no basta entonces para explicar la diferencia, y la causa no queda establecida por esta evaluación.

La consecuencia para este trabajo es directa y conviene enunciarla sin atenuantes. Las cifras de cobertura y de tiempo de las Secciones 12.3 a 12.5 describen el comportamiento del sistema con las versiones de herramienta vigentes el 11 de agosto de 2026, sobre el anfitrión allí especificado. No describen una propiedad estable del pipeline. Una reconstrucción posterior de la imagen del Servicio de Escaneo puede producir, y de hecho produjo, un recuento de hallazgos un 42 % mayor. Mientras las versiones no se fijan explícitamente y se publiquen junto con las métricas, ninguna medición de este trabajo es reproducible en sentido estricto, y la comparación entre rondas sucesivas carece de base. La Sección 17 identifica la fijación de versiones como requisito de cualquier medición futura.

Debe señalarse, por último, que este resultado no invalida las conclusiones cualitativas de las secciones anteriores. La proporción de tiempo dedicada a orquestación se mantiene en el mismo orden en las tres campañas —0,33 %, 0,11 % y 0,15 %—, y la concentración de la dispersión en una única herramienta se observa en las tres. Lo que no se reproduce son los valores absolutos, que es precisamente lo que un lector podría verse tentado a extrapolar.

12.6.4 Campañas presentes en el repositorio y no integradas al análisis

El repositorio declarado en la Sección 20 conserva, además de las tres campañas analizadas, otras tres ejecuciones que este trabajo no integra a sus cifras. Se declaran aquí porque están al alcance de cualquier lector que acepte la invitación a verificar los datos crudos, y porque dos de ellas obligan a acotar afirmaciones de esta misma sección.

La primera es una campaña contra OWASP Juice Shop del 2026-08-22, anterior a las campañas B y C y contenida en el propio commit declarado. De sus cinco corridas una falló y quedó registrada como fallida; de las cuatro restantes, dos produjeron 32 hallazgos y las otras dos, 9 y 14, por ausencia completa del aporte de OWASP ZAP en un caso y de Nuclei en el otro. Las dos siguientes son posteriores a la entrega: una contra DVWA del 2026-09-15, de quince corridas, y una contra Juice Shop del 2026-09-16, de diez. Ninguna de las dos se integra al análisis, y conviene enunciar el motivo con precisión, porque el criterio ingenuo —que corrieron sobre código posterior al declarado— tampoco distinguiría a las campañas analizadas, que tampoco corrieron sobre él (Sección 20). El motivo es otro y es triple: se ejecutaron tres semanas después del período de medición, sobre un anfitrión distinto del de las campañas B y C, y sobre un estado del Servicio de Escaneo que incorpora cambios funcionales posteriores al cierre de la medición, entre ellos una corrección de la autenticación contra DVWA. Comparar sus cifras con las de A, B o C mezclaría esas tres diferencias en un solo número, que es precisamente lo que la Sección 12.6.3 muestra que no puede hacerse.

Lo que muestran, en cambio, sí afecta a la lectura de dos resultados. El primero es la tasa de fallos parciales silenciosos de la Sección 12.6.2. En las tres campañas analizadas la firma se observó en dos de quince corridas. En las dos campañas contra Juice Shop no integradas al análisis, ocho de las trece corridas registradas como completadas carecen total o parcialmente del aporte de al menos una herramienta: seis de las nueve corridas completadas del 2026-09-16 y dos de las cuatro del 2026-08-22. La campaña del 2026-09-15 contra DVWA, por el contrario, no presenta ninguna en quince corridas. La tasa de uno en cinco describe entonces la condición evaluada y no acota el fenómeno: sobre Juice Shop es sustancialmente mayor. La conclusión de fondo no cambia, se refuerza, porque el problema señalado no es la frecuencia del fallo sino la incapacidad del sistema para distinguir una herramienta que no encuentra nada de una que no llega a ejecutarse.

El segundo es el indicio causal de la Sección 12.6.3. La campaña del 2026-09-15 corrió contra DVWA sobre el mismo anfitrión de la campaña A y con las mismas versiones de herramienta que ella, Nikto 2.6.0 y Nuclei v3.11.0, y registró once hallazgos de Nikto en cada una de sus quince corridas con una duración media de esa herramienta de 118,4 s, es decir agotando igualmente su presupuesto fijo de 120 segundos. La observación rompe el vínculo sobre el que se apoyaba el indicio: Nikto 2.6.0 puede aportar hallazgos parseables sobre el mismo objetivo y el mismo anfitrión, y puede hacerlo sin dejar de detenerse por límite de tiempo. El cambio de versión pierde así su condición de explicación preferente y la causa de la diferencia entre las campañas A y C queda sin establecer, lo que es consistente con el argumento de la Sección 14.1: en el período medido ninguna de las variables que determinan el recuento —versiones de herramienta, anfitrión, imagen del objetivo y código del propio sistema— estaba fijada.

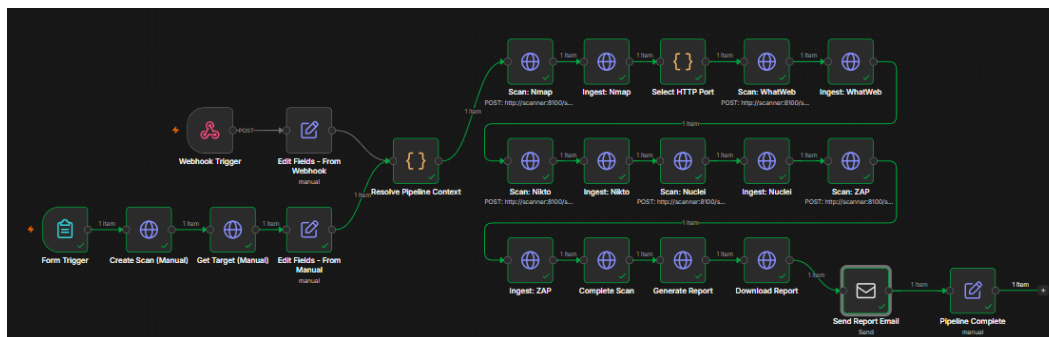
Una última observación completa el cuadro sin contradecir nada. Esa misma campaña del 2026-09-15 registró 43 hallazgos en trece de sus quince corridas y 42 en dos, de modo que la invariancia perfecta del conjunto de hallazgos tampoco se reproduce contra DVWA cuando la muestra crece, aun manteniendo fijo el anfitrión. Es coherente con la lectura que las Secciones 6, 12.6.2 y 14.3 hacen de H2: verificada en la campaña A y refutada como enunciado general. Ninguna de las tres campañas descritas en este apartado altera, por tanto, una conclusión del trabajo; lo que hacen es fijar con mayor precisión el alcance de las que ya estaban enunciadas.

Evidencia Visual del Sistema en Ejecución

Las cuatro capturas siguientes documentan el sistema en ejecución real contra el escaneo d18d49ad-bc52-4d47-96a4-e96985b184d5 (target juice-shop-demo, disparado manualmente vía n8n el 2026-08-14). Cubren las cuatro superficies que el resto del documento describe sólo en prosa: la orquestación (Figura 9) y el informe entregado en sus dos formatos de lectura directa (Figuras 10 y 11). La cuarta es la respuesta cruda de la API sobre la que se construye todo lo anterior (Figura 12).

Figura 9

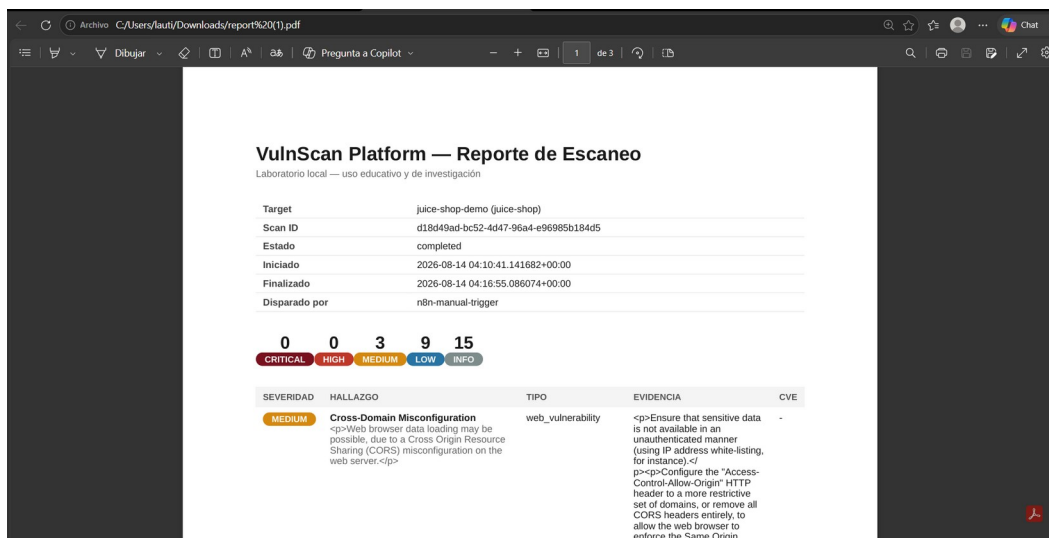
Workflow vulnscan-pipeline completo en n8n, ejecución exitosa de punta a punta (todos los nodos en verde)



Nota. Elaboración propia.

Figura 10

Informe de escaneo generado por el Servicio de Reportes en formato PDF, abierto en el visor de PDF del navegador



Nota. Elaboración propia.

Figura 11

El mismo informe de escaneo en formato HTML, abierto desde el archivo descargado

SEVERIDAD	HALLAZGO	TIPO	EVIDENCIA	CVE
MEDIO	Cross-Domain Misconfiguration -p>Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server -p>	web_vulnerability	-p>Ensure that sensitive data is not available in an unauthenticated manner (using IP address white-listing, for instance) -p>-p>Configure the "Access-Control-Allow-Origin" HTTP header to a more restrictive set of domains, or remove all CORS headers entirely, to allow the web browser to enforce the Same Origin Policy (SOP) in a more restrictive manner -p>	-
MEDIO	Content Security Policy (CSP) Header Not Set -p>Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML, frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files -p>	web_vulnerability	-p>Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header -p>	-
MEDIO	Prometheus Metrics - Detect Prometheus metrics page was detected.	http	http://juice-shop:3000/metrics	-
LOW	Timestamp Disclosure - Unix -p>A timestamp was disclosed by the application/web server - Unix -p>	web_vulnerability	-p>Manually confirm that the timestamp data is not sensitive, and that the data cannot be aggregated to disclose exploitable patterns -p>	-
LOW	Suggested security header missing: referer-policy.	web_misconfiguration	GET /	-
LOW	Suggested security header missing: strict-transport-security.	web_misconfiguration	GET /	-
LOW	Suggested security header missing: permissions-policy.	web_misconfiguration	GET /	-
LOW	Suggested security header missing: content-security-policy.	web_misconfiguration	GET /	-
LOW	Retrieved access-control-allow-origin header: .	web_misconfiguration	GET /	-
LOW	Uncommon header(s) 'x-recruiting' found, with contents: #ijobs.	web_misconfiguration	GET /	-
LOW	robots.txt: Entry 'ftp' is returned a non-forbidden or redirect HTTP code (200).	web_misconfiguration	GET /ftp/	-
INFO	robots.txt: 1 entry which should be removed/removed	web_misconfiguration	GET /robots.txt	-

Nota. Elaboración propia.

Figura 12

Respuesta cruda de GET /scans/{id}/findings contra el Backend, capturada en una terminal PowerShell (Invoke-RestMethod con el header X-API-Key)

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\lauti> Invoke-RestMethod -Uri "http://localhost:8081/scans/d18d49ad-bc52-4d47-96a4-e96985b184d5/findings" -Headers @{ "X-API-Key"="change_me_local_dev_only" } |
ConvertTo-Json -Depth 10
{
  "value": [
    {
      "id": "b1493b0f-0e37-4b86-b784-90a067bfa7a0",
      "scan_id": "d18d49ad-bc52-4d47-96a4-e96985b184d5",
      "scan_task_id": "78d8b997-50e7-4812-b7ec-2eb858e0e34b",
      "service_id": null,
      "title": "Cross-Domain Misconfiguration",
      "description": "\u003cp\u003eWeb browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web se",
      "finding_type": "web_vulnerability",
      "evidence": "\u003cp\u003eEnsure that sensitive data is not available in an unauthenticated manner (using IP address white-listing, for instance).\u003c/p\u003eConfigure the \"Access-Control-Allow-Origin\" HTTP header to a more restrictive set of domains, or remove all CORS headers entirely, to allow th",
      "confidence": "2",
      "cvss_score": null,
      "cvss_vector": null,
      "severity": "medium",
      "created_at": "2026-08-14T04:16:55.026589Z",
      "cve_references": [
      ]
    },
    {
      "id": "5b12f7cd-ab4b-4dc4-5b30-2891b7e425c6",
      "scan_id": "d18d49ad-bc52-4d47-96a4-e96985b184d5",
      "scan_task_id": "78d8b997-50e7-4812-b7ec-2eb858e0e34b",
      "service_id": null,
      "title": "Information Disclosure - Suspicious Comments",
      "description": "\u003cp\u003eThe response appears to contain suspicious comments which may help an attacker.\u003c/p\u003e",
      "finding_type": "web_vulnerability",
      "evidence": "\u003cp\u003eRemove all comments that return information that may help an attacker and fix any underlying problems they refer to.\u003c/p\u003e",
      "confidence": "1",
      "cvss_score": null,
      "cvss_vector": null,
      "severity": "info",
      "created_at": "2026-08-14T04:16:55.026589Z",
      "cve_references": [
      ]
    }
  ]
}
```

Nota. Elaboración propia.

13. Análisis de Riesgos

Todo sistema tecnológico que interactúa con infraestructura de red y procesa información sensible de seguridad conlleva riesgos inherentes que deben ser identificados, evaluados y gestionados explícitamente. El presente análisis aborda los riesgos operativos, de seguridad y de disponibilidad más relevantes del sistema implementado. Sigue la metodología de evaluación cualitativa de riesgos definida por la norma ISO/IEC 27005:2022 (International Organization for Standardization, 2022b), que sustituyó a la edición de 2018. Esa revisión reorganizó la guía en torno al proceso de gestión de riesgos del sistema de gestión de seguridad de la información (International Organization for Standardization, 2022a).

13.1 Activos, Amenazas y Criterios de Aceptación

La norma (International Organization for Standardization, 2022b) estructura la evaluación en torno a tres elementos previos a la valoración de riesgos: el inventario de activos que se protegen, el emparejamiento entre amenazas y las vulnerabilidades que podrían explotar, y los criterios bajo los cuales un riesgo se acepta sin mitigación adicional. Se enuncian aquí para que la valoración de las subsecciones siguientes sea auditable y no meramente declarativa.

Activos. El sistema protege cuatro clases de activo, en orden decreciente de criticidad. Primero, las cuatro claves compartidas que gobiernan el acceso: la del Backend, que autoriza el registro de objetivos y el disparo de escaneos; la interna, que habilita la invocación del Servicio de Escaneo y del de Reportes; el secreto del webhook de n8n; y la clave de retorno con la que n8n reporta el avance y el cierre de cada escaneo al Backend. Segundo, la base de datos de hallazgos, que reúne un inventario de debilidades conocidas del laboratorio y sería material de reconocimiento para un atacante. Tercero, los informes generados, que contienen la misma información en forma consolidada y legible. Cuarto, la capacidad de alcance de red del Servicio de Escaneo, único componente con ruta hacia las aplicaciones objetivo, cuyo uso indebido permitiría dirigir herramientas de escaneo activo contra destinos no autorizados.

Emparejamiento amenaza-vulnerabilidad. La amenaza principal es el acceso no autorizado a la API del Backend desde la red del anfitrión, dado que el servicio publica su puerto y la autenticación descansa en una clave compartida embebida en el paquete del frontend. La vulnerabilidad que la habilita es precisamente la ausencia de un esquema de credenciales por usuario, decisión de alcance documentada en la Sección 10.2. Una segunda amenaza es la invocación directa del Servicio de Escaneo por un componente comprometido de la red interna, cuya vulnerabilidad asociada sería la aceptación de un objetivo arbitrario. El sistema la neutraliza validando la lista blanca en el propio Servicio de Escaneo y no sólo en el Backend. Una tercera es la alteración de las herramientas o plantillas en su origen, tratada en la subsección de riesgos de cadena de suministro.

Criterios de aceptación. El sistema opera exclusivamente en un laboratorio local, aislado de infraestructura productiva y operado por un único responsable. Por eso se aceptan sin mitigación adicional los riesgos de nivel resultante bajo. También se aceptan los de nivel medio cuyo impacto se limite a la disponibilidad del propio laboratorio, sin afectar la confidencialidad de la información ni la integridad de un sistema externo. Los riesgos de nivel alto se aceptan únicamente cuando su mitigación exigiría abandonar una premisa explícita del proyecto, como

ocurre con la ausencia de autenticación por usuario, y en tal caso se documentan como limitación en la Sección 14 en lugar de presentarse como resueltos. **Ningún riesgo con impacto potencial sobre sistemas de terceros se considera aceptable**, criterio que sostiene la doble aplicación de la lista blanca descrita en la Sección 10.2.

13.2 Riesgos Operativos

El riesgo de desactualización de las plantillas de Nuclei es uno de los riesgos operativos de mayor impacto. La cobertura de detección de Nuclei depende de la vigencia de sus plantillas, un conjunto mantenido por la comunidad y actualizado continuamente (ProjectDiscovery, 2024), que en el sistema se embeben en la imagen del Servicio de Escaneo en tiempo de construcción para no penalizar la primera ejecución. Si la imagen no se reconstruye periódicamente, la cobertura se degrada frente a vulnerabilidades publicadas con posterioridad. La probabilidad es media y el impacto alto (falsa sensación de seguridad por informes incompletos). La mitigación recomendada es reconstruir la imagen del Scanner periódicamente y registrar la fecha de las plantillas en los metadatos del escaneo.

El riesgo de fallos de conectividad entre componentes, especialmente entre n8n y los servicios que invoca por HTTP, puede interrumpir la ejecución del pipeline. Este riesgo tiene probabilidad media e impacto moderado, dado que un fallo no genera daño directo sino la ausencia del informe esperado. La mitigación recomendada incluye reintentos automáticos con retroceso exponencial en los nodos HTTP del workflow y notificaciones de error ante ejecuciones incompletas. El sistema ya contempla parcialmente este riesgo: la generación del reporte se ejecuta con tolerancia a fallos, y el Backend marca un escaneo como fallido si no logra disparar el pipeline, evitando escaneos en estado “en ejecución” que nunca progresan.

13.3 Riesgos de Seguridad

La exposición de la interfaz web de n8n en un puerto accesible desde la red abre un riesgo relevante. El acceso no autorizado permitiría ver y modificar los workflows, acceder a credenciales almacenadas y, potencialmente, alterar el comportamiento del pipeline. Este riesgo tiene probabilidad baja en un entorno de laboratorio aislado pero impacto alto. Las mitigaciones incluyen autenticación robusta (el sistema contempla autenticación básica para la interfaz de n8n), restricción del puerto mediante firewall, y uso de HTTPS mediante un proxy inverso con certificado TLS en despliegues expuestos.

El registro y disparo de escaneos a través de la API pública del Backend es otro riesgo de seguridad relevante: un atacante con acceso a la red de la plataforma podría registrar objetivos arbitrarios o disparar pipelines sin autorización. Este riesgo tiene probabilidad baja e impacto alto. La mitigación implementada es el header X-API-Key obligatorio en todo endpoint salvo /health. El riesgo residual —que la key queda inlineada en el bundle JS del Frontend en tiempo de build— se documenta como limitación en la Sección 14.2, no como vulnerabilidad no mitigada.

Los datos que el sistema genera (inventario de servicios y tecnologías, y catálogo de vulnerabilidades detectadas) son información sensible. La interceptación de reportes o el acceso no autorizado a la base de datos representan riesgos de confidencialidad de impacto alto. Las mitigaciones incluyen el cifrado en tránsito, el cifrado en reposo de la base de datos, y controles de acceso basados en roles. Cabe destacar que el sistema ya incorpora dos controles de

seguridad relevantes por diseño: la segmentación de red que impide a la mayoría de los servicios alcanzar el laboratorio, y el autoescapado de los reportes HTML/PDF que previene la inyección de scripts provenientes de la salida de las herramientas.

13.4 Riesgos de Disponibilidad

El sistema depende de la disponibilidad del equipo anfitrión y del motor de contenedores (Docker Inc., 2023). Conviene señalar que la contenerización desplaza el riesgo además de reducirlo: Mills et al. (2023) evaluaron longitudinalmente la seguridad de imágenes Docker reales y documentan que las imágenes base acumulan vulnerabilidades conocidas con el paso del tiempo si no se reconstruyen, de modo que la reproducibilidad que aporta la imagen fija convive con una degradación silenciosa de su superficie de ataque. Un fallo de hardware, del sistema operativo o de Docker resultará en la imposibilidad de ejecutar escaneos durante el período de inactividad. Este riesgo tiene probabilidad baja e impacto moderado. Las mitigaciones recomendadas incluyen monitoreo del anfitrión, políticas de reinicio automático de los contenedores (el sistema ya define restart: unless-stopped en los servicios de larga duración) y la documentación del procedimiento de recuperación.

Tabla 22

Matriz de riesgos identificados, probabilidad, impacto y mitigación

Riesgo	Probabilidad	Impacto	Nivel	Mitigación principal
Desactualización de plantillas Nuclei	Media	Alto	Alto	Reconstrucción periódica de la imagen del Scanner
Fallo de conectividad n8n –servicios	Media	Medio	Medio	Reintentos + tolerancia a fallos (ya parcial)
Acceso no autorizado a n8n	Baja	Alto	Medio	Autenticación + firewall + HTTPS
Interceptación de informes / acceso a la BD	Baja	Alto	Medio	Cifrado en tránsito y en reposo + RBAC
Fallo del equipo anfitrión	Baja	Medio	Bajo	Monitoreo + reinicio automático de contenedores
Acceso no autorizado a la API del Backend	Baja	Alto	Medio	Header X-API-Key obligatorio en todo endpoint salvo /health; ver Sección 14.2 sobre el riesgo residual de la key inlineada en el bundle del

Riesgo	Probabilidad	Impacto	Nivel	Mitigación principal
				Frontend
Alteración de herramientas o plantillas en su origen	Media	Alto	Alto	Verificación de instalación con fallo explícito en la construcción; registro de la versión efectiva

Nota. Elaboración propia, siguiendo la metodología de ISO/IEC 27005:2022. Las mitigaciones de la última columna no comparten el mismo estatus, y conviene distinguirlas. Están implementadas en el sistema evaluado la verificación de instalación de plantillas con fallo explícito en la construcción de la imagen y el header X-API-Key obligatorio en todo endpoint salvo /health. Las restantes —cifrado en tránsito y en reposo, control de acceso basado en roles, firewall, HTTPS, monitoreo y reinicio automático— son controles previstos para un despliegue productivo y no forman parte del alcance de este trabajo (Sección 14.2).

Cadena de suministro de herramientas y plantillas. La imagen del Servicio de Escaneo resuelve en tiempo de construcción la versión de Nuclei, su conjunto de plantillas, Nikto y OWASP ZAP desde sus repositorios de origen, sin fijar versión. Esto mantiene el sistema actualizado sin intervención, pero traslada al proceso de construcción una dependencia de terceros que el proyecto no controla. El riesgo se materializó de forma verificable: una construcción de la imagen produjo un Nuclei sin plantillas instaladas, y como el paso de instalación devolvía código de salida cero pese a no haber instalado nada, el fallo sólo se manifestaba en tiempo de escaneo, con el sistema privado de la única herramienta capaz de emitir severidad crítica. La mitigación aplicada verifica que el directorio de plantillas quede efectivamente poblado y hace fallar la construcción en caso contrario, además de registrar la versión instalada; fijar las versiones de las cinco herramientas queda identificado como trabajo futuro.

Camino de excepción ante ausencia de servicio web. Cuando Nmap no detecta ningún puerto HTTP abierto sobre el objetivo, el nodo de selección de puerto deriva la ejecución a una rama que marca el escaneo como fallido con un mensaje explícito, en lugar de continuar hacia las herramientas de aplicación web sin destino válido. Esta decisión evita el modo de fallo más problemático de un pipeline orquestado: un escaneo que permanece indefinidamente en estado de ejecución porque ningún componente lo cierra.

14. Limitaciones del Estudio

Una evaluación rigurosa y académicamente honesta del presente trabajo exige el reconocimiento explícito de las limitaciones que condicionan el alcance y la generalizabilidad de sus resultados, organizadas en tres categorías.

14.1 Limitaciones del Entorno de Evaluación

La evaluación se realizó en un entorno de laboratorio contenerizado de pequeña escala, compuesto por dos aplicaciones web deliberadamente vulnerables. Las campañas de medición se ejecutaron sobre dos máquinas anfitrionas de arquitectura distinta (Sección 12.6), lo que permite contrastar la estabilidad de los resultados pero sigue siendo insuficiente para caracterizar su dependencia del hardware. La comparación de la Sección 12.6.3, que repite la medición sobre el mismo objetivo, muestra que la dependencia de las condiciones de ejecución no es menor: el recuento de hallazgos varió un 42,5 % entre ambas campañas. Esa variación no es atribuible al anfitrión, porque entre una y otra cambiaron además las versiones de herramienta, las imágenes del laboratorio y el código del sistema (Secciones 9.3, 12.6.3 y 20). Este tamaño reducido implica que los tiempos de ejecución registrados no pueden extrapolarse directamente a entornos de producción con mayor número y diversidad de objetivos. En particular, el tiempo del escaneo activo de ZAP, que domina el pipeline en la campaña B, escala con la complejidad de la aplicación analizada; en aplicaciones mayores, el tiempo total podría incrementarse en una proporción que este laboratorio no permite estimar. Las métricas de tiempo deben tratarse como una referencia acotada al contexto experimental específico.

Las aplicaciones del laboratorio (Juice Shop, DVWA) están diseñadas para ser detectablemente vulnerables. Sin embargo, la medición de recall/precisión contra los catálogos oficiales (Sección 9.3(c) y Sección 12.5) muestra que esto no se traduce en tasas de detección de alta confianza en la práctica (ver limitación técnica específica más abajo), lo que matiza, en vez de confirmar, la expectativa de un escenario favorable. Este sesgo de selección implica que los resultados de detección representan un escenario favorable y no el caso promedio de un entorno productivo, donde la cobertura efectiva de las herramientas puede ser menor (especialmente frente a vulnerabilidades de lógica de negocio propia, configuraciones específicas o vulnerabilidades de día cero).

Una limitación adicional del entorno, de naturaleza distinta a las anteriores, es que la imagen del Servicio de Escaneo resuelve en tiempo de construcción la versión de cada herramienta sin fijarla, de modo que las cifras reportadas describen el conjunto de comprobaciones vigente en la fecha de la campaña y no una propiedad estable del sistema (ver Sección 13). Esta limitación dejó de ser conjetural: la réplica de la Sección 12.6.3 la documenta con datos propios. Que esto importa no es una conjetura del presente trabajo: Potti et al. (2025) dedican un estudio completo a comparar dos versiones consecutivas de OWASP ZAP, la 2.12.0 y la 2.13.0, contra el OWASP Benchmark, precisamente porque el rendimiento de detección varía entre versiones de una misma herramienta. Si dos versiones sucesivas de un mismo escáner justifican un estudio comparativo, una campaña de medición que no declara qué versión ejecutó no es reproducible en sentido estricto. Fijar las versiones de las cinco herramientas y publicarlas junto con las métricas queda identificado como requisito de cualquier ronda de medición posterior (Sección 17).

14.2 Limitaciones Técnicas del Sistema

El comportamiento predeterminado del pipeline es el escaneo no autenticado: analiza las aplicaciones desde la perspectiva de un cliente externo sin credenciales. Ese modo tiene una cobertura inherentemente limitada, porque no detecta debilidades que solo son visibles con acceso autenticado a la aplicación. El sistema implementa además un modo autenticado opcional (`options.authenticated=true` del Servicio de Escaneo), que no forma parte del workflow predeterminado y sobre el que se construyen las Tablas 17 y 18 (Sección 12.5). Lo que queda como trabajo futuro es generalizar esa autenticación a objetivos distintos de DVWA e integrar un motor de escaneo de red autenticado (ver Sección 17).

El modelo de autenticación es una API key compartida (X-API-Key), no un sistema de usuarios: no hay auditoría de quién disparó cada escaneo, no hay revocación de acceso individual, y la key queda visible en el bundle JS del Frontend (inlineada en tiempo de build). Es una decisión de alcance explícita para un laboratorio de un solo operador, no una omisión.

La orquestación mediante n8n ejecuta las herramientas de aplicación web secuencialmente dentro de un único workflow. En escenarios donde se requiera analizar simultáneamente múltiples objetivos o ejecutar los escaneos concurrentemente, esta linealidad puede convertirse en un cuello de botella. La ejecución paralela es una mejora razonable pero no un requisito de corrección para el alcance actual. El Frontend de visualización (React + TypeScript, ver Sección 10.1) sí forma parte del sistema implementado y fue verificado visualmente contra el stack en ejecución. No obstante, no se sometió a una evaluación de usabilidad con usuarios externos, limitación que se retoma más abajo (Sección 14.3).

La clasificación de severidad del sistema se basa exclusivamente en CVSS (Sección 8.5): no incorpora el Exploit Prediction Scoring System (EPSS) ni el catálogo Known Exploited Vulnerabilities (KEV) de CISA, ambos complementarios a CVSS para priorizar remediación según probabilidad real de explotación en vez de severidad técnica aislada. Incorporar estas señales queda como trabajo futuro (ver Sección 17).

El modo de invocación de ZAP (`-quickurl/-quickout`, ver Anexo B) usa el rastreador clásico, que no ejecuta JavaScript. Sobre Juice Shop (una SPA construida en Angular, cuya navegación se resuelve enteramente del lado del cliente) este rastreador no puede descubrir las rutas ni los parámetros que la aplicación genera en tiempo de ejecución. En consecuencia, el escaneo activo de ZAP contra Juice Shop queda acotado casi por completo a la página raíz servida estáticamente y a comprobaciones pasivas de cabeceras HTTP (ver Sección 12.5 para las cifras de recall/precisión por herramienta contra el catálogo público de Juice Shop). Esta limitación compromete la afirmación de “cobertura estratificada” del §8.4 en lo que respecta específicamente a ZAP sobre objetivos de tipo SPA. No se incorpora el rastreador AJAX de ZAP en esta ronda por ser un cambio de comportamiento del sistema bajo evaluación, no una corrección de redacción; queda como línea de trabajo futuro (ver Sección 17).

El sistema no detecta fallos parciales de una herramienta. Un escaneo se marca como completado cuando todas las invocaciones devuelven una respuesta satisfactoria, sin verificar que cada herramienta haya producido un resultado coherente con su comportamiento habitual. La Sección 12.6.2 documenta una ejecución en la que Nuclei no aportó ningún hallazgo y terminó en una fracción de su tiempo típico, sin que el pipeline lo señalara de ninguna forma. Incorporar una verificación de plausibilidad por herramienta —por ejemplo, contrastar duración y volumen de salida contra los valores registrados en ejecuciones anteriores sobre el mismo objetivo— queda identificado como trabajo futuro (Sección 17).

14.3 Limitaciones del Diseño del Estudio

El número de ejecuciones registradas es de quince para el sistema automatizado, repartidas en tres campañas de cinco corridas no acumulables entre sí (Sección 9.8), y de dos para el proceso manual, insuficiente para sustentar afirmaciones estadísticas inferenciales con un nivel de confianza aceptable. Con muestras de este tamaño no es posible calcular intervalos de confianza ni aplicar pruebas de hipótesis paramétricas. Las conclusiones cuantitativas deben interpretarse estrictamente como observaciones descriptivas preliminares, sin fuerza probatoria estadística independiente: el bloque de la Sección 6 conserva una única hipótesis contrastable, verificada en la campaña A y refutada como enunciado general en las dos réplicas. Cualquier generalización requiere validación en estudios con mayor tamaño muestral y mayor diversidad de entornos.

Adicionalmente, el brazo manual se ejecutó con los propios autores como operadores, lo que introduce un sesgo de familiaridad de dirección indeterminada que la repetición entre ellos no corrige: su conocimiento profundo de las herramientas puede haber reducido el tiempo respecto al de un operador sin experiencia, comprimiendo la diferencia observada. Pero un analista con práctica rutinaria podría, a su vez, ejecutarlo en menos tiempo que el registrado aquí. Esta es una razón adicional, más allá del tamaño muestral de dos corridas, para no sobreinterpretar la magnitud exacta de la comparación cuantitativa reportada en la Sección 12.4. Por último, los informes generados automáticamente no fueron evaluados por usuarios externos, por lo que la dimensión de utilidad operativa de los documentos producidos permanece sin validación empírica en este trabajo.

Una limitación de distinto orden afecta a H2. Tal como quedó operacionalizada —variación del conjunto de hallazgos entre ejecuciones sucesivas del pipeline contra el mismo objetivo— su poder falsador es débil: el pipeline es determinista y el objetivo permanece inalterado entre corridas, de modo que el resultado esperable es precisamente el que se obtuvo, cero variación. Que la hipótesis se sostenga confirma que la implementación no introduce no determinismo observable, lo cual es un requisito de ingeniería verificado y no un hallazgo empírico. Una formulación con capacidad real de refutación exigiría introducir una fuente genuina de variación —distintos objetivos, distintas máquinas anfitrionas, o reconstrucciones de la imagen con versiones de herramientas actualizadas, que la Sección 14.1 identifica como fuente de variación demostrada—. Ese diseño se ejecutó parcialmente: la réplica de la Sección 12.6 introduce un segundo objetivo y un segundo anfitrión como fuentes de variación, y bajo esa condición H2 no se sostiene. La hipótesis debe leerse en consecuencia, y en los mismos términos que las Secciones 6 y 12.6.2, como una verificación de determinismo que se sostuvo en la campaña A y dejó de sostenerse al variar el objetivo o el anfitrión —incluida la campaña C, que corrió contra el mismo DVWA—, y no como una prueba de consistencia en sentido fuerte.

15. Discusión

Los resultados de la Sección 12 admiten una lectura en tres planos: qué permiten afirmar sobre el sistema construido, cómo se sitúan frente a los antecedentes del área, y qué queda fuera del alcance de lo que este diseño experimental puede establecer.

15.1 La orquestación no es donde está el tiempo

El resultado cuantitativo más nítido es también el menos intuitivo: de los 259,06 s que tarda una ejecución completa del pipeline, sólo 0,85 s (el 0,3 %) corresponden a orquestación. El 99,7 % restante es ejecución de herramientas, un piso que ningún operador, humano o automatizado, puede comprimir.

Esto reordena la contribución del trabajo. La automatización no acelera el análisis porque haga más rápido lo que las herramientas hacen. Lo hace porque elimina el trabajo humano de transición entre ellas: consolidar salidas heterogéneas, decidir qué herramienta aplicar sobre qué servicio, clasificar severidades bajo un criterio uniforme y redactar el informe. Ese trabajo aparece cronometrado en la Sección 12.2: consolidar y clasificar los hallazgos, y redactar el informe, insumieron juntos 47 minutos 10 segundos de trabajo atendido en la corrida manual A y 46 minutos 23 segundos en la B —46 minutos 47 segundos de media—, frente a los 0,24 s de orquestación del pipeline. Su eliminación es además verificable por inspección: el sistema produce un informe estructurado en cuatro formatos sin intervención entre la solicitud y la entrega.

Un segundo resultado corrige una suposición del propio trabajo. El diseño original del pipeline asumía que el escaneo activo de OWASP ZAP dominaría el tiempo total, supuesto que la Sección 11.6 recoge explícitamente. Contra DVWA no es así: **Nikto consume el 46 % del tiempo de herramientas** (120,1 s de media) frente al 20 % de ZAP (52,1 s). La descomposición de la varianza (Sección 12.3.1) despeja el punto: la desviación estándar de Nikto sobre las cinco corridas es de 0,01 s, de modo que su duración no mide nada del objetivo. La causa es de diseño y no de la herramienta, ya que el adaptador invoca Nikto con un presupuesto fijo de 120 segundos, de modo que su duración es una decisión de configuración y no una propiedad del objetivo. La suposición original resultó correcta contra Juice Shop, cuya superficie es mayor: la campaña de la Sección 12.6 la midió y allí ZAP consume el 51,5 % del tiempo de herramientas frente al 25,7 % de Nikto, la relación exactamente inversa a la observada contra DVWA.

15.2 La cobertura estratificada depende del objetivo

La Sección 8.4 justifica el conjunto de cinco herramientas por su cobertura estratificada: cada una atacaría una capa distinta del objetivo. Los datos de solapamiento muestran que esa propiedad no es general, sino condicional al objetivo analizado.

Contra DVWA sin autenticar, las herramientas resultaron redundantes: las tres entradas del catálogo cubiertas lo fueron simultáneamente por Nuclei y por ZAP, ninguna aportó cobertura exclusiva, y el recall del sistema completo coincide con el de cualquiera de sus componentes por separado. Bajo la condición autenticada y con anotación manual (Sección 12.5) el cuadro cambia: ZAP aporta dos de las tres entradas cubiertas, ausencia de token anti-CSRF e inyección SQL, que ninguna otra herramienta encuentra, de modo que la redundancia observada depende

también de la condición de acceso y no sólo del objetivo. Contra Juice Shop el resultado se invierte: solapamiento nulo, con ZAP aportando cuatro entradas y Nikto una, todas exclusivas. La Sección 12.5.1 añade el reverso de esa observación: en ese mismo objetivo Nuclei emite trece hallazgos sin una sola correspondencia, y el 78,6 % de lo que el pipeline entrega no coincide con ninguna entrada del catálogo. La estratificación, cuando el objetivo la permite, se paga en triaje.

La lectura conjunta es que la estratificación produce cobertura adicional cuando hay superficie que estratificar. Sobre una aplicación cuya superficie accesible sin autenticar se reduce a cabeceras HTTP y configuración del servidor, las herramientas convergen sobre los mismos hallazgos.

Esta observación admite un contraste inusualmente directo con la literatura. Amankwah et al. (2020) compararon ocho escáneres de vulnerabilidades web, comerciales y de código abierto, entre ellos OWASP ZAP, contra WebGoat y DVWA, empleando precisión y recall entre sus métricas de evaluación. Coinciden con este trabajo en la herramienta, en uno de los dos objetivos y en dos de las métricas, lo que vuelve el contraste especialmente informativo. Su conclusión respalda la decisión de diseño que sostiene este sistema: encuentran que, si bien los escáneres comerciales resultan eficaces, algunos de código abierto, ZAP entre ellos, también lo son. Cierran recomendando mejorar la cobertura, elevar las tasas de detección y reducir los falsos positivos, las tres dimensiones que el presente trabajo mide y en las que reporta resultados modestos.

La diferencia metodológica entre ambos estudios es, sin embargo, la que más importa para interpretar las cifras propias. Aquel trabajo dispone de una correspondencia validada entre hallazgo y vulnerabilidad conocida; la medición presentada aquí no la construyó, y empareja por coincidencia léxica. Las cifras de precisión y recall de las Tablas 15 a 17 no son, por tanto, comparables con las suyas: son indicativas del orden de magnitud bajo un criterio de emparejamiento más débil. **Esa distancia metodológica, y no una diferencia de capacidad entre herramientas, es la explicación más probable de la brecha entre ambos conjuntos de resultados.**

Un segundo contraste, más reciente y de mayor escala, apunta en la misma dirección. Qadir et al. (2025) evaluaron nueve herramientas de prueba de seguridad, cuatro de análisis estático y cinco de análisis dinámico entre las que se cuenta OWASP ZAP, sobre setenta y cinco aplicaciones web reales de los sectores de salud, educación y tecnología. Dos de sus resultados son directamente pertinentes aquí. El primero respalda la elección de herramienta de este trabajo: ZAP resultó la de mejor desempeño entre las herramientas de análisis dinámico evaluadas. El segundo es más incómodo y más útil: el conjunto completo de nueve herramientas no detectó ninguna vulnerabilidad en una de las categorías del OWASP Top 10 ni en ocho categorías CWE. Es decir que la existencia de clases enteras de vulnerabilidad que el instrumental automático no alcanza no es una particularidad de este pipeline, sino un límite documentado del enfoque. El resultado propio, ocho de once entradas del catálogo no detectadas y todas ellas dependientes de completar un flujo de la aplicación, se lee mejor como una instancia de ese límite general que como una deficiencia de la implementación.

15.3 La severidad reportada es en parte un artefacto del criterio

La distribución de severidad de la Sección 12.5 no describe únicamente al objetivo: describe también la política de clasificación del sistema. La Sección 12.5.2 lo cuantifica: de los 116 hallazgos producidos en las tres condiciones, sólo tres traen una puntuación CVSS emitida por la herramienta. En el 97,4 % restante la severidad la asigna la política de normalización. La escala de ZAP alcanza cuatro niveles frente a los cinco del sistema, de modo que ninguna de sus cuarenta alertas puede clasificarse como crítica. Los veinticuatro hallazgos de Nikto se fuerzan sin excepción a severidad baja por no aportar señal propia. En consecuencia, la fila “crítica” sólo puede alimentarse de Nuclei —y en efecto sus dos únicos ocupantes provienen de allí—, y la fila “baja” está inflada por decisión de diseño.

Esta dependencia conecta con una discusión activa en el área sobre los límites de CVSS como criterio de priorización. Jacobs et al. (2021) proponen EPSS precisamente porque la puntuación de severidad, tomada aisladamente, no ordena bien el trabajo de remediación: no incorpora probabilidad de explotación real. El problema que enfrenta este sistema es anterior y más elemental, homogeneizar escalas heterogéneas antes de poder priorizar, pero apunta en la misma dirección: **una severidad normalizada es una construcción del sistema que la normaliza, y presentarla como propiedad del objetivo sería incorrecto**. El trabajo adopta por eso la práctica de reportar la distribución siempre desagregada por herramienta.

15.4 Qué aporta el enfoque de orquestación

El uso de n8n como orquestador, con la ejecución y la normalización alojadas en servicios propios, sitúa a este trabajo en la misma familia que las plataformas SOAR. Bridges et al. (2023) evaluaron herramientas SOAR con operadores de cuatro centros de operaciones de seguridad sobre cinco tipos de investigación estructurada, y su hallazgo principal interpela directamente a un trabajo como este: la sobreautomatización preocupaba a los analistas más experimentados, y las herramientas preferidas fueron las que equilibraban la automatización con el apoyo a la decisión humana. El diseño aquí presentado conserva deliberadamente dos decisiones dentro del orquestador, la selección del puerto a analizar y la resolución del contexto de ejecución, en lugar de automatizar el ciclo por completo. Esa decisión no se tomó por el argumento de Bridges et al., sino por razones de implementación, pero coincide con él, y conviene señalar la coincidencia antes que presentar la automatización total como una virtud en sí misma. Marandi et al. (2023), por su parte, automatizaron escaneo de seguridad dentro de un pipeline DevSecOps de integración continua. Comparten con este trabajo el objetivo de eliminar la intervención manual entre etapas y difieren en el punto de integración, ya que la integración continua se dispara por cambios en el código mientras que el diseño presentado se dispara por objetivo.

La decisión arquitectónica de mantener la lógica de ejecución, normalización y clasificación en servicios versionados, y no en nodos del workflow, es la que hace que agregar una herramienta sea un adaptador más una línea de registro. Nasab et al. (2023) analizaron 861 puntos de seguridad extraídos de diez sistemas de microservicios de código abierto y de 306 consultas de Stack Overflow, y derivaron un catálogo de 28 prácticas de seguridad que validaron con 74 profesionales del área. Su motivación explícita es una brecha de conocimiento entre quienes construyen sistemas de microservicios y lo que su aseguramiento efectivamente exige. Las decisiones de seguridad de la Sección 10.2 pertenecen a esa clase de prácticas concretas y verificables: la lista blanca aplicada en dos puntos independientes, la validación fail-fast al

arranque y la ejecución de contenedores sin privilegios de root. Eso permite situar este trabajo dentro de esa conversación, en lugar de invocar la arquitectura de microservicios como una virtud genérica.

Esta preocupación por dónde reside la lógica de seguridad no es marginal en la literatura reciente. Mohammed et al. (2025), en una revisión sistemática de treinta y seis estudios publicados entre 2012 y 2025, identifican la orquestación de cadenas de herramientas y la integración fluida de herramientas de seguridad entre las brechas abiertas del área, junto con la ausencia de marcos completos para su despliegue efectivo. El presente trabajo no resuelve esa brecha, que es de alcance mucho mayor, pero sí ofrece un punto de datos concreto sobre una de sus dimensiones. Mantener la ejecución y la normalización fuera del orquestador reduce la incorporación de una herramienta nueva a dos modificaciones localizadas y versionadas. La alternativa es codificarla en nodos del workflow, donde la lógica queda fuera del control de versiones y de la suite de pruebas.

15.5 Lo que este diseño no permite afirmar

Tres afirmaciones quedan fuera del alcance de la evidencia producida, y conviene enunciarlas con precisión.

Primero, **la capacidad de detección del sistema quedó establecida sólo de forma parcial**. El emparejamiento automático por coincidencia de CVE exacto o por tipo más ubicación no produjo resultados en ninguno de los dos laboratorios. La anotación manual de la Sección 12.5 acota ese resultado: contra DVWA autenticado el pipeline cubre tres de once entradas con correspondencia verificada, y las ocho restantes corresponden a vulnerabilidades que exigen completar un flujo de la aplicación. La causa del bajo recall no es únicamente la del instrumento de emparejamiento: los catálogos de verdad de referencia describen desafíos en el vocabulario de la aplicación, y las herramientas reportan en el vocabulario de la comprobación técnica. Salvar esa distancia exige anotación manual de la correspondencia, realizada para DVWA autenticado y reportada en la Tabla 18.

Conviene señalar que el patrón observado, detección de lo expuesto en la respuesta del servidor y fracaso ante lo que exige completar un flujo, reaparece en trabajos que evalúan instrumental mucho más sofisticado. Liu et al. (2025) evaluaron veinte agentes basados en modelos de lenguaje sobre tres vulnerabilidades representativas y llevaron los tres de mejor desempeño a un conjunto de ochenta CVE reales de siete tipos de vulnerabilidad. Encontraron que alcanzan un desempeño razonable en vulnerabilidades simples de biblioteca pero fracasan sistemáticamente en las que requieren entornos de múltiples componentes; su rendimiento cae más de un treinta y tres por ciento cuando la información de autenticación es incompleta. Ambas observaciones coinciden con las de este trabajo, incluida la sensibilidad a la autenticación que la Sección 12.5 documenta. Esto no vuelve mejor el resultado propio, pero sí sugiere que la frontera encontrada es la del enfoque automatizado frente a la lógica de aplicación, y no un artefacto de la selección de herramientas de este pipeline.

Segundo, **la reducción de tiempo frente a un operador humano se cuantificó** sobre dos corridas (Sección 12.4), no sobre una muestra que sostenga inferencia estadística. La Sección 9.9 declara el sesgo de familiaridad de los operadores, que son los propios autores del sistema, como amenaza a la validez de esa cifra concreta. La magnitud exacta debe leerse con esa

salvedad, aunque la dirección del resultado —una reducción de órdenes de magnitud en el tiempo atendido— es consistente con la composición del tiempo del pipeline reportada en la Sección 12.3.

Tercero, **la validez externa está acotada al laboratorio**. Quince ejecuciones repartidas en tres campañas, sobre dos objetivos y dos máquinas anfitrionas, no sostienen inferencia estadística ni generalización a entornos de producción: cada condición conserva un tamaño de cinco, y la comparación entre campañas está confundida por el cambio simultáneo de varias variables (Sección 12.6). Los estadísticos de la Sección 12 son descriptivos.

Sostener estas tres limitaciones simultáneamente con los resultados positivos no debilita el trabajo: delimita qué parte de él es conocimiento establecido y qué parte es artefacto construido y verificado. La contribución central es del segundo tipo, y es verificable con independencia, ya que el repositorio, el commit del código y los artefactos crudos de las tres campañas y de las dos corridas manuales están publicados y declarados en la Sección 20.

16. Conclusiones

16.1 Respuesta a las Preguntas de Investigación

Antes de enunciar las conclusiones generales, corresponde responder explícitamente las cinco preguntas formuladas en la Sección 5, indicando en cada caso dónde se apoya la respuesta y con qué grado de certeza se sostiene.

Pregunta 1, sobre la automatización del ciclo completo y lo que resiste la delegación. El ciclo se automatiza efectivamente: una sola invocación desencadena descubrimiento, fingerprinting, escaneo con cinco herramientas, normalización, persistencia y generación de informes en cuatro formatos, sin intervención humana entre etapas (Secciones 10 y 11). Dos decisiones, sin embargo, resisten la delegación y permanecen dentro del orquestador: la resolución del contexto de ejecución entre sus dos disparadores y la selección del puerto sobre el cual atacar, esta última derivada de la respuesta de Nmap (Sección 11.6). Ambas son decisiones de negocio con consecuencias de seguridad, y afirmar que el orquestador no contiene lógica alguna sería inexacto. La formulación que el trabajo sostiene es más acotada: la ejecución, la normalización y la clasificación residen en servicios propios versionados, y el orquestador conserva únicamente esas dos decisiones.

Pregunta 2, sobre la reducción de tiempo frente a un operador humano. Se responde con dos corridas cronometradas del brazo manual, una por cada autor (Sección 12.2, Anexo K): el proceso manual insumió 1 h 31 min 54 s y 1 h 30 min 20 s de trabajo atendido, y 1 h 39 min 03 s y 1 h 39 min 12 s de tiempo de reloj, frente a los 0,24 s de orquestación y 167,25 s de tiempo total del pipeline automatizado en condiciones equivalentes (Sección 12.4). El alcance de esas cifras debe declararse sin atenuantes: provienen de dos corridas, ejecutadas por los propios autores del sistema, con el sesgo de familiaridad que la Sección 9.9 describe y que un segundo autor no corrige. No son una medición con fuerza estadística, pero la dirección y el orden de magnitud de la reducción son consistentes con la composición del tiempo del pipeline reportada en la Sección 12.3.

Pregunta 3, sobre la normalización y lo que se pierde en ella. La normalización es posible y está implementada: cinco formatos de salida heterogéneos convergen en una representación común de hallazgo, servicio y tecnología (Sección 11.4). La pérdida de información es real y acotada, y el trabajo la documenta en lugar de negarla: Nikto no aporta señal de severidad, de modo que sus hallazgos se clasifican conservadoramente como severidad baja. La escala de cuatro niveles de ZAP se proyecta sobre una de cinco, lo que impide que cualquier alerta suya alcance la categoría crítica. La consecuencia, cuantificada en las Secciones 12.5 y 15.3, es que la distribución de severidad resultante describe en parte al criterio de clasificación y no sólo al objetivo analizado.

Pregunta 4, sobre la proporción del tiempo dedicada a la coordinación. Se responde con medición directa: sobre las cinco ejecuciones de la campaña A, la coordinación entre etapas, incluida la generación del informe, consume 0,85 s de media frente a 259,06 s de pipeline total, esto es, el 0,3 % (Sección 12.3). El 99,7 % restante corresponde a la ejecución de las herramientas, un piso común a cualquier operador. La generación del informe en los cuatro formatos se verificó por separado en menos de un segundo por formato. La lectura correcta de

este resultado no es que la automatización sea veloz, sino que el margen de mejora por optimización del orquestador es despreciable frente al costo de las herramientas que coordina. La descomposición de esa medición por herramienta, en la Sección 12.3.1, identifica además el origen de la variabilidad del tiempo total.

Pregunta 5, sobre qué clases de vulnerabilidad detecta el pipeline y cuáles no. Se responde mediante la anotación manual del conjunto de verdad de referencia (Sección 12.5, Tabla 18). Sobre DVWA autenticado, el pipeline cubre tres de las once entradas del catálogo: ausencia de política de seguridad de contenido, ausencia de token anti-CSRF e inyección SQL. Las ocho restantes no se detectan, y su distribución no es aleatoria: corresponden en su totalidad a vulnerabilidades que exigen completar un flujo de la aplicación, como subir un archivo y verificar su ejecución, inyectar carga en un formulario u observar identificadores de sesión sucesivos. La característica del diseño que explica esa frontera es el alcance del rastreo, ya que OWASP ZAP se invoca en modo de una sola pasada con rastreador clásico, que descubre poca superficie sobre una aplicación de navegación autenticada. De ahí que la ampliación del rastreo sea la línea de trabajo futuro priorizada en la Sección 17.

16.2 Hallazgos principales

Además de responder las preguntas planteadas, la evaluación produjo seis resultados que no estaban anticipados en el diseño del estudio y que conviene enunciar por separado, porque describen propiedades del sistema construido que no se deducen de su arquitectura.

Primero, el costo de la orquestación es despreciable frente al de la ejecución. De los 259,06 s que tarda una ejecución completa, 0,85 s corresponden a coordinación entre etapas: el 0,3 %. El 99,7 % restante es tiempo de herramienta, un piso que ningún operador puede comprimir. La contribución de la automatización no está, por tanto, en acelerar el análisis, sino en eliminar el trabajo humano de transición entre etapas (Sección 12.3).

Segundo, la variabilidad del pipeline se concentra siempre en una sola herramienta, pero no siempre en la misma. En la campaña A, contra DVWA, el escaneo activo de OWASP ZAP concentra el 96,1 % de la varianza del tiempo total pese a representar el 20,2 % de su duración media. La correlación entre su duración y la total es de 0,998, y excluido ZAP el coeficiente de variación del pipeline cae de 4,14 % a 0,34 %. Las dos réplicas invierten el cuadro: en la campaña B, contra Juice Shop, y en la C, contra el mismo DVWA de la campaña A, la varianza de ZAP es nula y la dispersión corresponde a Nuclei, con el 101,0 % y el 98,6 % respectivamente. La identidad de la herramienta dominante no depende entonces del objetivo, porque cambió sin que el objetivo cambiara (Sección 12.6.1). En el extremo opuesto, la desviación estándar de Nikto sobre cinco corridas es de 0,01 s, porque su adaptador lo invoca con un presupuesto fijo: su duración es una constante de configuración y no una medición del objetivo. Este resultado da mecanismo a la estabilidad enunciada por H2, pero no permite señalar un punto de intervención estable: en cada condición la dispersión tiene un origen único e identificable, y cuál sea no puede anticiparse sin medirlo (Sección 12.3.1).

Tercero, la utilidad de las herramientas depende del objetivo, salvo en un caso. La precisión de OWASP ZAP se mantiene entre 0,73 y 0,92 a través de las tres condiciones evaluadas, mientras que la de Nuclei cae de 0,79 a 0,00 y la de Nikto de 0,47 a 0,11 al pasar de DVWA a

Juice Shop. La cobertura estratificada que justifica el conjunto de cinco herramientas produce cobertura adicional sólo cuando el objetivo ofrece superficie que estratificar (Sección 12.5.1).

Cuarto, la severidad reportada es, en su casi totalidad, una construcción del sistema. De los 116 hallazgos producidos en las tres condiciones, sólo tres traen una puntuación CVSS emitida por la herramienta; en el 97,4 % restante la severidad la asigna la política de normalización. Los veinticuatro hallazgos de Nikto están clasificados como severidad baja sin excepción, y OWASP ZAP no puede aportar un solo hallazgo crítico por construcción de su escala. Presentar esa distribución como propiedad del objetivo analizado sería incorrecto, y por esa razón todo este trabajo la reporta desagregada por herramienta (Sección 12.5.2).

Quinto, la consistencia entre ejecuciones no es una propiedad general del sistema. Dos réplicas sobre un segundo anfitrión refutan H2 como enunciado general. En la campaña original, contra DVWA, las cinco corridas produjeron conjuntos idénticos. En las dos réplicas no: contra Juice Shop una de cinco produjo 31 hallazgos en lugar de 44, y contra el mismo DVWA sobre el otro anfitrión una produjo 44 en lugar de 46. La consistencia perfecta se observó en una de tres campañas, y el componente responsable fue distinto en cada caso —Nuclei en una, ZAP en la otra—. El escaneo se registró igualmente como completado, lo que revela que el sistema no distingue una herramienta sin resultados de una que no llegó a ejecutarse. La causa raíz del episodio no quedó establecida —no se reprodujo en invocaciones aisladas— pero el hecho y su consecuencia sobre la hipótesis sí (Sección 12.6.2).

Sexto, las cifras absolutas no son reproducibles fuera de las condiciones en que se midieron. La tercera campaña repitió la medición original sobre el mismo objetivo, trece días después, sobre otro anfitrión y con dos herramientas actualizadas sin intervención: el tiempo total cayó un 35,4 % y el recuento de hallazgos aumentó un 42,5 %, de 32 a 45,6 de media. La causa no queda establecida: el indicio disponible apuntaba al cambio de versiones antes que al de hardware, porque Nikto dejó de agotar su presupuesto fijo de 120 segundos y pasó a aportar catorce hallazgos por corrida donde antes no aportaba ninguno parseable, pero una campaña posterior con la versión anterior de Nikto produjo hallazgos sin dejar de agotar ese presupuesto (Sección 12.6.4). Las conclusiones cualitativas se mantienen en las tres campañas; los valores absolutos, no (Sección 12.6.3).

Los seis comparten una característica que conviene explicitar: ninguno se obtuvo midiendo el sistema contra un patrón externo, sino descomponiendo su propio comportamiento. Es el tipo de conocimiento que un artefacto construido permite producir sobre sí mismo, y su alcance está acotado en consecuencia al laboratorio, a las quince corridas registradas en tres campañas y a los dos objetivos evaluados (Sección 14).

El presente trabajo construyó, en el entorno de laboratorio definido y con el alcance experimental descrito en la metodología, una plataforma automatizada de análisis de vulnerabilidades de aplicaciones web. La plataforma ejecuta el ciclo completo sin intervención manual entre etapas: descubrimiento de servicios, escaneo de aplicación web con múltiples herramientas, normalización de hallazgos heterogéneos y generación de informes en cuatro formatos. Los objetivos específicos 1 a 5 quedan así alcanzados, según lo descrito en las Secciones 10 y 11. El componente evaluativo del objetivo general, tal como quedó reformulado en la Sección 4.1, consiste en evaluar mediante la descomposición interna de los tiempos del propio pipeline el

trabajo humano de consolidación que la automatización elimina entre etapas, y se completó: la Sección 12.3 lo cuantifica y la Sección 12.4 lo responde en los términos de PM-1. La comparación cronometrada contra un proceso manual equivalente, que el objetivo original incluía y que la reformulación había diferido, se ejecutó finalmente en dos corridas, una por cada autor (Sección 12.2) y se contrasta con el pipeline automatizado en la Sección 12.4. El objetivo general queda así cubierto en sus dos componentes, con una diferencia de fuerza probatoria que corresponde declarar: el componente de ingeniería se apoya en quince ejecuciones repartidas en tres campañas, mientras que el comparativo se apoya en dos corridas cronometradas, ejecutadas por los propios autores del sistema, cuyo sesgo de familiaridad caracteriza la Sección 9.9 y que la repetición entre ellos no corrige. El objetivo específico 6 queda por tanto alcanzado en cuanto a la descomposición de tiempos, al análisis de consistencia entre ejecuciones y a la medición de cobertura contra el catálogo de verdad de referencia, y alcanzado con fuerza probatoria acotada en cuanto a la comparación cronometrada contra el proceso manual.

Los resultados cuantitativos de la evaluación deben interpretarse como evidencia descriptiva preliminar, dado el tamaño muestral (Sección 14.3). Más allá de ellos, el trabajo aporta un valor de ingeniería de software concreto: un diseño desacoplado que separa la orquestación de la lógica de negocio. Ese diseño integra cinco herramientas heterogéneas bajo un patrón de adaptadores extensible, normaliza resultados dispares a una representación común y los clasifica bajo un criterio de severidad uniforme. Persiste, además, todo el ciclo de vida en un esquema relacional cuidadosamente diseñado. Este conjunto de decisiones es directamente relevante para el perfil de egreso de la Tecnicatura Universitaria en Programación.

El uso exclusivo de herramientas de código abierto elimina las barreras de licenciamiento de las soluciones comerciales, reduciendo las restricciones de acceso para instituciones educativas y organizaciones con presupuestos acotados. Es importante aclarar, no obstante, que esta accesibilidad económica no implica equiparación funcional plena con plataformas comerciales especializadas, cuyas capacidades de cobertura, escalabilidad y soporte son superiores en dimensiones no abordadas experimentalmente en este trabajo.

Finalmente, el proceso de diseño e implementación fue una experiencia formativa de alto valor. Exigió articular en un pipeline coherente los formatos de salida de cinco herramientas, el modelo relacional de PostgreSQL, la lógica de normalización y clasificación y la generación de informes en cuatro formatos. Esa articulación integró en la práctica competencias de desarrollo de software, arquitectura de sistemas, contenerización, seguridad informática, diseño de bases de datos y documentación técnica.

17. Trabajos Futuros

La versión actual del sistema es funcional, está verificada mediante pruebas automatizadas y ejecución end-to-end dentro del entorno de laboratorio (Sección 12.3), y está contrastada contra dos corridas cronometradas del proceso manual de referencia, con el alcance que declara la Sección 14.3. Sobre esa base es posible proyectar numerosas líneas de mejora y extensión.

Dos líneas se priorizan por evidencia propia y encabezan por eso esta sección. La primera, de mayor impacto potencial sobre la cobertura, es la ampliación del rastreo de OWASP ZAP. La anotación manual de la Sección 12.5 muestra que las ocho entradas del catálogo de DVWA no cubiertas exigen completar un flujo de la aplicación. Son las tres variantes de cross-site scripting, la inyección SQL ciega, la carga de archivos, la ejecución de comandos, la inclusión de archivos y los identificadores de sesión débiles. Lo que el sistema sí detecta es lo observable directamente en la respuesta del servidor. ZAP se invoca actualmente en modo de una sola pasada, cuyo rastreador clásico no ejecuta JavaScript y descubre poca superficie sobre una aplicación de navegación autenticada. Incorporar el rastreador AJAX, o una fase de rastreo autenticada de mayor profundidad previa al escaneo activo, ataca directamente la causa identificada. No se propone por completitud, como las líneas que siguen a las dos priorizadas, sino porque la medición del propio trabajo señala dónde está el límite.

La segunda línea priorizada es la única que repara la principal debilidad de la evidencia producida por este trabajo, y se liga directamente a la Pregunta de Investigación N.º 2 (Sección 5) y a la Sección 14.3: llevar el diseño cuasi-experimental original más allá de las dos corridas obtenidas en esta ronda, ambas de autores del sistema. Concretamente, repetir la cronometría con operadores externos al equipo autor, bajo un protocolo que controle el sesgo de espera activa y el de familiaridad (Sección 9.9). El componente comparativo del objetivo general pasaría así de un dato indicativo a evidencia con fuerza estadística.

Una primera línea es la integración de un motor de escaneo de vulnerabilidades de red autenticado, como OpenVAS/GVM (Greenbone Networks GmbH, 2023). A diferencia de las herramientas web actuales, un motor de este tipo, operado con credenciales válidas (SSH, Secure Shell, para Linux, WMI, Windows Management Instrumentation, para Windows), permitiría detectar vulnerabilidades internas no visibles desde el exterior: software local desactualizado, configuraciones de permisos incorrectas o secretos almacenados en archivos. Su naturaleza asíncrona (crear objetivo → crear tarea → sondear estado) y su gestión de credenciales requerirían adaptar el patrón de adaptadores actual, pero el diseño desacoplado del sistema facilita esta extensión sin alterar los componentes existentes.

Una segunda línea es la generalización de los escaneos autenticados en las propias herramientas web, hoy disponibles como modo opcional contra DVWA (Sección 14.2), y la ejecución paralela de las herramientas de aplicación, que reduciría el tiempo total del pipeline en escenarios con múltiples objetivos.

Una tercera línea consiste en cerrar el ciclo de gestión mediante dos capacidades complementarias. La primera es la notificación automática por correo electrónico de los reportes generados a los responsables de seguridad. La segunda, la integración con plataformas de gestión de incidentes (Jira, ServiceNow, Redmine), para asignar automáticamente tareas de remediación y seguir el estado de cada hallazgo hasta su verificación en ciclos posteriores.

Una cuarta línea, de menor ambición técnica pero de efecto inmediato sobre la calidad de la evidencia, es la fijación explícita de las versiones de las cinco herramientas de escaneo y del conjunto de plantillas de Nuclei. Hoy la imagen del Servicio de Escaneo las resuelve en tiempo de construcción sin fijarlas, con dos consecuencias ya observadas y documentadas en las Secciones 13 y 14.1: una construcción produjo un Nuclei sin plantillas instaladas, y el recuento de hallazgos de Nikto varió entre campañas ejecutadas con el mismo comando contra el mismo objetivo. Mientras las versiones no estén fijadas y publicadas junto con las métricas, ninguna cifra de cobertura de este trabajo es reproducible en sentido estricto, y la comparabilidad entre rondas de medición sucesivas queda comprometida. A la fijación de versiones corresponde sumar una medida de trazabilidad que este trabajo echó en falta al documentarse: que el script de medición registre en cada artefacto el identificador de revisión del repositorio en curso y el digest de las imágenes empleadas. Sin ese registro, la correspondencia entre una campaña y el estado del código debe reconstruirse desde las marcas temporales, como detalla la Sección 20. Que esto no es una precaución excesiva lo muestra Potti et al. (2025), cuyo objeto de estudio es precisamente la diferencia de rendimiento entre dos versiones consecutivas de OWASP ZAP.

Una línea menor pero de corrección inmediata es sustituir la heurística de selección de puerto por una iteración sobre todos los puertos HTTP que Nmap detecte, de modo que un objetivo con más de un servicio web quede cubierto por completo en lugar de sólo en su primer puerto (Sección 11.6).

Una línea de corrección inmediata, surgida de la propia evaluación, es la verificación de plausibilidad por herramienta. El episodio documentado en la Sección 12.6.2 —una ejecución en la que Nuclei no aportó ningún hallazgo y terminó en una fracción de su tiempo habitual, sin que el sistema lo señalara— muestra que marcar un escaneo como completado en función únicamente del código de respuesta de cada invocación es insuficiente. Contrastar la duración y el volumen de salida de cada herramienta contra los valores registrados en ejecuciones previas sobre el mismo objetivo permitiría marcar la ejecución como parcial en lugar de completa, que es la diferencia entre un resultado incompleto y un resultado incompleto que nadie advierte.

Finalmente, un dashboard de visualización histórica construido sobre Grafana conectado a PostgreSQL permitiría analizar la evolución de la postura de seguridad a lo largo del tiempo: acumulación de vulnerabilidades sin remediar, tendencias por severidad y hosts con hallazgos recurrentes. Se elige Grafana frente a alternativas como Metabase o construir vistas propias porque ya es un estándar de facto para series temporales sobre bases de datos relacionales, con soporte nativo para PostgreSQL sin capas de integración adicionales.

18. Consideraciones Éticas

El desarrollo y la evaluación de este sistema plantean cuestiones éticas de dos órdenes. Unas son comunes a todo trabajo con herramientas de seguridad ofensiva: la autorización, el doble uso y el tratamiento de la información producida. Otras son propias de este trabajo y de su diseño experimental, y se tratan al final de la sección. Como marco de referencia se adopta el Código de Ética y Conducta Profesional de la Association for Computing Machinery (ACM, 2018), cuyos principios se citan donde corresponden.

Autorización y marco legal. Todas las pruebas se ejecutaron exclusivamente en el laboratorio contenerizado descrito en la Sección 9, sobre aplicaciones deliberadamente vulnerables desplegadas por los propios autores; en ningún caso se escanearon sistemas externos. Esta restricción no es un formalismo. Escanear sistemas sin la autorización explícita, previa y documentada de sus propietarios constituye una conducta ilícita en la casi totalidad de las jurisdicciones. En la República Argentina, la Ley 26.388 de 2008 incorporó al Código Penal las figuras de acceso indebido a un sistema informático, daño informático y violación de secretos y de la privacidad. El principio 2.8 del código de ACM expresa la misma exigencia en términos profesionales: acceder a recursos de cómputo y de comunicación solo con autorización, o cuando lo exija el bien público (ACM, 2018). Esa excepción no alcanza a un trabajo académico.

La autorización como control técnico. El sistema no deja el cumplimiento de esa exigencia librado a la disciplina del operador: la materializa en el código. La lista blanca de objetivos se aplica dos veces, al registrar el objetivo en el Backend y en cada invocación del Servicio de Escaneo, que es el único componente con ruta de red hacia el laboratorio (Sección 10.2). La configuración que la define no admite valores por defecto, y si está vacía el servicio se niega a arrancar. Dirigir el sistema contra un objetivo no autorizado exige, en consecuencia, modificar deliberadamente su configuración: no puede ocurrir por descuido. Es una aplicación directa del principio 1.2 del mismo código, que obliga a evitar el daño y a mitigar el que no se haya previsto (ACM, 2018).

Doble uso. Nmap, WhatWeb, Nikto, Nuclei y OWASP ZAP sirven tanto a la seguridad defensiva como al reconocimiento previo a un ataque, y un pipeline que las encadena y consolida sus resultados amplifica ambas posibilidades. Por esa razón el sistema se distribuye con la lista blanca limitada a los dos objetivos del laboratorio. El trabajo, además, no publica hallazgos sobre ningún sistema real: todos los resultados reportados corresponden a aplicaciones creadas para ser vulnerables con fines de formación.

Tratamiento de la información producida. El inventario de servicios, tecnologías y vulnerabilidades que genera el sistema es información sensible, porque describe con precisión cómo comprometer al objetivo analizado. En el laboratorio no contiene datos personales. Sobre un objetivo real podría incluirlos, entre la evidencia que se almacena con cada hallazgo, y su tratamiento quedaría alcanzado por la Ley 25.326 de 2000, de Protección de los Datos Personales. El análisis de riesgos de la Sección 13.3 identifica como mitigaciones el cifrado en tránsito y en reposo y el control de acceso basado en roles. El sistema actual autentica mediante claves compartidas, sin registro de quién disparó cada escaneo (Sección 14.2): es suficiente para un laboratorio, pero no ofrece la trazabilidad que exigiría un uso productivo. Si el sistema se empleara sobre software de terceros, los hallazgos deberían comunicarse mediante un proceso

de divulgación coordinada como el que describe la norma ISO/IEC 29147 (International Organization for Standardization, 2018).

Integridad de la investigación. Este trabajo presenta además una particularidad propia: los autores evaluaron un sistema construido por ellos mismos, y cada uno de ellos ejecutó una de las dos corridas de la condición de control. Esa doble posición configura un conflicto de intereses, porque quienes se benefician de un resultado favorable controlan también su medición. No pudo eliminarse con los recursos disponibles, y la respuesta adoptada fue hacerlo visible. El sesgo de familiaridad se declara como amenaza a la validez (Sección 9.9), y las corridas manuales se reportan como dos observaciones de operadores no independientes del sistema, sin fuerza estadística. Los datos del sistema automatizado provienen de marcas temporales registradas por la base de datos, no de la apreciación de los autores. Los resultados contrarios a las hipótesis propias, como la refutación de H2 en las réplicas, se reportan con el mismo detalle que los favorables (Sección 12.6.2). Por la misma razón, la nota de la Tabla 10 deja constancia de una inconsistencia detectada en la hoja de cronometraje en lugar de silenciarla: la etapa afectada se volvió a cronometrar en dos sesiones posteriores, una por cada columna de tiempo, y las celdas resultantes se marcan en la propia tabla como recronometradas, con su fecha, en lugar de presentarse como si procedieran de la sesión original. Se declara asimismo que el segundo operador conocía el resultado de la primera corrida antes de ejecutar la suya, de modo que la coincidencia entre ambas documenta la reproducibilidad del procedimiento pero no constituye una verificación independiente.

Formación. En el contexto académico, la formación ética es tan importante como la técnica. El laboratorio aislado ofrece un entorno realista para adquirir estas competencias sin riesgo para terceros, y su valor pedagógico está documentado por Księżopolski et al. (2022) y Su (2024). Ese aislamiento, sin embargo, solo protege mientras dura el ejercicio: fuera del laboratorio, las competencias adquiridas solo pueden aplicarse con la autorización explícita y documentada del propietario de los sistemas analizados.

19. Referencias Bibliográficas

Las referencias siguen el formato APA (American Psychological Association), 7.^a edición.

- Amankwah, R., Chen, J., Kudjo, P. K., & Towey, D. (2020). An empirical comparison of commercial and open-source web vulnerability scanners. *Software: Practice and Experience*, 50(9), 1842–1857. <https://doi.org/10.1002/spe.2870>
- Association for Computing Machinery. (2018). *ACM code of ethics and professional conduct*. <https://www.acm.org/code-of-ethics>
- Barrionuevo, D., Zea Cárdenas, M., & Parisi, G. (2015). *Sistema integrado de soporte para análisis de vulnerabilidades en sistemas web*. Universidad Tecnológica Nacional. <https://hdl.handle.net/20.500.12272/12911>
- Bridges, R. A., Rice, A. E., Oesch, S., Nichols, J. A., Watson, C., Spakes, K., Norem, S., Huettel, M., Jewell, B., Weber, B., Gannon, C., Bizovi, O., Hollifield, S. C., & Erwin, S. (2023). Testing SOAR tools in use. *Computers & Security*, 129, Article 103201. <https://doi.org/10.1016/j.cose.2023.103201>
- Cespedes Ortega, R. G., Fernandez Quatrini, R. A., Flores, S. A., & Giralda, R. (2023). *Junkode: Documentador automático de código fuente* [Trabajo final de grado, Universidad Tecnológica Nacional, Facultad Regional Mendoza]. RIA. <http://hdl.handle.net/20.500.12272/7762>
- Cifre, S. A. (2020). *Modelo de seguridad para la gestión de vulnerabilidades de servidores en nubes privadas* [Tesis de maestría, Universidad Tecnológica Nacional]. RIA. <http://hdl.handle.net/20.500.12272/6050>
- Ciperiani, M. J. (2025). *Lriot: Laboratorio remoto IoT* [Trabajo final de grado, Universidad Tecnológica Nacional, Facultad Regional Mendoza]. RIA. <https://hdl.handle.net/20.500.12272/12768>
- Cuevas, J. C., Muñoz, R., & Di Gionantonio, M. A. (2018). *Análisis de vulnerabilidades de sistemas web en desarrollo y en producción*. Universidad Tecnológica Nacional. <https://hdl.handle.net/20.500.12272/12938>
- Cuevas, J. C., Muñoz, R., & Gibellini, F. (2016). *Revisión del estado del arte y de las técnicas y herramientas orientados al desarrollo de un sistema de soporte para el análisis de vulnerabilidades*. Universidad Tecnológica Nacional. <https://hdl.handle.net/20.500.12272/12893>
- Docker Inc. (2023). *Docker documentation*. Recuperado el 25 de septiembre de 2026, de <https://docs.docker.com/>
- Engelbreton, P. (2013). *The basics of hacking and penetration testing: Ethical hacking and penetration testing made easy* (2nd ed.). Syngress.
- Farmer, D., & Venema, W. (1993, 2 de diciembre). Improving the security of your site by breaking into it [Publicación en comp.security.unix]. Sun Microsystems; Eindhoven University of Technology.
- FIRST.Org, Inc. (2019). *Common vulnerability scoring system version 3.1: Specification document*. <https://www.first.org/cvss/specification-document>

- Giannone, A. O. (2019). *Método de inclusión de hacking ético en el proceso de testing de software* [Tesis de maestría, Universidad Tecnológica Nacional, Facultad Regional Buenos Aires]. RIA. <http://hdl.handle.net/20.500.12272/4068>
- Greenbone Networks GmbH. (2023). *Greenbone enterprise documentation*. Recuperado el 25 de septiembre de 2026, de <https://docs.greenbone.net/>
- Greenbone Networks GmbH. (2024). Greenbone, GVM, OpenVAS and how they are connected. Greenbone Community Documentation. Recuperado el 25 de septiembre de 2026, de <https://greenbone.github.io/docs/latest/background.html>
- International Organization for Standardization. (2018). *Information technology — Security techniques — Vulnerability disclosure* (ISO/IEC 29147:2018).
- International Organization for Standardization. (2022a). *Information security, cybersecurity and privacy protection — Information security management systems — Requirements* (ISO/IEC 27001:2022).
- International Organization for Standardization. (2022b). *Information technology — Security techniques — Information security risk management* (ISO/IEC 27005:2022).
- ISC2. (2024). 2024 ISC2 cybersecurity workforce study. <https://www.isc2.org/Insights/2024/10/ISC2-2024-Cybersecurity-Workforce-Study>
- Jacobs, J., Romanosky, S., Edwards, B., Roytman, M., & Adjerid, I. (2021). Exploit prediction scoring system (EPSS). *Digital Threats: Research and Practice*, 2(3), Article 20. <https://doi.org/10.1145/3436242>
- Kim, D., & Solomon, M. G. (2018). *Fundamentals of information systems security* (3rd ed.). Jones & Bartlett Learning.
- Księżopolski, B., Mazur, K., Miśkiewicz, M., & Rusinek, D. (2022). Teaching a hands-on CTF-based web application security course. *Electronics*, 11(21), 3517. <https://doi.org/10.3390/electronics11213517>
- Ley 25.326 de 2000. *Protección de los datos personales*. 4 de octubre de 2000. Boletín Oficial N.º 29.517. <https://servicios.infoleg.gob.ar/infolegInternet/verNorma.do?id=64790>
- Ley 26.388 de 2008. *Modificación del Código Penal*. 4 de junio de 2008. Boletín Oficial N.º 31.433. <https://servicios.infoleg.gob.ar/infolegInternet/verNorma.do?id=141790>
- Liu, B., Zhao, Y., Xu, G., & Wang, H. (2025). *LLM agents for automated web vulnerability reproduction: Are we there yet?* (arXiv:2510.14700). arXiv. <https://doi.org/10.48550/arXiv.2510.14700>
- Lyon, G. F. (2009). *Nmap network scanning: The official Nmap project guide to network discovery and security scanning*. Insecure.Com LLC.
- Marandi, M., Bertia, A., & Silas, S. (2023). Implementing and automating security scanning to a DevSecOps CI/CD pipeline. En 2023 World Conference on Communication & Computing (WCONF). IEEE. <https://doi.org/10.1109/WCONF58270.2023.10235015>
- Mills, A., White, J., & Legg, P. (2023). Longitudinal risk-based security assessment of docker software container images. *Computers & Security*, 135, Article 103478. <https://doi.org/10.1016/j.cose.2023.103478>
- MITRE Corporation. (2024). *Common vulnerabilities and exposures (CVE)*. Recuperado el 25 de septiembre de 2026, de <https://www.cve.org/>

- Mohammed, K. I., Shanmugam, B., & El-Den, J. (2025). Evolution of DevSecOps and its influence on application security: A systematic literature review. *Technologies*, 13(12), 548. <https://doi.org/10.3390/technologies13120548>
- n8n GmbH. (2024). *n8n documentation*. Recuperado el 25 de septiembre de 2026, de <https://docs.n8n.io/>
- Nasab, A. R., Shahin, M., Hoseyni Raviz, S. A., Liang, P., Mashmool, A., & Lenarduzzi, V. (2023). An empirical study of security practices for microservices systems. *Journal of Systems and Software*, 198, Article 111563. <https://doi.org/10.1016/j.jss.2022.111563>
- National Institute of Standards and Technology (NIST). (2023a). *National vulnerability database (NVD)*. Recuperado el 25 de septiembre de 2026, de <https://nvd.nist.gov/>
- National Institute of Standards and Technology (NIST). (2023b). *Glossary*. Computer Security Resource Center. Recuperado el 25 de septiembre de 2026, de <https://csrc.nist.gov/glossary>
- OWASP Foundation. (2021). *OWASP Top Ten 2021*. <https://owasp.org/Top10/2021/>
- OWASP Foundation. (2024). *OWASP Zed Attack Proxy (ZAP) documentation*. Recuperado el 25 de septiembre de 2026, de <https://www.zaproxy.org/docs/>
- OWASP Foundation. (2025). *OWASP Top 10:2025*. <https://top10.owasp.org/2025/>
- Potti, U.-S., Huang, H.-S., Chen, H.-T., & Sun, H.-M. (2025). *Security testing framework for web applications: Benchmarking ZAP V2.12.0 and V2.13.0 by OWASP as an example* (arXiv:2501.05907). arXiv. <https://doi.org/10.48550/arXiv.2501.05907>
- ProjectDiscovery. (2024). *Nuclei documentation*. Recuperado el 25 de septiembre de 2026, de <https://docs.projectdiscovery.io/opensource/nuclei/overview>
- Qadir, S., Waheed, E., Khanum, A., & Jehan, S. (2025). Comparative evaluation of approaches & tools for effective security testing of web applications. *PeerJ Computer Science*, 11, e2821. <https://doi.org/10.7717/peerj-cs.2821>
- Rotella, J. (2016). *Cálculo del impacto económico de soluciones de gestión de identidades y accesos en las organizaciones* [Tesis de maestría, Universidad Tecnológica Nacional, Facultad Regional Mendoza]. RIA. <http://hdl.handle.net/20.500.12272/10459>
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- Su, J.-M. (2024). WebHOLE: Developing a web-based hands-on learning environment to assist beginners in learning web application security. *Education and Information Technologies*, 29(6). <https://doi.org/10.1007/s10639-023-12090-z>
- Sullo, C., & Lodge, D. (2023). *Nikto* (versiones 2.6.0 y 2.6.1) [Software]. GitHub. Recuperado el 25 de septiembre de 2026, de <https://github.com/sullo/nikto>
- Tenable, Inc. (2026). *Nessus Professional* [Página de producto]. Recuperado el 25 de septiembre de 2026, de <https://www.tenable.com/products/nessus/nessus-professional>

- Verizon. (2026). *2026 Data breach investigations report*. Recuperado el 25 de septiembre de 2026, de <https://www.verizon.com/business/resources/reports/dbir/>
- Wieringa, R. J. (2014). *Design science methodology for information systems and software engineering*. Springer. <https://doi.org/10.1007/978-3-662-43839-8>
- Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer. <https://doi.org/10.1007/978-3-642-29044-2>

20. Anexos Técnicos

El repositorio completo del proyecto (código fuente de los cinco servicios, configuración de despliegue, migraciones de base de datos y el workflow de n8n) está disponible públicamente en https://github.com/lautaroCast/Ciberseguridad_Prog2026. Los siguientes anexos referencian archivos reales de ese repositorio; el commit 0357df2 de la rama main, con fecha 26 de agosto de 2026, corresponde al estado del código descrito en este trabajo. Puede verificarse clonando el repositorio y ejecutando `git show 0357df2`, que devuelve esa misma fecha y el árbol de archivos al que remiten los anexos. Ese estado está marcado además con la etiqueta tesis-entrega del repositorio. La rama main continuó recibiendo correcciones y pruebas después de esa fecha, ninguna de las cuales se describe ni se mide en este trabajo: quien clone la rama en su estado actual encontrará un código posterior al descrito, y debe situarse en la etiqueta para reproducir lo que los anexos documentan. Corresponde precisar qué relación guarda ese commit con las mediciones del Capítulo 12, porque no es la que podría suponerse. El commit 0357df2 describe el estado del código al cierre del período de medición y no el estado sobre el cual se ejecutó cada campaña: el script de medición no registra el identificador de revisión en curso, de modo que la correspondencia debe reconstruirse a partir de la marca de captura de entorno de cada artefacto y del historial de la rama principal. Reconstruida de ese modo, es la siguiente. La campaña A se capturó el 2026-08-11 a las 00:18 UTC, cuando la rama principal estaba en el commit aadd368, del 6 de agosto, ochenta y nueve commits antes del declarado. La campaña B se capturó el 2026-08-24 a las 16:59 UTC, sobre b0a1bd3, del 23 de agosto. La campaña C se capturó ese mismo día a las 19:21 UTC, sobre ecb6d42, siete commits antes del declarado. Las campañas B y C y las dos corridas del brazo manual emplearon además la misma imagen del Servicio de Escaneo, construida el 2026-08-21 y no reconstruida hasta el 23 de septiembre. Entre la captura de la campaña A y el commit declarado median seis cambios en los normalizadores de hallazgos, de modo que las distribuciones de severidad de las Tablas 14 y 20 corresponden al estado vigente en cada captura y no al del commit declarado. Ninguna de estas precisiones altera una cifra del Capítulo 12, que se reproduce desde los artefactos publicados. Lo que altera es el alcance de la afirmación: el código que este trabajo describe y el que produjo cada medición son estados próximos pero no idénticos, y por eso la Sección 17 propone registrar la revisión en curso dentro de cada artefacto, para que una medición futura no dependa de esta reconstrucción. Los datos crudos de las mediciones se publican bajo una etiqueta propia porque se incorporaron al repositorio después del commit del código, no porque sean posteriores a él: los artefactos de las tres campañas se capturaron en agosto de 2026 — cada JSON registra su propia marca de captura— y se publicaron el 24 de septiembre; las transcripciones de las dos hojas de cronometraje se volcaron ese mismo día, y la de la corrida A se actualizó el 25 de septiembre al incorporar el recronometraje del tiempo de reloj de su etapa de Nmap. La etiqueta tesis-datos marca el estado en que el repositorio contiene los artefactos de las tres campañas (`scripts/measurement_campaign_results/`, con un README que asocia cada archivo a la campaña que el Capítulo 12 nombra) y la transcripción de las dos hojas de cronometraje del brazo manual (`docs/manual-run-sheets/`). Ese mismo directorio conserva tres campañas que este trabajo no integra a sus cifras y que la Sección 12.6.4 declara y analiza. Los hallazgos crudos de las tres corridas sobre las que se calculan las métricas de cobertura del Capítulo 12 —el escaneo f9760239 contra DVWA, el 0a605f82 contra DVWA autenticado y el c2421b99 contra Juice Shop— se publican por separado en `scripts/ground_truth/reported-in-report`, junto con un archivo que explica en qué difieren de los valores que produce la versión actual del comparador y por qué, según detalla la nota de la Tabla 19. El repositorio conserva asimismo, bajo `docs/`, los informes de las evaluaciones externas recibidas durante el desarrollo y las correcciones aplicadas en respuesta a cada una, identificadas por el código con que cada auditoría las enumeró. Se mantienen deliberadamente como traza del proceso de corrección del trabajo, no como parte de su contenido.

Anexo A — Esquema Completo de Base de Datos

El esquema real se define mediante modelos de SQLAlchemy 2.0 en database/models/ y se versiona con Alembic. La siguiente es una representación en SQL de las tablas principales, con claves primarias UUID y tipos enumerados nativos de PostgreSQL para los vocabularios cerrados, verificada contra los modelos reales del repositorio.

```
-- Tipos enumerados (vocabularios cerrados)
CREATE TYPE scan_status AS ENUM ('pending','running','completed','failed','cancelled');
CREATE TYPE scan_task_status AS ENUM ('pending','running','completed','failed','skipped');
CREATE TYPE severity_level AS ENUM ('info','low','medium','high','critical');
CREATE TYPE report_format AS ENUM ('pdf','html','markdown','json');
CREATE TYPE user_role AS ENUM ('admin','analyst');

-- Objetivos registrados para análisis
CREATE TABLE targets (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR NOT NULL UNIQUE,
  host VARCHAR NOT NULL,
  description TEXT,
  is_lab_target BOOLEAN NOT NULL DEFAULT TRUE,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at TIMESTAMPTZ
);

-- Ejecuciones del pipeline sobre un objetivo
CREATE TABLE scans (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  target_id UUID NOT NULL REFERENCES targets(id) ON DELETE CASCADE,
  status scan_status NOT NULL DEFAULT 'pending',
  pipeline_run_id VARCHAR,
  triggered_by VARCHAR,
  started_at TIMESTAMPTZ,
  finished_at TIMESTAMPTZ,
  error_message TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Invocación de una herramienta concreta dentro de un escaneo
CREATE TABLE scan_tasks (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  scan_id UUID NOT NULL REFERENCES scans(id) ON DELETE CASCADE,
  tool_name VARCHAR NOT NULL, -- texto libre: nmap, whatweb, nikto, nuclei, zap...
  status scan_task_status NOT NULL DEFAULT 'pending',
  command TEXT,
  raw_output TEXT,
  raw_output_path VARCHAR,
  started_at TIMESTAMPTZ,
  finished_at TIMESTAMPTZ,
  error_message TEXT,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

```

-- Servicios de red descubiertos (Nmap)
CREATE TABLE services (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  scan_id     UUID NOT NULL REFERENCES scans(id) ON DELETE CASCADE,
  host        VARCHAR NOT NULL,
  port        INTEGER NOT NULL,
  protocol    VARCHAR,
  service_name VARCHAR,
  product     VARCHAR,
  version     VARCHAR,
  banner      TEXT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Tecnologías identificadas (WhatWeb)
CREATE TABLE technologies (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  scan_id     UUID NOT NULL REFERENCES scans(id) ON DELETE CASCADE,
  name        VARCHAR NOT NULL,
  version     VARCHAR,
  category    VARCHAR,
  detected_by VARCHAR,
  confidence  VARCHAR,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Hallazgos normalizados
CREATE TABLE findings (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  scan_id     UUID NOT NULL REFERENCES scans(id) ON DELETE CASCADE,
  scan_task_id UUID NOT NULL REFERENCES scan_tasks(id) ON DELETE CASCADE,
  service_id  UUID REFERENCES services(id),      -- nullable
  title       VARCHAR NOT NULL,
  description  TEXT,
  finding_type VARCHAR NOT NULL,                -- texto libre
  evidence     TEXT,
  confidence   VARCHAR,
  cvss_score   NUMERIC(3,1),
  cvss_vector  VARCHAR,
  severity     severity_level NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Referencias CVE de un hallazgo (relación 0..N)
CREATE TABLE cve_references (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  finding_id  UUID NOT NULL REFERENCES findings(id) ON DELETE CASCADE,
  cve_id      VARCHAR NOT NULL,
  cvss_score  NUMERIC(3,1),
  cvss_vector VARCHAR,
  description  TEXT,

```

```

source_url VARCHAR,
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Metadatos de reportes generados
CREATE TABLE reports (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  scan_id     UUID NOT NULL REFERENCES scans(id) ON DELETE CASCADE,
  format      report_format NOT NULL,
  file_path   VARCHAR,
  generated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  generated_by VARCHAR
);

-- Placeholder para autenticación multiusuario futura (no consumido aún)
CREATE TABLE users (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email       VARCHAR NOT NULL UNIQUE,
  hashed_password VARCHAR NOT NULL,
  full_name    VARCHAR,
  role        user_role NOT NULL DEFAULT 'analyst',
  is_active   BOOLEAN NOT NULL DEFAULT TRUE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

Anexo B — Invocaciones Reales de las Herramientas de Escaneo

Extraídas de los adaptadores del Servicio de Escaneo (scanner/app/adapters/). Cada línea es la argumentación efectiva que construye el método build_command() del adaptador correspondiente.

```

# Nmap — descubrimiento de servicios y versiones, XML por stdout
nmap -sV -T4 -oX - <target> [-p <ports>]

# WhatWeb — fingerprinting de tecnologías, JSON por stdout
# -q --color=never son obligatorios: sin ellos WhatWeb corrompe el JSON
whatweb --log-json=- -q --color=never -a <aggression> <scheme>://<target>:<port>

# Nikto — configuraciones inseguras, JSON escrito a archivo (no stdout)
perl /opt/nikto/program/nikto.pl -h <scheme>://<target>:<port> \
  -Format json -output <archivo> -nointeractive -maxtime <max_time>

# Nuclei — escaneo por plantillas, JSON Lines por stdout
# plantillas embebidas en la imagen; -duc evita la comprobación de updates
nuclei -u <scheme>://<target>:<port> -jsonl -silent -duc \
  [-severity <sev>] [-tags <tags>]

# OWASP ZAP — spider + escaneo activo, JSON escrito a archivo
zap.sh -cmd -quickurl <scheme>://<target>:<port> -quickout <archivo>.json

```

Anexo C — Estructura del Informe en Markdown (plantilla Jinja2)

Esta es la plantilla propia del formato Markdown, distinta de la que comparten HTML y PDF (ver Sección 11.5). El Servicio de Reportes la renderiza ordenando los hallazgos por severidad. Estructura de referencia:

Informe de Seguridad — Análisis de Vulnerabilidades

****Objetivo:**** {{ target.name }} ({{ target.host }})
****Escaneo:**** {{ scan.id }} — Estado: {{ scan.status }}
****Fecha de generación:**** {{ generated_at }}
****Total de hallazgos:**** {{ findings|length }}
(Críticos: {{ counts.critical }} | Altos: {{ counts.high }} |
Medios: {{ counts.medium }} | Bajos: {{ counts.low }} | Info: {{ counts.info }})

Distribución por Severidad

Severidad	Cantidad
Crítica	{{ counts.critical }}
Alta	{{ counts.high }}
Media	{{ counts.medium }}
Baja	{{ counts.low }}
Informativa	{{ counts.info }}

Detalle de Hallazgos

{% for f in findings %}
[{{ f.severity }}] {{ f.title }}{% if f.cvss_score %} — CVSS: {{ f.cvss_score }}{% endif %}

- ****Tipo:**** {{ f.finding_type }} | ****Herramienta:**** (tarea {{ f.scan_task_id }})
{% if f.cve_references %}- ****CVE:**** {{ f.cve_references | join(', ', attribute='cve_id') }}{% endif %}
- ****Descripción:**** {{ f.description }}
- ****Evidencia:**** {{ f.evidence }}
{% endfor %}

Informe generado automáticamente por VulnScan Platform.

Anexo D — Secuencia de Nodos del Workflow en n8n

Todos los nodos, salvo los triggers y los pasos de resolución/selección, son llamadas HTTP a los servicios propios. La lógica de ejecución, normalización y clasificación reside en servicios propios versionados; n8n conserva dos decisiones propias no delegadas a ningún servicio: la selección de puerto HTTP (nodo 7) y la resolución de contexto entre triggers (nodo 2). Los nombres entre corchetes de la secuencia numerada a continuación son los nombres reales de los nodos tal como figuran en el workflow exportado (n8n/workflows/vulnscan-pipeline.json del repositorio); la numeración simplifica a 13 pasos lógicos varios nodos auxiliares del JSON real (edición de campos, descarga y envío del reporte) que no alteran la secuencia conceptual descrita.

[1] Webhook Trigger / Form Trigger → Puntos de entrada (producción / demo)

- [2] Resolve Pipeline Context → Unifica ambas ramas; resuelve scan_id/target_id/host
- [3] Scan: Nmap → HTTP POST /scan/nmap (Scanner Service)
- [4] Ingest: Nmap → HTTP POST /scans/{id}/tasks (Backend → tabla services)
- [5] Scan: WhatWeb → HTTP POST /scan/whatweb (Scanner Service)
- [6] Ingest: WhatWeb → HTTP POST /scans/{id}/tasks (Backend → tabla technologies)
- [7] Select HTTP Port → Elige el puerto HTTP a partir de la salida de Nmap
- [8] Scan+Ingest: Nikto → /scan/nikto + /scans/{id}/tasks
- [9] Scan+Ingest: Nuclei → /scan/nuclei + /scans/{id}/tasks (findings + cve_references)
- [10] Scan+Ingest: ZAP → /scan/zap + /scans/{id}/tasks
- [11] Complete Scan → HTTP POST /scans/{id}/complete (Backend)
- [12] Generate Report → HTTP POST /scans/{id}/reports?format=pdf (continueOnFail)
- [13] Pipeline Complete → Consolida el resultado de la ejecución

Anexo E — Metodología de Verdad de Referencia (Ground Truth)

Construida a partir de las fuentes oficiales de cada aplicación de laboratorio, no estimada ni inferida: juice_shop_catalog.json (52 entradas, filtradas desde el challenges.yml real extraído del contenedor de Juice Shop en ejecución, a las categorías con firma observable por red) y dvwa_catalog.json (11 entradas, desde la estructura real de directorios de módulos vulnerables del contenedor de DVWA). El matcher (scripts/ground_truth/match_findings.py) compara los hallazgos de un scan real contra el catálogo en tres niveles de confianza que nunca se mezclan en un solo número: (1) coincidencia de CVE exacto, (2) coincidencia de tipo de hallazgo más solapamiento de ubicación, (3) coincidencia débil de palabra clave; reporta recall, precisión, F1 y tasa de falsos positivos por herramienta. Resultados y su interpretación se presentan en la Sección 9.3(c) y la Sección 12.5; el detalle completo de la metodología, incluidos los criterios de exclusión de desafíos sin señal detectable por red, se documenta en scripts/ground_truth/README.md del repositorio.

Para medir el efecto de la autenticación sobre la cobertura, scripts/ground_truth/_run_authenticated_dvwa_scan.py repite esta medición contra DVWA en modo autenticado (options.authenticated=true del Scanner Service — ver Sección 9.3(c) y scanner/app/services/dvwa_auth.py): hace login con las credenciales por defecto de DVWA, fija el nivel de seguridad en “low”, dispara Nikto/Nuclei/ZAP con la cookie de sesión inyectada (-Add-header, -H, y el add-on Replacer de ZAP respectivamente), e ingiere los resultados vía la misma API del Backend que usa n8n. Los artefactos de esa corrida (sample_run_dvwa_authenticated_findings.json, _tasks.json, _match_report.json) quedan junto a los de la campaña original en scripts/ground_truth/. Resultados en la Sección 12.5 (Tabla 17). Este modo es opt-in: no forma parte del workflow de n8n por defecto, que sigue escaneando sin autenticar.

Anexo F — docker-compose.yml Completo

Definición real de todos los servicios, redes y volúmenes de la plataforma, tal como existe en el repositorio del proyecto (raíz del repo). Sostiene directamente las afirmaciones de las Secciones 10 y 11 sobre segmentación de red, endurecimiento de contenedores y healthchecks.

```
# =====
# VulnScan Platform — Docker Compose orchestration
#
# Single entry point for the whole system: `docker compose up -d` must leave
# every service healthy with no manual follow-up steps.
#
# Two networks by design:
# - app-network: platform services talk to each other (frontend, backend,
```

```

# scanner, reports, n8n, db).
# - lab-network: segmented network for vulnerable lab targets. Only the
# scanner service bridges into both networks, so the lab is never
# reachable from the frontend/n8n/backend/db — they simply have no
# route to lab-network at all. NOTE: this is deliberately a plain
# bridge network, not `internal: true`. Docker silently skips host
# port publishing for containers whose only network is internal
# (verified empirically while building this module — `docker port`
# returns nothing even though `ports:` is configured), which would
# make Juice Shop/DVWA unreachable from the host browser. The actual
# safety guarantee — the platform never scans anything outside this
# lab — is enforced at the application layer: the Backend API
# (Módulo 3) only accepts targets whitelisted via `targets.is_lab_target`.
# See docs/lab.md.
#
# This file is a scaffold (Módulo 0): networks, volumes and service stubs
# are defined now; each service's build context is filled in as its module
# is completed. Services are commented out until their Dockerfile exists so
# `docker compose up -d` stays runnable at every stage of development.
# =====

networks:
  app-network:
    driver: bridge
  lab-network:
    driver: bridge

volumes:
  postgres-data:
  n8n-data:
  reports-data:

services:
  # -----
  # Database — PostgreSQL (Módulo 1)
  # -----
  db:
    image: postgres:16-alpine
    container_name: ${COMPOSE_PROJECT_NAME:-vulnscan}-db
    restart: unless-stopped
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres-data:/var/lib/postgresql/data
    ports:
      - "${POSTGRES_PORT:-5432}:5432"
    networks:
      - app-network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s

  # -----
  # Migrate — one-shot Alembic runner, applies the schema then exits

```

```

# (Módulo 1). Any service depending on the schema should declare
# `depends_on: migrate: condition: service_completed_successfully`.
# -----
migrate:
  build:
    context: ./database
  container_name: ${COMPOSE_PROJECT_NAME:-vulnscan}-migrate
  environment:
    DATABASE_URL: ${DATABASE_URL}
  networks:
    - app-network
  depends_on:
    db:
      condition: service_healthy
  restart: "no"

# -----
# Backend API — FastAPI orchestrator (Módulo 3)
#
# Build context is the repo root, not ./backend: the image needs both
# backend/app and database/models (see backend/Dockerfile) since the
# Backend reuses the same ORM models Alembic migrates, rather than
# redefining the schema.
# -----
backend:
  build:
    context: .
    dockerfile: backend/Dockerfile
  container_name: ${COMPOSE_PROJECT_NAME:-vulnscan}-backend
  restart: unless-stopped
  environment:
    DATABASE_URL: ${DATABASE_URL}
    BACKEND_LOG_LEVEL: ${BACKEND_LOG_LEVEL:-info}
    BACKEND_CORS_ORIGINS: ${BACKEND_CORS_ORIGINS}
    # No :-default here on purpose: an empty/unset whitelist must fail
    # loudly at startup (see app/config.py's validator) rather than boot
    # successfully and silently reject every target registration.
    BACKEND_ALLOWED_LAB_HOSTS: ${BACKEND_ALLOWED_LAB_HOSTS}
    # Same reasoning as above: no :-default, see app/config.py's validator.
    N8N_WEBHOOK_BASE_URL: ${N8N_WEBHOOK_BASE_URL}
    REPORTS_BASE_URL: ${REPORTS_BASE_URL}
    # No :-default on either: an unset key would boot successfully and
    # then reject every request with no clear cause (see app/config.py's
    # validators).
    BACKEND_API_KEY: ${BACKEND_API_KEY}
    INTERNAL_API_KEY: ${INTERNAL_API_KEY}
  ports:
    - "${BACKEND_PORT:-8000}:8000"
  networks:
    - app-network
  depends_on:
    db:
      condition: service_healthy
    migrate:
      condition: service_completed_successfully
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
    interval: 10s
    timeout: 5s

```

retries: 5
start_period: 15s

```
# -----
# Scanner Service — Nmap/Nuclei/Nikto/WhatWeb/ZAP adapters (Módulo 4)
#
# The only service that bridges app-network and lab-network: n8n/backend
# reach it over app-network, and it's the one thing allowed to reach the
# lab targets. No DB coupling of its own (not "stateless" in the strict
# sense — see docs/security.md for why that wording was wrong).
#
# No `ports:`: reachable only from app-network/lab-network,
# authenticated via X-Internal-Token (INTERNAL_API_KEY below). It also
# independently validates `target` against SCANNER_ALLOWED_LAB_HOSTS on
# every /scan/{tool} call — the same whitelist the Backend checks at
# target-registration time, sourced from the same .env value below, but
# enforced here too because this is the only component with an actual
# network route to lab-network. A check only in the Backend would not be
# real defense in depth (see docs/security.md).
# -----
scanner:
  build:
    context: ./scanner
  container_name: ${COMPOSE_PROJECT_NAME:-vulnscan}-scanner
  restart: unless-stopped
  environment:
    SCANNER_MAX_TIMEOUT_SECONDS: ${SCANNER_MAX_TIMEOUT_SECONDS:-900}
    INTERNAL_API_KEY: ${INTERNAL_API_KEY}
    SCANNER_ALLOWED_LAB_HOSTS: ${BACKEND_ALLOWED_LAB_HOSTS}
  cap_drop:
    - ALL
  # NET_RAW/NET_ADMIN preserve Nmap's SYN-scan behavior under the
  # non-root `appuser` (see scanner/Dockerfile) — without them nmap
  # still works, it just silently falls back to a slower TCP connect
  # scan. No read_only here: ZAP writes session/config data under $HOME
  # at scan time and that hasn't been verified path-by-path yet.
  cap_add:
    - NET_RAW
    - NET_ADMIN
  security_opt:
    - no-new-privileges:true
  networks:
    - app-network
    - lab-network
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8100/health"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 15s

# -----
# Reports Service — Jinja2 + WeasyPrint (Módulo 7)
#
# No DB coupling and no knowledge of any other service (not "stateless"
# in the strict sense — it persists rendered files across requests on
# its own volume, see docs/security.md). The Backend gathers
# target/scan/findings data and
# pushes it here in one request; this service only ever renders and
```

```

# serves files on its own private `reports-data` volume (not shared with
# the Backend — downloads are proxied over HTTP, see
# backend/app/routers/reports.py).
#
# No `ports:`: nothing outside app-network needs to reach
# this directly — only the Backend calls it, authenticated via
# X-Internal-Token (INTERNAL_API_KEY below).
# -----
reports:
  build:
    context: ./reports
  container_name: ${COMPOSE_PROJECT_NAME:-vulnscan}-reports
  restart: unless-stopped
  environment:
    REPORTS_OUTPUT_DIR: ${REPORTS_OUTPUT_DIR:-/data/reports}
    INTERNAL_API_KEY: ${INTERNAL_API_KEY}
  volumes:
    - reports-data:/data/reports
  cap_drop:
    - ALL
  security_opt:
    - no-new-privileges:true
  read_only: true
  tmpfs:
    - /tmp
    - /home/appuser/.cache
    # fontconfig's own cachedir (/etc/fonts/fonts.conf) — neither /tmp nor
    # ~/.cache cover it, so without this it silently rebuilds the font
    # index on every PDF render instead of caching it. uid/gid must match
    # appuser (1000) — tmpfs mounts default to root:root, unwritable by
    # the non-root user this container runs as.
    - /var/cache/fontconfig:uid=1000,gid=1000
  networks:
    - app-network
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8200/health"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 15s

# -----
# n8n — pipeline orchestration (Módulo 6)
#
# Orchestrator: execution, normalization and severity classification
# live in the Backend/Scanner, not here — but two Code nodes do make
# real decisions of their own (which port to attack, resolving the two
# trigger shapes into one context); see n8n/README.md, not "no business
# logic at all." Two entry points feed the same pipeline:
# the Webhook trigger (called by the Backend's
# POST /targets/{id}/pipeline, the production path) and a Form trigger
# (opened directly in n8n's UI, for running/demoing the pipeline without
# the Backend or a frontend — see n8n/README.md).
# -----
n8n:
  image: n8nio/n8n:latest
  container_name: ${COMPOSE_PROJECT_NAME:-vulnscan}-n8n
  restart: unless-stopped
  environment:

```

```

N8N_HOST: ${N8N_HOST}
# n8n's own process always listens on 5678 inside the container —
# this is unrelated to which host port it's published on below.
N8N_PORT: 5678
N8N_PROTOCOL: ${N8N_PROTOCOL}
N8N_BASIC_AUTH_ACTIVE: ${N8N_BASIC_AUTH_ACTIVE}
N8N_BASIC_AUTH_USER: ${N8N_BASIC_AUTH_USER}
N8N_BASIC_AUTH_PASSWORD: ${N8N_BASIC_AUTH_PASSWORD}
N8N_ENCRYPTION_KEY: ${N8N_ENCRYPTION_KEY}
# n8n self-generates URLs (Form/Webhook "Production URL", etc.) from
# N8N_HOST/N8N_PORT/N8N_PROTOCOL above, which describe the internal
# listen address, not the host-facing one. If N8N_PORT (below, the
# *host* port) ever has to be remapped — e.g. 5678 already taken by
# an unrelated container — those self-generated URLs go stale unless
# told explicitly what the real external address is.
N8N_EDITOR_BASE_URL: ${N8N_PROTOCOL:-http}://${N8N_HOST:-localhost}:${N8N_PORT:-5678}/
# Basic auth protects the editor UI/API only — it does not gate
# webhook URLs, so the Backend's service-to-service call to the
# pipeline trigger needs no credentials.
# Read by the workflow's HTTP Request nodes via `$.env.*` expressions
# — every node calling the Backend/Scanner sends one of
# these as X-API-Key/X-Internal-Token. See
# n8n/workflows/vulnscan-pipeline.json.
BACKEND_API_KEY: ${BACKEND_API_KEY}
INTERNAL_API_KEY: ${INTERNAL_API_KEY}
# n8n >=2.0 blocks $.env access in node expressions by default (a
# breaking change) — without this, every node above silently never
# resolves its header value and the pipeline stalls before ever
# reaching the Scanner Service. Confirmed empirically while building
# this module: the workflow hung indefinitely with zero calls ever
# reaching scanner/backend until this was set.
N8N_BLOCK_ENV_ACCESS_IN_NODE: "false"
volumes:
  - n8n-data:/home/node/.n8n
  - ./n8n/workflows:/workflows:ro
ports:
  - "${N8N_PORT:-5678}:5678"
networks:
  - app-network
depends_on:
  backend:
    condition: service_healthy
healthcheck:
  test: ["CMD", "wget", "--spider", "-q", "http://localhost:5678/healthz"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 20s

# -----
# Frontend — React + TypeScript dashboard
# -----
frontend:
  build:
    context: ./frontend
    args:
      # Vite inlines VITE_* vars into the static bundle at build time —
      # passing this at container-run time (e.g. via env_file) would be a
      # no-op against the already-compiled JS, so it has to arrive here.

```

```

VITE_API_BASE_URL: ${VITE_API_BASE_URL}
# Reuses BACKEND_API_KEY: the Frontend is a user-facing client of
# the Backend just like n8n is, so it needs the same secret. See
# docs/security.md for what this does and doesn't protect against
# once it's baked into a public JS bundle.
VITE_API_KEY: ${BACKEND_API_KEY}
container_name: ${COMPOSE_PROJECT_NAME:-vulnscan}-frontend
restart: unless-stopped
# nginx's master process needs root to bind port 80 and to set up its
# cache dirs/worker privilege drop on startup — cap_drop/cap_add here
# is the lighter-touch equivalent of the non-root Dockerfile treatment
# used for backend/reports, without risking a broken build. Confirmed
# empirically: CAP_CHOWN/SETUID/SETGID are all needed, not just
# NET_BIND_SERVICE — without them the official entrypoint fails at
# `chown("/var/cache/nginx/client_temp"...)` before nginx even starts.
cap_drop:
- ALL
cap_add:
- NET_BIND_SERVICE
- CHOWN
- SETUID
- SETGID
security_opt:
- no-new-privileges:true
ports:
- "${FRONTEND_PORT:-8080}:80"
networks:
- app-network
depends_on:
  backend:
    condition: service_healthy
healthcheck:
  test: ["CMD", "wget", "--spider", "-q", "http://localhost:80/health"]
  interval: 10s
  timeout: 5s
  retries: 5
  start_period: 10s

# -----
# Vulnerable lab targets — segmented network (Módulo 2)
#
# Both apps live exclusively on lab-network: app-network services
# (frontend, n8n, backend, db) have no route to them at all. Only the
# Scanner Service (Módulo 4) bridges both networks. Host ports are still
# published for local browser access. See docs/lab.md for the
# legal/ethical scope of this lab and why the network is not marked
# `internal: true`.
# -----
juice-shop:
  image: bkimminich/juice-shop:latest
  container_name: ${COMPOSE_PROJECT_NAME:-vulnscan}-juice-shop
  restart: unless-stopped
  ports:
  - "${JUICE_SHOP_PORT:-3000}:3000"
  networks:
  - lab-network
  healthcheck:
    # Image is distroless (no shell, no curl/wget): probe with Node's
    # own HTTP client instead of a CMD-SHELL healthcheck.

```

```

test:
[
  "CMD",
  "/nodejs/bin/node",
  "-e",
  "require('http').get('http://127.0.0.1:3000/', r => process.exit(r.statusCode < 500 ? 0 : 1)).on('error', () => process.exit(1))",
]
interval: 10s
timeout: 5s
retries: 10
start_period: 20s

```

```

dvwa:
image: vulnerables/web-dvwa:latest
container_name: ${COMPOSE_PROJECT_NAME:-vulnscan}-dvwa
restart: unless-stopped
ports:
- "${DVWA_PORT:-3001}:80"
networks:
- lab-network
healthcheck:
# Image has no curl/wget either; a bare TCP probe on Apache's port
# is enough here since dvwa-init does its own HTTP-level readiness
# check (and retries) before touching the database. /dev/tcp only
# exists in bash, not the image's default /bin/sh (dash) — invoke
# bash explicitly rather than using CMD-SHELL.
test: ["CMD", "bash", "-c", "exec 3<>/dev/tcp/127.0.0.1/80"]
interval: 5s
timeout: 5s
retries: 10
start_period: 15s

```

```

# One-shot: replays the setup.php "Create / Reset Database" submission so
# DVWA is fully usable without a manual browser step. See
# lab/dvwa-init/entrypoint.sh.

```

```

dvwa-init:
build:
context: ./lab/dvwa-init
container_name: ${COMPOSE_PROJECT_NAME:-vulnscan}-dvwa-init
environment:
DVWA_URL: http://dvwa:80
networks:
- lab-network
depends_on:
dvwa:
condition: service_healthy
restart: "no"

```

Anexo G — Extracto del Workflow JSON de n8n

El archivo completo (n8n/workflows/vulnscan-pipeline.json, 42 nodos, 1.486 líneas) es el workflow real exportado desde n8n y referenciado en el Anexo D; no se reproduce íntegro aquí por su extensión y por contener metadatos de UI (coordenadas de posición, identificadores internos) sin valor de lectura. El siguiente extracto muestra dos nodos reales consecutivos del pipeline (“Scan: Nmap” e “Ingest: Nmap”, pasos 3 y 4 del Anexo D), suficientes para verificar que cada etapa es efectivamente una llamada HTTP real hacia los servicios propios, con autenticación por header, tal como describe la Sección 11.6:

```
[
```

```

{
  "parameters": {
    "method": "POST",
    "url": "http://scanner:8100/scan/nmap",
    "sendBody": true,
    "specifyBody": "json",
    "jsonBody": "={{ { \"target\": $json.host } }}",
    "options": {
      "timeout": 60000
    },
    "sendHeaders": true,
    "headerParameters": {
      "parameters": [
        {
          "name": "X-Internal-Token",
          "value": "={{ $env.INTERNAL_API_KEY }}"
        }
      ]
    }
  },
  "id": "5326e457-5919-4f2e-bca3-98977aa3c2e5",
  "name": "Scan: Nmap",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 4.2,
  "position": [
    5168,
    1376
  ],
  "continueOnFail": true
},
{
  "parameters": {
    "method": "POST",
    "url": "http://backend:8000/scans/{{ $('Resolve Pipeline Context').item.json.scan_id }}/tasks",
    "sendBody": true,
    "specifyBody": "json",
    "jsonBody": "={{ $json }}",
    "options": {
      "timeout": 30000
    },
    "sendHeaders": true,
    "headerParameters": {
      "parameters": [
        {
          "name": "X-N8N-Callback-Key",
          "value": "={{ $env.N8N_CALLBACK_API_KEY }}"
        }
      ]
    }
  },
  "id": "e6afbe12-38d4-4a59-95e3-f3723f7b56d0",
  "name": "Ingest: Nmap",
  "type": "n8n-nodes-base.httpRequest",
  "typeVersion": 4.2,
  "position": [
    5328,
    1376
  ],
  "continueOnFail": true
}

```

Anexo H — Salida Cruda de una Herramienta de Escaneo

```
{
  "tool": "whatweb",
  "target": "dvwa",
  "command": "whatweb --log-json=- -q --color=never -a 3 http://dvwa:80",
  "status": "completed",
  "started_at": "2026-08-15T21:09:49.099906Z",
  "finished_at": "2026-08-15T21:10:07.673710Z",
  "raw_output": "[\n{\n\"target\": \"http://dvwa:80\", \"http_status\": 302, \"request_config\": {\n\"headers\": {\n\"User-Agent\": \"WhatWeb/0.5.5\"},\n\"plugins\": {\n\"Apache\": {\n\"version\": [\"2.4.25\"],\n\"Cookies\": {\n\"string\": [\"PHPSESSID\", \"security\"]},\n\"Country\": {\n\"string\": [\"RESERVED\"],\n\"module\": [\"ZZ\"]},\n\"HTTPServer\": {\n\"os\": [\"Debian Linux\"],\n\"string\": [\"Apache/2.4.25 (Debian)\"]},\n\"IP\": {\n\"string\": [\"172.19.0.3\"]},\n\"RedirectLocation\": {\n\"string\": [\"login.php\"]}}}\n,\n{\n\"target\": \"http://dvwa/login.php\", \"http_status\": 200, \"request_config\": {\n\"headers\": {\n\"User-Agent\": \"WhatWeb/0.5.5\"},\n\"plugins\": {\n\"Apache\": {\n\"version\": [\"2.4.25\"],\n\"Cookies\": {\n\"string\": [\"PHPSESSID\", \"security\"]},\n\"Country\": {\n\"string\": [\"RESERVED\"],\n\"module\": [\"ZZ\"]},\n\"DVWA\": {},\n\"HTTPServer\": {\n\"os\": [\"Debian Linux\"],\n\"string\": [\"Apache/2.4.25 (Debian)\"]},\n\"IP\": {\n\"string\": [\"172.19.0.3\"]},\n\"PasswordField\": {\n\"string\": [\"password\"]},\n\"Title\": {\n\"string\": [\"Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*\"]}}}\n]\n\",
  \"parsed\": [
    {
      \"target\": \"http://dvwa:80\",
      \"http_status\": 302,
      \"request_config\": {
        \"headers\": {
          \"User-Agent\": \"WhatWeb/0.5.5\"
        }
      },
      \"plugins\": {
        \"Apache\": {
          \"version\": [
            \"2.4.25\"
          ]
        },
        \"Cookies\": {
          \"string\": [
            \"PHPSESSID\",
            \"security\"
          ]
        },
        \"Country\": {
          \"string\": [
            \"RESERVED\"
          ],
          \"module\": [
            \"ZZ\"
          ]
        },
        \"HTTPServer\": {
          \"os\": [
            \"Debian Linux\"
          ]
        }
      }
    }
  ]
}
```

```

    ],
    "string": [
        "Apache/2.4.25 (Debian)"
    ]
},
"IP": {
    "string": [
        "172.19.0.3"
    ]
},
"RedirectLocation": {
    "string": [
        "login.php"
    ]
}
},
{
    "target": "http://dvwa/login.php",
    "http_status": 200,
    "request_config": {
        "headers": {
            "User-Agent": "WhatWeb/0.5.5"
        }
    },
    "plugins": {
        "Apache": {
            "version": [
                "2.4.25"
            ]
        },
        "Cookies": {
            "string": [
                "PHPSESSID",
                "security"
            ]
        },
        "Country": {
            "string": [
                "RESERVED"
            ],
            "module": [
                "ZZ"
            ]
        },
        "DVWA": {},
        "HTTPServer": {
            "os": [
                "Debian Linux"
            ],
            "string": [
                "Apache/2.4.25 (Debian)"
            ]
        },
        "IP": {
            "string": [
                "172.19.0.3"
            ]
        },
    },

```

```

    "PasswordField": {
      "string": [
        "password"
      ]
    },
    "Title": {
      "string": [
        "Login :: Damn Vulnerable Web Application (DVWA) v1.10 *Development*"
      ]
    }
  }
}
],
"error_message": null
}

```

Anexo I — Informes de Ejemplo Generados

Los formatos PDF y HTML ya se muestran en las Figuras 10 y 11 (abiertos en sus visores respectivos). A continuación, extractos reales de los otros dos formatos soportados, generados para el mismo escaneo (d18d49ad-bc52-4d47-96a4-e96985b184d5, target juice-shop-demo, 2026-08-15):

Formato Markdown (primeras líneas):

VulnScan Platform — Reporte de Escaneo

```

**Target:** juice-shop-demo (juice-shop)
**Scan ID:** d18d49ad-bc52-4d47-96a4-e96985b184d5
**Estado:** completed

```

Resumen por severidad

```

| Severidad | Cantidad |
| --- | --- |
| MEDIUM | 3 |
| LOW | 9 |
| INFO | 15 |

```

Hallazgos

[MEDIUM] Cross-Domain Misconfiguration

- **Tipo:** web_vulnerability

- **Descripción:** Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS) misconfiguration on the web server.

[... continúa con el resto de los hallazgos ...]

Formato JSON (encabezado y primer hallazgo):

```

{
  "target": {
    "id": "9bd7313a-55b9-44c1-8649-ac875519d08b",
    "name": "juice-shop-demo",
    "host": "juice-shop",
    "description": "Demo target for manual pipeline testing"
  },
  "scan": {
    "id": "d18d49ad-bc52-4d47-96a4-e96985b184d5",
    "status": "completed",
    "triggered_by": "n8n-manual-trigger",
    "started_at": "2026-08-14T04:10:41.141682Z",
    "finished_at": "2026-08-14T04:16:55.086074Z"
  }
}

```

```

},
"severity_counts": {
  "medium": 3,
  "info": 15,
  "low": 9
},
"findings": [
  {
    "id": "b1493b0f-0e37-4b96-b784-90a067bfa7a0",
    "title": "Cross-Domain Misconfiguration",
    "description": "<p>Web browser data loading may be possible, due to a Cross Origin Resource Sharing (CORS)
misconfiguration on the web server.</p>",
    "finding_type": "web_vulnerability",
    "evidence": "<p>Ensure that sensitive data is not available in an unauthenticated manner (using IP address white-listing,
for instance).</p><p>Configure the \"Access-Control-Allow-Origin\" HTTP header to a more restrictive set of domains, or
remove all CORS headers entirely, to allow the web browser to enforce the Same Origin Policy (SOP) in a more restrictive
manner.</p>",
    "confidence": "2",
    "cvss_score": null,
    "cvss_vector": null,
    "severity": "medium",
    "created_at": "2026-08-14T04:16:55.026589Z",
    "cve_references": []
  },
  "... (resto de los hallazgos omitido)"
]
}

```

Anexo J — .env.example Completo

Plantilla real de variables de entorno del repositorio (mención original en la Sección 11.1): declara toda la configuración que el operador debe proveer para levantar el sistema, incluida la whitelist de hosts de laboratorio (BACKEND_ALLOWED_LAB_HOSTS / SCANNER_ALLOWED_LAB_HOSTS) que sostiene las garantías de aislamiento descritas en la Sección 10.2.

```

# =====
# VULNSCAN PLATFORM — Environment configuration
# Copy this file to ".env" and adjust values if needed.
# Every service reads its configuration exclusively from environment
# variables so that `docker compose up -d` requires no manual setup.
# =====

# -----
# General
# -----
# Project namespace used for container names / Docker network prefixes.
COMPOSE_PROJECT_NAME=vulnscan

# Deployment environment: local | ci
ENVIRONMENT=local

# -----
# PostgreSQL (database service)
# -----
POSTGRES_HOST=db
POSTGRES_PORT=5432
POSTGRES_DB=vulnscan
POSTGRES_USER=vulnscan

```

```

POSTGRES_PASSWORD=change_me_local_dev_only

# Full SQLAlchemy-style DSN, consumed by backend/database services.
DATABASE_URL=postgresql+psycopg://${POSTGRES_USER}:${POSTGRES_PASSWORD}@${POSTGRES_HOST}:${POSTGRES_PORT}/${POSTGRES_DB}

# -----
# Backend API
# -----
BACKEND_PORT=8000
BACKEND_LOG_LEVEL=info
# Comma-separated origins allowed to call the API (frontend dev/prod URLs).
BACKEND_CORS_ORIGINS=http://localhost:5173,http://localhost:8080
# Whitelist enforced when registering targets: only these hostnames
# (resolvable on lab-network) can ever be scanned. Add a host here when a
# new lab target is added in lab/ (Módulo 2).
BACKEND_ALLOWED_LAB_HOSTS=juice-shop,dvwa
# Shared secret required as the X-API-Key header on every Backend request
# except /health. The Frontend and n8n both send this. See docs/security.md
# for what this single-shared-key model does and doesn't protect against.
BACKEND_API_KEY=change_me_local_dev_only

# -----
# Internal service-to-service secret
# Required as the X-Internal-Token header by Scanner and Reports, which
# never talk to the Frontend directly and no longer publish their ports to
# the host — only the Backend and n8n (both on app-network) hold this.
# -----
INTERNAL_API_KEY=change_me_local_dev_only

# -----
# Scanner Service
# -----
SCANNER_PORT=8100
# Hard upper bound (seconds) for any single tool execution, enforced by the
# scanner service regardless of what the caller requests.
SCANNER_MAX_TIMEOUT_SECONDS=900
# Second, independent copy of the lab whitelist (see BACKEND_ALLOWED_LAB_HOSTS
# above) — checked on every /scan/{tool} call, not just at target
# registration. Keep this equal to BACKEND_ALLOWED_LAB_HOSTS; docker-compose.yml
# sources both from the same variable so they can't drift apart.
SCANNER_ALLOWED_LAB_HOSTS=juice-shop,dvwa

# -----
# Reports Service
# -----
REPORTS_PORT=8200
REPORTS_OUTPUT_DIR=/data/reports
REPORTS_BASE_URL=http://reports:8200

# -----
# n8n
# -----
N8N_PORT=5678
N8N_HOST=localhost
N8N_PROTOCOL=http
N8N_ENCRYPTION_KEY=change_me_local_dev_only
N8N_BASIC_AUTH_ACTIVE=true
N8N_BASIC_AUTH_USER=admin

```

```

N8N_BASIC_AUTH_PASSWORD=change_me_local_dev_only
# Base URL the backend uses to reach n8n's webhook endpoints.
N8N_WEBHOOK_BASE_URL=http://n8n:5678
# Shared secret the Backend sends as X-Webhook-Secret and the workflow's
# "Check Webhook Secret" node validates before running the pipeline.
N8N_WEBHOOK_SECRET=change_me_local_dev_only
# Distinct from BACKEND_API_KEY on purpose: only n8n's own callback nodes
# use it, on POST /scans/{id}/tasks and POST /scans/{id}/complete.
N8N_CALLBACK_API_KEY=change_me_local_dev_only

# -----
# Frontend
# -----
FRONTEND_PORT=8080
# Base URL the browser uses to reach the backend API.
VITE_API_BASE_URL=http://localhost:8000

# -----
# Lab targets (vulnerable applications — local only, isolated network)
# -----
JUICE_SHOP_PORT=3000
DVWA_PORT=3001

```

Anexo K — Protocolo del Brazo Manual

Este anexo documenta el procedimiento ejecutado en la Fase 2 del protocolo experimental (Sección 9.6). Se incorpora para que la condición de control sea replicable por un tercero bajo las mismas condiciones que la condición automatizada, y para que cualquier repetición futura de esta medición —con mayor tamaño muestral o un operador sin el sesgo de familiaridad declarado en la Sección 9.9— pueda seguir exactamente el mismo protocolo.

Las invocaciones que siguen son las mismas que construyen los adaptadores del Servicio de Escaneo (Anexo B), de modo que el operador manual ejecuta exactamente las mismas herramientas con los mismos parámetros que el pipeline. Cada etapa se cronometra por separado y se registran dos tiempos: el tiempo atendido, durante el cual el operador ejecuta alguna acción, y el tiempo total de reloj, que incluye la espera. La distinción es necesaria porque la Sección 12.3 establece que la ejecución de las herramientas constituye un piso común a ambas condiciones: lo que la automatización elimina es el tiempo atendido.

Etapas 1 — Nmap. Descubrimiento de servicios sobre el objetivo y lectura de la salida XML para identificar el puerto con servicio HTTP, que es la entrada de las etapas siguientes.

```
nmap -sV -T4 -oX - dvwa
```

Etapas 2 — WhatWeb. Identificación de las tecnologías del servidor e interpretación del fingerprint devuelto en JSON.

```
whatweb http://dvwa:80 --log-json=- -q --color=never -a 3
```

Etapas 3 — Nikto. Detección de configuraciones inseguras, con espera hasta que la herramienta termine —por cuenta propia o por agotamiento de su presupuesto de tiempo, según la versión: en la campaña A lo agotaba y en la C terminó en 41,7 s— y lectura del informe JSON.

```
perl /opt/nikto/program/nikto.pl -h http://dvwa:80 -Format json -output nikto.json -maxtime 120s
```

Etapla 4 — Nuclei. Escaneo por plantillas y lectura de la salida en JSON Lines, un hallazgo por línea, con su severidad declarada.

```
nuclei -u http://dvwa:80 -jsonl -silent -duc
```

Etapla 5 — OWASP ZAP. Escaneo activo y lectura del informe de alertas, cada una con su riskdesc.

```
zap.sh -cmd -quickurl http://dvwa:80 -quickout zap.json
```

Etapla 6 — Consolidación y clasificación de severidad. Sin asistencia del sistema, el operador reúne los hallazgos de las tres herramientas de detección en una lista única, identifica y unifica los duplicados entre herramientas, y traduce las tres escalas de severidad a la escala común de cinco niveles. Debe registrarse el número de hallazgos resultante tras la deduplicación.

Etapla 7 — Elaboración del informe. Redacción de un informe con objetivo analizado, servicios y tecnologías detectados, tabla de hallazgos ordenada por severidad y resumen ejecutivo, equivalente en contenido al que emite el Servicio de Reportes.

El registro de la corrida debe consignar, además de los tiempos por etapa, la fecha, el operador, la máquina anfitriona y las versiones de herramienta vigentes, dado que la Sección 12.6.3 documenta que estas últimas afectan materialmente al recuento de hallazgos. Al tratarse de dos corridas ejecutadas por los propios autores, el resultado debe reportarse como un par de observaciones y no como comparación estadística, y declarando el sesgo de familiaridad del operador que la Sección 9.9 describe.

Hoja de registro de tiempos. El operador registra, para cada una de las siete etapas, dos valores: el tiempo atendido —aquel en que ejecuta una acción: tipear, leer, decidir o transcribir — y el tiempo total de reloj, que incluye la espera a que la herramienta termine. La distinción es necesaria porque el tiempo de ejecución de las herramientas es un piso común a las dos condiciones y no es lo que la automatización elimina (Sección 9.9). Cada etapa consigna además el dato producido que alimenta la etapa siguiente: el puerto HTTP identificado, las tecnologías detectadas y el número de hallazgos anotados. El cierre de la hoja recoge la suma de ambas columnas, la fecha y hora de inicio, el operador, el objetivo, la máquina anfitriona y las versiones de herramienta. Las dos corridas reportadas en la Tabla 10 se registraron con esta hoja; su transcripción se publica en el repositorio declarado en la Sección 20.

Plantilla del informe manual. El informe que el operador redacta en la Etapa 7 reproduce la estructura que emite el Servicio de Reportes, para que ambas condiciones produzcan un entregable equivalente y el tiempo medido sea comparable: identificación del objetivo analizado y fecha de la evaluación; inventario de servicios y tecnologías detectados; tabla de hallazgos ordenada por severidad, con herramienta de origen, título, descripción y evidencia de cada uno; recuento por nivel de severidad; y un resumen ejecutivo. Toda entrada del informe manual se construye a partir de las salidas crudas de las herramientas, sin consultar resultados de ejecuciones anteriores del pipeline.